

**Heaven's Light is Our Guide**

**RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY**



**A THESIS REPORT**

**ON**

**AI-Enhanced Stereo Matching for High-Accuracy  
Depth Mapping and 3D Reconstruction**

**PREPARED**

**BY**

**MD. RAIHANUL HAQUE RAHI (Roll No.: 2008011)**

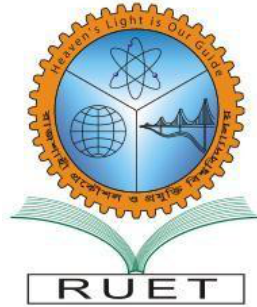
**NAZIB ABRAR (Roll No.: 2008026)**

**DEPARTMENT OF MECHATRONICS ENGINEERING**

**JULY 2026**

Heaven's light is our guide

Rajshahi University of Engineering & Technology, Bangladesh



Department of Mechatronics Engineering

A thesis report on

**AI-Enhanced Stereo Matching for High-Accuracy  
Depth Mapping and 3D Reconstruction**

This report is submitted in partial fulfilment of the requirements for the degree of  
Bachelor of Science in Mechatronics Engineering

**Supervised by**

Md Zunaid Hossen  
Lecturer  
Department of Mechatronics  
Engineering  
RUET, Rajshahi

**Submitted by**

Md. Raihanul Haque Rahi  
Roll No: 2008011  
  
Nazib Abrar  
Roll No: 2008026

July 2026

Heaven's Light is Our Guide  
**RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY**  
Department of Mechatronics Engineering



**CERTIFICATE**

This is to certify that the thesis work “**AI-Enhanced Stereo Matching for High-Accuracy Depth Mapping and 3D Reconstruction**” by **Md. Raihanul Haque Rahi**, bearing Roll No. **2008011** & **Nazib Abrar**, bearing Roll No. **2008026** has been carried out under my supervision in partial fulfillment of the requirements for the degree of Bachelor of Science in Mechatronics Engineering.

**Signature of the Supervisor**

**Signature of the Students**

.....  
**(Md Zunaid Hossen)**  
Lecturer  
Department of Mechatronics Engineering  
Rajshahi University of Engineering &  
Technology

.....  
**(Md. Raihanul Haque Rahi)**  
Roll No: 2008011  
  
.....  
**(Nazib Abrar)**  
Roll No: 2008026

**Countersigned**

**Signature of the External Examiner**

.....  
**(Md. Firoj Ali)**  
Associate Professor and Head  
Department of Mechatronics Engineering  
Rajshahi University of Engineering &  
Technology

.....  
**(Md. Hafiz Ahamed)**  
Assistant Professor  
Department of Mechatronics Engineering  
Rajshahi University of Engineering &  
Technology

## **Declaration**

We, Md. Raihanul Haque Rahi and Nazib Abrar, affirm with full responsibility that the thesis entitled “AI-Enhanced Stereo Matching for High-Accuracy Depth Mapping and 3D Reconstruction” is solely authored by us, and that the work presented herein is our own except where otherwise stated. We gratefully acknowledge the guidance provided by Md Zunaid Hossen, Lecturer, Department of Mechatronics Engineering, Rajshahi University of Engineering & Technology.

We further declare that this thesis does not, to the best of our knowledge, infringe upon any existing copyright, and that all external work, published or unpublished, has been duly acknowledged and cited.

This thesis has not been submitted previously, in whole or in part, for any other degree or diploma at this or any other institution.

### **Authors**

Md. Raihanul Haque Rahi  
Roll No: 2008011

Nazib Abrar  
Roll No: 2008026

## Acknowledgement

First of all, we express our deepest gratitude to Almighty Allah, the Most Gracious and the Most Merciful, for blessing us with the strength, patience, and determination required to complete this thesis.

We would like to express our heartfelt gratitude and sincere appreciation to our supervisor, Md. Zunaid Hossen, Lecturer, Department of Mechatronics Engineering, RUET. His constructive guidance, regular feedback, encouragement, and careful review of our work shaped the direction of this research. His advice was invaluable during the design of the network, the experimental evaluation, and the preparation of this thesis book. We are deeply grateful for the time and effort he devoted to supporting us throughout the project.

We sincerely thank the honorable external examiner for reviewing this work and for providing valuable observations and feedback. We are also grateful to the Head of the Department of Mechatronics Engineering, RUET, for providing direction and for supporting the academic environment in which this research was completed.

We would like to thank all faculty members and office staff of the Department of Mechatronics Engineering for their assistance, guidance, and cooperation. We are especially grateful for the laboratory access, computational facilities, and administrative support that helped us complete the data collection, software development, experiments, and thesis documentation. We also thank our classmates, seniors, juniors, and well-wishers for their technical suggestions, encouragement, and support throughout this journey.

Finally, we express our deepest appreciation to our parents and family members for their unconditional love, patience, prayers, and constant encouragement. Their support gave us the confidence to continue through the demanding stages of this work and to bring the thesis to completion.

RUET, Rajshahi  
July 2026

**Authors**  
Md. Raihanul Haque Rahi  
Roll No: 2008011

Nazib Abrar  
Roll No: 2008026

## Abstract

Mobile robots, drones, and augmented-reality devices need dense depth in real time, and they must compute it within a power budget of only a few watts. Binocular stereo matching supplies this depth passively from two ordinary cameras, but the most accurate stereo networks reach their accuracy with foundation-scale backbones of hundreds of millions of parameters, a computational cost embedded hardware cannot pay. Closing that gap is the central problem this thesis addresses, together with a practical second constraint: the compact stereo rigs these platforms carry are rarely perfectly rectified, so a network built for them must also tolerate small camera misalignments rather than assume the ideal geometry lab benchmarks provide. This thesis presents StereoLite, a 2.96 million parameter stereo matching network designed to meet both constraints. Rather than searching a full four-dimensional cost volume the way heavier, more accurate networks do, StereoLite estimates disparity for each small image tile and repeatedly corrects that estimate with a learned recurrent update, an approach that keeps computation and memory small while recovering most of the accuracy a full search would give. Concretely, a group-wise correlation volume at one sixteenth resolution initializes each tile’s disparity and confidence; a dedicated left-image context encoder then feeds convolutional GRU updates that refine every tile across three resolutions; a narrow-band geometry encoding volume adds a long-range correction through a learned gate that acts only where it helps; and plane-equation rendering with learned convex upsampling recovers a sharp, full-resolution disparity map from the coarse tile grid. Trained only on synthetic SceneFlow data, the network reaches 0.78 px end-point error and 3.40 percent D1-all on the full FlyingThings3D test set of 4,370 pairs. Without ever training on real images, it scores 10.9 percent D1-all zero-shot on the unseen Middlebury 2014 benchmark, showing that its accuracy transfers to a real domain it never saw during training. One forward pass at  $384 \times 640$  takes 49.8 ms on a laptop RTX 3050 GPU in half precision; after an inference-optimization pass and eight-bit quantization, it runs in 36.3 ms, or 27.5 frames per second, on the target Jetson Orin Nano, meeting the real-time target on embedded hardware. The network also tolerates the imperfect rectification a low-cost stereo rig produces: under a controlled vertical misalignment sweep, end-point error rises only from 1.03 to 1.53 px across the sub-pixel to one-pixel range such a rig produces, and on real indoor scenes captured with the project’s own rig, the predicted disparity back-projects through the calibrated geometry into metric colored point clouds. The training code, the evaluation drivers, and the trained checkpoint are archived so that every reported number can be reproduced.

# List of Contents

|                                                                          |            |
|--------------------------------------------------------------------------|------------|
| <b>Declaration</b>                                                       | <b>i</b>   |
| <b>Acknowledgement</b>                                                   | <b>ii</b>  |
| <b>Abstract</b>                                                          | <b>iii</b> |
| <b>List of Figures</b>                                                   | <b>vii</b> |
| <b>List of Tables</b>                                                    | <b>ix</b>  |
| <b>List of Abbreviations</b>                                             | <b>x</b>   |
| <b>1 Introduction</b>                                                    | <b>1</b>   |
| 1.1 Overview . . . . .                                                   | 1          |
| 1.2 Background and Motivation . . . . .                                  | 1          |
| 1.3 Research Questions / Problem Statements . . . . .                    | 3          |
| 1.4 Research Objectives . . . . .                                        | 3          |
| 1.5 Scope of the Research . . . . .                                      | 4          |
| 1.6 Key Contributions . . . . .                                          | 5          |
| 1.7 Organization of the Report . . . . .                                 | 6          |
| <b>2 Literature Review</b>                                               | <b>8</b>   |
| 2.1 Introduction . . . . .                                               | 8          |
| 2.2 Existing Research . . . . .                                          | 8          |
| 2.2.1 Stereo Matching Fundamentals . . . . .                             | 8          |
| 2.2.2 The Deep Learning Transition . . . . .                             | 13         |
| 2.2.3 The 3D Cost-Volume Era . . . . .                                   | 14         |
| 2.2.4 Iterative Refinement . . . . .                                     | 15         |
| 2.2.5 Foundation-Model Era . . . . .                                     | 17         |
| 2.2.6 Requirements of Edge-Deployable Stereo Matching . . . . .          | 19         |
| 2.2.7 Classification of Efficient Stereo Methods . . . . .               | 19         |
| 2.2.8 Backbone Substitution . . . . .                                    | 21         |
| 2.2.9 Cost-Volume Compression . . . . .                                  | 21         |
| 2.2.10 Iterative-Loop Compression . . . . .                              | 21         |
| 2.2.11 Knowledge Distillation . . . . .                                  | 22         |
| 2.2.12 Architectural Compression and Adaptive Compute . . . . .          | 23         |
| 2.3 Research Gap . . . . .                                               | 24         |
| 2.4 Comparative Analysis . . . . .                                       | 25         |
| 2.4.1 Cross-Domain Generalization . . . . .                              | 25         |
| 2.4.2 Benchmarks and Datasets . . . . .                                  | 26         |
| 2.4.3 Accuracy, Parameters, and Latency Across Method Families . . . . . | 27         |
| 2.5 Summary . . . . .                                                    | 28         |

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| <b>3</b> | <b>Engineering Design and Methodology</b>                   | <b>30</b> |
| 3.1      | Introduction . . . . .                                      | 30        |
| 3.2      | Research Methodology . . . . .                              | 30        |
| 3.3      | Design Requirements and Requisites . . . . .                | 31        |
| 3.4      | Design Criteria . . . . .                                   | 32        |
| 3.5      | Design Steps . . . . .                                      | 32        |
| 3.6      | Modeling of Proposed System . . . . .                       | 32        |
| 3.7      | Modeling of Conceptual Design . . . . .                     | 34        |
| 3.8      | Preliminary Design . . . . .                                | 35        |
| 3.9      | Detailed Design . . . . .                                   | 36        |
| 3.9.1    | Shared Feature Encoder and Context Encoder . . . . .        | 36        |
| 3.9.2    | Tile Initialization and Cost Volume . . . . .               | 38        |
| 3.9.3    | Recurrent Tile Refinement . . . . .                         | 38        |
| 3.9.4    | Plane-Aware Cross-Scale Propagation . . . . .               | 40        |
| 3.9.5    | Geometry Encoding Volume and Fail-Soft Fusion . . . . .     | 41        |
| 3.9.6    | Plane Rendering and Learned Convex Upsampling . . . . .     | 42        |
| 3.9.7    | From Disparity to 3D Reconstruction . . . . .               | 43        |
| 3.10     | Design Assessment and Performance Indices . . . . .         | 44        |
| 3.11     | Summary . . . . .                                           | 45        |
| <b>4</b> | <b>System Development and Implementation</b>                | <b>46</b> |
| 4.1      | Introduction . . . . .                                      | 46        |
| 4.2      | Software Development . . . . .                              | 46        |
| 4.3      | Network Implementation . . . . .                            | 47        |
| 4.4      | Optimization and Configuration Selection . . . . .          | 47        |
| 4.5      | Training Objective and Protocol . . . . .                   | 49        |
| 4.5.1    | Training Objective . . . . .                                | 49        |
| 4.5.2    | Datasets and Training Protocol . . . . .                    | 50        |
| 4.6      | Hardware and Embedded Deployment . . . . .                  | 52        |
| 4.7      | Principle of Operation . . . . .                            | 52        |
| 4.8      | Summary . . . . .                                           | 53        |
| <b>5</b> | <b>Experimental Setup and Data Collection</b>               | <b>55</b> |
| 5.1      | Introduction . . . . .                                      | 55        |
| 5.2      | Experimental Setup and Test Bench . . . . .                 | 55        |
| 5.3      | Calibration and Rectification . . . . .                     | 56        |
| 5.4      | Datasets . . . . .                                          | 58        |
| 5.4.1    | Synthetic Training and Test Corpus: SceneFlow . . . . .     | 58        |
| 5.4.2    | Zero-Shot Cross-Domain Benchmark: Middlebury 2014 . . . . . | 58        |
| 5.4.3    | Physical Stereo Rig Captures . . . . .                      | 59        |
| 5.5      | Data Collection . . . . .                                   | 59        |
| 5.6      | Image Preprocessing Pipeline . . . . .                      | 59        |
| 5.7      | Experimental Procedure and Evaluation Metrics . . . . .     | 61        |
| 5.8      | Summary . . . . .                                           | 62        |

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>6</b> | <b>Results and Discussion</b>                                           | <b>64</b> |
| 6.1      | Introduction . . . . .                                                  | 64        |
| 6.2      | Experimental Results . . . . .                                          | 64        |
| 6.2.1    | Training Dynamics . . . . .                                             | 64        |
| 6.2.2    | In-Domain Accuracy: SceneFlow Test . . . . .                            | 65        |
| 6.3      | Comparison . . . . .                                                    | 66        |
| 6.3.1    | Comparison with Published Methods on SceneFlow . . . . .                | 66        |
| 6.3.2    | Zero-Shot Cross-Domain: Middlebury 2014 . . . . .                       | 67        |
| 6.4      | Error Analysis . . . . .                                                | 68        |
| 6.4.1    | Robustness to Rectification Imperfection . . . . .                      | 68        |
| 6.4.2    | Qualitative Results . . . . .                                           | 69        |
| 6.4.3    | 3D Reconstruction of Real Scenes . . . . .                              | 70        |
| 6.4.4    | Comparison with LiDAR Point Clouds . . . . .                            | 71        |
| 6.5      | Validation . . . . .                                                    | 72        |
| 6.5.1    | Design-Space Exploration of the Tile Chassis . . . . .                  | 72        |
| 6.5.2    | Architecture Ablation of the Recurrent Family . . . . .                 | 73        |
| 6.5.3    | Training-Recipe and Efficiency Studies . . . . .                        | 74        |
| 6.6      | Performance Evaluation . . . . .                                        | 76        |
| 6.6.1    | Efficiency and Deployment Measurements . . . . .                        | 76        |
| 6.7      | Discussion . . . . .                                                    | 76        |
| 6.8      | Summary . . . . .                                                       | 77        |
| <b>7</b> | <b>Societal, Environmental Impact and Sustainability</b>                | <b>78</b> |
| 7.1      | Introduction . . . . .                                                  | 78        |
| 7.2      | Impact of the Thesis on Societal, Health, and Cultural Issues . . . . . | 78        |
| 7.3      | Safety and Ethical Considerations . . . . .                             | 79        |
| 7.4      | Alignment with Sustainable Development Goals (SDGs) . . . . .           | 80        |
| 7.5      | Summary . . . . .                                                       | 81        |
| <b>8</b> | <b>Thesis Management and Finance</b>                                    | <b>82</b> |
| 8.1      | Introduction . . . . .                                                  | 82        |
| 8.2      | Management of the Thesis . . . . .                                      | 82        |
| 8.2.1    | Time Frame / Gantt Chart . . . . .                                      | 83        |
| 8.3      | Overall Budget . . . . .                                                | 84        |
| 8.3.1    | Component and Software Budget . . . . .                                 | 84        |
| 8.4      | Resource Planning . . . . .                                             | 85        |
| 8.5      | Individual and Team Work Contribution . . . . .                         | 86        |
| 8.6      | Summary . . . . .                                                       | 86        |
| <b>9</b> | <b>Conclusions, Limitations and Future Works</b>                        | <b>88</b> |
| 9.1      | Conclusion . . . . .                                                    | 88        |
| 9.2      | Limitations . . . . .                                                   | 88        |
| 9.3      | Future Works . . . . .                                                  | 89        |
|          | <b>References</b>                                                       | <b>90</b> |
|          | <b>Appendix A List of Modern Tools Usage</b>                            | <b>95</b> |
|          | <b>Appendix B Program Outcome Mapping</b>                               | <b>96</b> |

|                   |                                                         |            |
|-------------------|---------------------------------------------------------|------------|
| <b>Appendix C</b> | <b>Addressing Knowledge Profile (KPs) in the Thesis</b> | <b>97</b>  |
| <b>Appendix D</b> | <b>Addressing Complex Engineering Problems</b>          | <b>99</b>  |
| <b>Appendix E</b> | <b>Addressing Complex Engineering Activities</b>        | <b>101</b> |
| <b>Appendix F</b> | <b>Technical Specifications</b>                         | <b>103</b> |
| F.1               | Model Configuration . . . . .                           | 103        |
| F.2               | Per-Scene Zero-Shot Results . . . . .                   | 104        |
| F.3               | Software and Hardware Environment . . . . .             | 104        |
| F.4               | Reproduction Pointers . . . . .                         | 104        |
| <b>Appendix G</b> | <b>Plagiarism Report</b>                                | <b>106</b> |
| <b>Appendix H</b> | <b>AI Writing Report</b>                                | <b>118</b> |

## List of Figures

|      |                                                                                                                                                                                                                     |    |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1  | Trainable parameter count against SceneFlow accuracy, with the targeted edge budget shaded. . . . .                                                                                                                 | 2  |
| 2.1  | Epipolar geometry constraining stereo correspondence search to one dimension (Fei-Fei Li, CS231A). . . . .                                                                                                          | 9  |
| 2.2  | Stereo rectification reprojecting tilted image planes onto a common plane (Fei-Fei Li, CS231A). . . . .                                                                                                             | 9  |
| 2.3  | Two-view stereo geometry relating disparity to depth. . . . .                                                                                                                                                       | 10 |
| 2.4  | The census transform for illumination-robust matching cost computation. . . . .                                                                                                                                     | 11 |
| 2.5  | The disparity space image, or cost volume, for stereo matching. . . . .                                                                                                                                             | 11 |
| 2.6  | The four-step classical stereo matching pipeline. . . . .                                                                                                                                                           | 12 |
| 2.7  | PSMNet architecture, representing the three-dimensional cost-volume paradigm. . . . .                                                                                                                               | 14 |
| 2.8  | RAFT-Stereo architecture, representing the iterative refinement paradigm. . . . .                                                                                                                                   | 16 |
| 2.9  | FoundationStereo architecture, representing the foundation-model-era paradigm. . . . .                                                                                                                              | 17 |
| 2.10 | Evolution of deep stereo matching, 2016 to 2026. . . . .                                                                                                                                                            | 18 |
| 2.11 | Edge-deployment requirements mapped to compression technique families. . . . .                                                                                                                                      | 20 |
| 2.12 | Taxonomy of compression techniques for deep stereo matching. . . . .                                                                                                                                                | 20 |
| 2.13 | HITNet architecture, representing the tile-hypothesis paradigm. . . . .                                                                                                                                             | 22 |
| 2.14 | LiteAnyStereo architecture, representing the distillation-trained efficient paradigm. . . . .                                                                                                                       | 22 |
| 2.15 | The three-stage knowledge-distillation pipeline of LiteAnyStereo. . . . .                                                                                                                                           | 23 |
| 2.16 | Accuracy against parameters and latency for the surveyed methods: (a) accuracy against trainable parameters. . . . .                                                                                                | 26 |
| 3.1  | Evidence-gated design methodology from literature through ablation, training, and evaluation to edge deployment. . . . .                                                                                            | 31 |
| 3.2  | StereoLite architecture: five numbered stages with supervised outputs marked by red dots. . . . .                                                                                                                   | 33 |
| 3.3  | Disparity as it leaves each stage of the pipeline, on a real Middlebury 2014 pair. . . . .                                                                                                                          | 37 |
| 3.4  | Shared YOLO26s matching encoder and separate left-image context encoder. . . . .                                                                                                                                    | 37 |
| 3.5  | Tile-state initialization: group-wise correlation volume regularized by 3-D CNN with soft-argmax outputs. The 24 hypotheses at 1/16 scale cover roughly 0 to 368 full-resolution pixels. . . . .                    | 39 |
| 3.6  | One recurrent refinement iteration with warping, local correlation lookup, and gated residual updates. . . . .                                                                                                      | 40 |
| 3.7  | Plane-aware cross-scale propagation using local slopes instead of bilinear interpolation. The slopes shift each child off the parent plane, and the factor of two rescales the disparity to the finer grid. . . . . | 41 |
| 3.8  | Narrow-band geometry encoding volume fused into tile estimates via learned fail-soft gate. . . . .                                                                                                                  | 42 |
| 3.9  | Learned convex upsampling via softmax-weighted 3x3 neighborhoods to full resolution. Two 2x stages (1/4 to 1/2 to full) each multiply disparity by two, with non-negative weights that sum to one. . . . .          | 43 |
| 3.10 | Module-wise parameter distribution: encoder and refinement stages hold 95% of 2.96 M parameters. . . . .                                                                                                            | 44 |

|      |                                                                                                                                     |    |
|------|-------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1  | Export pipeline from trained checkpoint to INT8 TensorRT engine. . . . .                                                            | 48 |
| 4.2  | Six disparity scales supervised with L1, gradient consistency, and threshold terms. . . . .                                         | 49 |
| 4.3  | Native-crop sampling (left) vs downscale-then-crop (right) training protocols. . . . .                                              | 51 |
| 4.4  | Training augmentations applied to a native co-located crop. . . . .                                                                 | 52 |
| 5.1  | Jetson Orin Nano edge accelerator, deployment target for inference. . . . .                                                         | 56 |
| 5.2  | Waveshare AR0144 stereo camera module with its 52 mm baseline. . . . .                                                              | 56 |
| 5.3  | Stereo rig capture and rectification geometry for metric depth. . . . .                                                             | 57 |
| 6.1  | Training loss and validation accuracy curves over the training run. . . . .                                                         | 65 |
| 6.2  | Predicted disparity across training checkpoints for two FlyingThings3D scenes. . . . .                                              | 65 |
| 6.3  | Left image, ground truth, prediction, and error map for held-out FlyingThings3D scenes. . . . .                                     | 66 |
| 6.4  | Per-scene zero-shot D1-all on Middlebury 2014, sorted by error. . . . .                                                             | 67 |
| 6.5  | Left image, ground truth, and prediction for the easiest and hardest Middlebury 2014 scenes. . . . .                                | 68 |
| 6.6  | End-point error and D1-all against vertical rectification offset. . . . .                                                           | 69 |
| 6.7  | Left image and predicted disparity for real indoor stereo pairs. . . . .                                                            | 70 |
| 6.8  | Left image and point clouds from three viewpoints for real indoor scenes. . . . .                                                   | 71 |
| 6.9  | Sparse LiDAR point cloud versus our dense stereo point cloud. . . . .                                                               | 72 |
| 6.10 | Ablation results for augmentation, inference optimization, boundary blur fixes, and input protocol. . . . .                         | 75 |
| 6.11 | Middlebury 2014 D1-all against trainable parameters for three zero-shot models. . . . .                                             | 77 |
| 8.1  | Gantt chart of the eleven project phases from July 2025 to July 2026; the main experimental period begins in December 2025. . . . . | 83 |

## List of Tables

|      |                                                                                                                                   |     |
|------|-----------------------------------------------------------------------------------------------------------------------------------|-----|
| 2.1  | Stereo benchmark datasets with train/test splits, resolutions, and ground-truth types. . . . .                                    | 27  |
| 2.2  | Parameters, SceneFlow error, KITTI D1-all, and latency for representative methods (values verified against cited papers). . . . . | 28  |
| 3.1  | Spatial size, channel count, and main operation at each pipeline scale. . . .                                                     | 34  |
| 3.2  | Architectural and training decisions with their supporting evidence. . . . .                                                      | 35  |
| 4.1  | Training configuration of the production run (run 20260704). . . . .                                                              | 47  |
| 4.2  | Inference optimizations and equivalence status measured at $384 \times 640$ on RTX 3050. . . . .                                  | 48  |
| 4.3  | Dataset split used for training, validation, and evaluation. . . . .                                                              | 50  |
| 5.1  | Manufacturer specifications of the AR0144 stereo camera. . . . .                                                                  | 57  |
| 5.2  | Data sources with pair counts, resolution, and ground truth. . . . .                                                              | 58  |
| 5.3  | Preprocessing applied to each data source before the network boundary. . .                                                        | 60  |
| 6.1  | Full FlyingThings3D test-set results across all eight evaluation metrics. . .                                                     | 66  |
| 6.2  | Comparison with published SceneFlow methods (protocol not directly comparable to ours). . . . .                                   | 66  |
| 6.3  | Zero-shot Middlebury 2014 comparison across three models (our reproduction of public checkpoints; T4 fp32 eager latency). . . . . | 67  |
| 6.4  | Accuracy on FlyingThings3D under simulated vertical rectification offset. .                                                       | 69  |
| 6.5  | Encoder ablation on the tile chassis under the matched 20-pair harness. . . .                                                     | 73  |
| 6.6  | Refinement-scheme ablation on the tile chassis under the matched harness. .                                                       | 73  |
| 6.7  | Loss-formulation ablation across nine objectives on the tile chassis. . . . .                                                     | 73  |
| 6.8  | Architecture ablation of the recurrent family that produced the final model.                                                      | 74  |
| 6.9  | Inference-efficiency ablation on the 100-pair harness, 9k steps (RTX 3050 fp32: 106.7 to 61.4 ms, Table 4.2). . . . .             | 75  |
| 6.10 | Boundary-sharpness fixes and their composition on a 500-pair windowed split.                                                      | 75  |
| 6.11 | Input-protocol ablation on the dual-axis evaluation (50 validation frames). .                                                     | 75  |
| 6.12 | Latency and memory across RTX 3050 and Jetson Orin Nano configurations (Jetson memory is estimated). . . . .                      | 76  |
| 8.1  | Eleven project phases from July 2025 to July 2026. . . . .                                                                        | 84  |
| 8.2  | Project expenditure in USD and BDT across hardware and cloud computing.                                                           | 84  |
| 8.3  | Primary CRediT responsibilities of each author. . . . .                                                                           | 86  |
| A.1  | Software tools and libraries used in the thesis. . . . .                                                                          | 95  |
| A.2  | Computing and sensing hardware used in the thesis. . . . .                                                                        | 95  |
| B.1  | Program outcome attainment mapped to thesis sections. . . . .                                                                     | 96  |
| C.1  | Knowledge-profile attainment mapped to thesis chapters. . . . .                                                                   | 97  |
| D.1  | Complex Engineering Problem attributes mapped to thesis sections. . . . .                                                         | 100 |
| F.1  | Complete architecture configuration of the trained checkpoint. . . . .                                                            | 103 |

|     |                                                         |     |
|-----|---------------------------------------------------------|-----|
| F.2 | Per-scene zero-shot results on Middlebury 2014. . . . . | 104 |
|-----|---------------------------------------------------------|-----|

## List of Abbreviations

|              |                                                                   |
|--------------|-------------------------------------------------------------------|
| $d$          | disparity, the horizontal pixel shift between rectified views     |
| $Z$          | depth of a scene point along the camera axis                      |
| $f$          | camera focal length in pixels                                     |
| $B$          | stereo baseline, the distance between the two camera centers      |
| $(c_x, c_y)$ | principal point of the camera                                     |
| EPE          | end-point error, the mean absolute disparity error                |
| bad- $t$     | percentage of valid pixels with error above $t$ pixels            |
| D1-all       | percentage of pixels with error above 3 px and 5% of ground truth |
| GRU          | gated recurrent unit                                              |
| GEV          | geometry encoding volume                                          |
| KD           | knowledge distillation                                            |
| SGM          | semi-global matching                                              |
| FPS          | frames per second                                                 |
| TOPS         | tera-operations per second                                        |
| INT8         | eight-bit integer arithmetic                                      |
| GT           | ground truth                                                      |

# Chapter 1

## Introduction

### 1.1 Overview

A machine that must act in three-dimensional space first needs to know how far away things are. A mobile robot needs it to avoid obstacles, a drone needs it to hold position, and an augmented-reality headset needs it to anchor virtual content to real surfaces. Binocular stereo recovers this depth passively from two inexpensive image sensors, but the dense correspondence it requires is computationally demanding, and the deep networks that now solve it most accurately are far too large and too slow for the embedded accelerators these platforms carry. This thesis designs, trains, and deploys a compact stereo matching network that narrows this gap. The network produces dense disparity, and from it metric three-dimensional reconstruction, within the parameter, latency, and memory budget of edge hardware.

The problem it addresses is a joint one: dense and accurate stereo depth on hardware that supplies only a few tera-operations per second, a few gigabytes of memory, and a few watts of power. This chapter states the background and motivation, the research questions the thesis answers, the objectives, the scope of the work, the key contributions, and the organization of the report.

### 1.2 Background and Motivation

Depth is not an optional cue. It is the quantity on which the safety and usefulness of spatially acting systems depend, and the sensing options that supply it differ sharply in cost and character. Scanning LiDAR measures time of flight along emitted laser beams and returns accurate but sparse points, at a price and power draw that exclude small platforms. Structured-light and time-of-flight cameras emit infrared patterns or pulses and recover dense depth indoors, but their emitters limit range, fail in sunlight, and interfere with one another when several devices share a space. Among the passive ways to recover depth, binocular stereo suits embedded systems because it emits nothing, draws little power, works indoors and outdoors alike, and uses two inexpensive image sensors rather than a laser scanner. Its cost is computational. The correspondence between the two views must be solved for every pixel, and that computation is what this thesis sets out to make affordable.

Binocular stereo recovers depth from the horizontal shift of a scene point between two horizontally separated views. That shift, the disparity, is inversely proportional to depth through a calibrated pinhole relation. A dense disparity map therefore yields a dense depth map, which downstream perception and reconstruction consume directly, and computing that disparity map accurately and quickly is the task this thesis addresses.

The systems considered in this thesis operate under similar constraints: limited computational resources, restricted power availability, and a need for real-time estimates. Their depth requirements, however, are not identical. A mobile robot primarily requires dependable obstacle depth over short and medium ranges, whereas a drone also demands a high update rate because it may approach obstacles more rapidly. In augmented reality, accurate object boundaries are especially important for realistic occlusion between virtual and physical content. A compact sensing platform, by contrast, places greater emphasis on metric accuracy for measurement and three-dimensional reconstruction. An embedded stereo network intended for these settings must therefore achieve a practical compromise among accuracy, boundary fidelity, inference time, and memory use. Establishing this compromise is a central concern of the proposed design.

Deep networks have transformed the accuracy of stereo matching. The most accurate systems now pair a foundation-scale monocular prior with an iterative refinement core; FoundationStereo, for example, reports 0.34 pixel end-point error on the SceneFlow benchmark with a backbone of roughly 340 million parameters [1]. That accuracy comes at a computational cost that embedded hardware cannot pay. The edge accelerators found on mobile robots and drones deliver a few tera-operations per second, carry a few gigabytes of memory, and operate within a power envelope of a few watts to a few tens of watts. A network with hundreds of millions of parameters and hundreds of milliseconds of latency does not fit that budget. The problem this thesis addresses is therefore not accuracy alone but the joint objective of accuracy, latency, and memory: a stereo network that is accurate enough to be useful, small enough to fit in on-device memory, and fast enough to run in real time on an edge accelerator. Figure 1.1 places the published literature on this trade-off and marks the region this thesis targets.

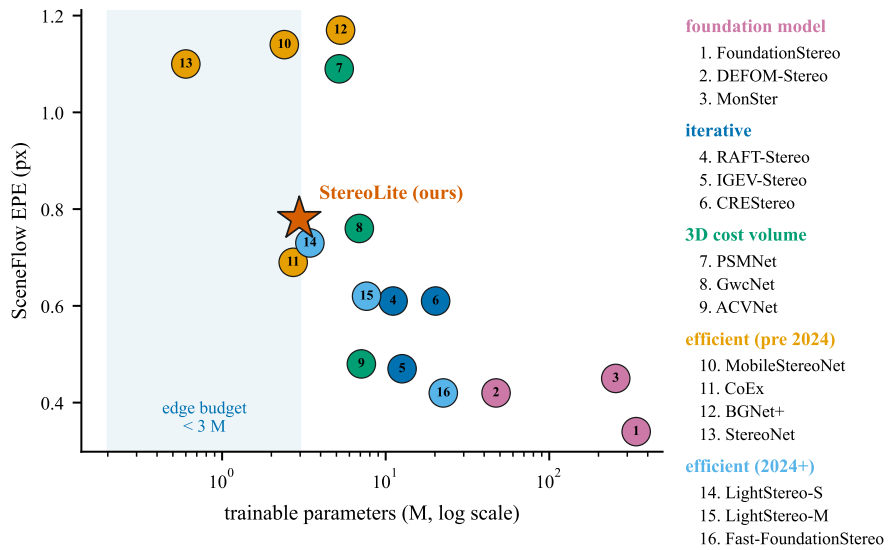


Figure 1.1: Trainable parameter count against SceneFlow accuracy, with the targeted edge budget shaded.

### 1.3 Research Questions / Problem Statements

Stereo matching takes a pair of rectified images and produces a dense disparity map, from which metric depth follows directly through the calibrated relation between disparity and depth. This thesis addresses that task under a hardware budget rather than an accuracy budget alone: a few million parameters, real-time latency on the target device, a small inference working set, and integer-friendly operators. The design assumes rectified binocular input, a maximum disparity of 192 pixels at the inference resolution, and training on synthetic data with dense ground truth.

Those requirements frame four research questions, each of which the thesis answers with measured evidence:

1. **RQ1.** Can a stereo network within a few million parameters, and inside the sub-3 million edge budget, reach competitive in-domain accuracy when trained on synthetic data with dense ground truth?
2. **RQ2.** Does such a compact, synthetic-only network generalize zero-shot to an unseen real domain, without any target-domain adaptation?
3. **RQ3.** Can it meet the real-time latency target on the intended edge device class, after inference optimization and eight-bit deployment?
4. **RQ4.** Does the design tolerate the small vertical misalignments of imperfect rectification, the condition a low-cost stereo rig produces, which is the second objective of this thesis?

### 1.4 Research Objectives

The thesis is built around two objectives, one for each half of the problem set out above: making stereo matching cheap enough to run on edge hardware, and making it robust to the imperfect camera geometry that such hardware supplies in practice. The objectives of this thesis are:

1. To design a computationally efficient stereo matching pipeline, leveraging AI-based disparity refinement to enhance depth estimation on resource-limited platforms.
2. To design an architecture able to withstand camera rectification imperfections.

The first objective is the heart of the thesis. Stereo matching is accurate today mainly because the leading networks are very large; the aim here is to keep most of that accuracy while shedding the size, so that depth estimation can run on the small, low-power computers that robots, drones, headsets, and handheld sensors actually carry, machines that offer only a few tera-operations per second, a few gigabytes of memory, and a few watts of power. The

phrase *AI-based disparity refinement* in the objective points to how this is done. The largest networks build one big cost volume and read the answer off it in a single step, and that step is where most of their cost lies. StereoLite instead starts from a rough disparity for each small image region and improves it a few times with a learned update; it is this repeated refinement, rather than a single expensive pass, that recovers accuracy without a large parameter count. Chapter 3 weighs the choice against the alternatives in full. The objective is met when the first three research questions are answered together: accuracy in domain, accuracy on an unseen real domain, and real-time speed on the edge board once the model is optimised and quantised.

The second objective addresses something the first does not. A network can score well on benchmark image pairs, which are aligned almost perfectly, and still struggle on a real camera, because a cheap stereo rig is never perfectly aligned: its two views are typically off by a fraction of a pixel to about a pixel in the vertical direction. An architecture that only works on flawless input is not yet ready for the hardware it is built for, so the second objective asks the network to hold its accuracy when that small misalignment is present. The thesis checks this two ways, which together answer the fourth research question: by shifting one image vertically by a known amount and watching how much the error grows, and by running the trained model on the project’s own, imperfectly aligned rig to confirm the error stays low.

One might ask why this thesis designs a new compact network instead of compressing an existing large one. There are two reasons. The accuracy of foundation-scale systems rests largely on their monocular priors and their sheer scale, and neither survives aggressive compression, so compressing such systems tends to remove the very component that makes them accurate. In addition, the tile-hypothesis and recurrent-refinement families were built for efficiency from the start and give a cleaner starting point for an edge budget.

## 1.5 Scope of the Research

It helps to state plainly what this thesis does and does not attempt, so the results are read against the right yardstick. The work is limited to passive, rectified, binocular stereo: depth is recovered from two horizontally separated cameras whose images have already been rectified, and nothing is emitted into the scene. The network is trained under supervision on synthetic image pairs with dense ground truth, kept compact at a few million parameters, and evaluated in three ways, on the synthetic test set it shares a domain with, zero-shot on an unseen real benchmark, and on the project’s own stereo rig. Its inference is then optimized and quantized to eight bits so it can be timed on the intended edge board rather than only on a desktop GPU. The recovered disparity is finally turned into metric depth and, for the rig scenes, into colored three-dimensional point clouds through the calibrated camera geometry; this reconstruction is the one named in the thesis title.

Several neighboring problems are deliberately left out, not because they lack value but because each would pull the work away from the edge-stereo question at its center. The thesis does not use active depth sensors such as LiDAR or structured light, does not train without labels through self-supervision, and does not estimate depth from a single image. It also stops at a working, measured research prototype: packaging the model as a deployment software development kit, and the systems engineering that a shipped product would require, are outside its scope. Where these boundaries touch a result, the relevant chapter says so, and the ones worth revisiting are collected as future work in Chapter 9.

## 1.6 Key Contributions

Meeting the two objectives above required building something (a network), deciding how to build it defensibly (a methodology), and then showing with measurements that both hold up (a set of results). The contributions of this thesis fall into those three kinds.

**An architecture.** This thesis contributes StereoLite, a 2.96 million parameter edge-budget stereo network purpose-built for objective 1. It combines a truncated YOLO feature encoder, a left-image recurrent context stream, plane-aware coarse-to-fine tile refinement, and a fail-soft quarter-resolution geometry encoding volume, four blocks chosen and combined specifically to keep parameter count, latency, and memory inside the edge budget without discarding the accuracy that larger cost-volume networks get from full-resolution search.

**A design methodology.** Building that network raised a recurring question: how does a designer justify keeping one block over an alternative under a hard parameter budget, rather than by intuition alone? This thesis answers it with an evidence-gated design methodology that admits an architectural choice only after it wins a controlled single-knob ablation on a fixed 100-pair harness, where competing variants share data, seed, and step budget so a measured difference is attributable to the design change alone. Every block listed above passed this test before being kept. The same evidence-gathering habit also produced a survey of the efficient and edge-deployable stereo families that grounds the design choices, stating what edge-deployable stereo requires and mapping the accuracy, latency, and memory trade-offs of the published literature.

**Results that follow from applying both.** Four measured outcomes demonstrate that the architecture and the methodology deliver on the two objectives:

- *In-domain accuracy (objective 1).* The trained checkpoint reaches 0.78 pixel end-point error and 3.40 percent D1-all on the full FlyingThings3D test set of 4,370 pairs, while running in real time on a laptop RTX 3050 GPU at 49.8 ms per forward pass at  $384 \times 640$  in half precision.
- *Cross-domain accuracy (objective 1).* The same training pipeline gives the compact

network 10.9 percent zero-shot Middlebury 2014 D1-all on a domain it never trains on, within four points of the larger distillation-trained LiteAnyStereo.

- *Edge deployment (objective 1)*. An equivalence-proven inference-optimization pass cuts latency by a factor of 1.74 with no loss of accuracy and identifies the operators that block eight-bit deployment, so that the optimized eight-bit model runs in 36.3 ms, or 27.5 frames per second, on the Jetson Orin Nano.
- *Rectification tolerance (objective 2)*. Under a controlled vertical misalignment sweep, end-point error rises only from 1.03 to 1.53 px across the sub-pixel to one-pixel range a calibrated rig produces, and on the project’s own rig the predicted disparity back-projects into metric colored point clouds on real indoor scenes.

## 1.7 Organization of the Report

The rest of this report is arranged as follows.

- Chapter 2 surveys the stereo-matching literature, moving from the classical pipeline through the deep-learning eras to the efficient and foundation-model families. It states what edge-deployable stereo requires and compares the published methods on accuracy, latency, and memory.
- Chapter 3 sets out the engineering design and methodology of StereoLite. It covers the design requirements and criteria, the design steps, the controlled ablation methodology, and the block-by-block architecture together with its assessment metrics.
- Chapter 4 describes the system development and implementation: the selection of parameters and components, the training objective and protocol, the software realization, and the edge-inference optimization.
- Chapter 5 covers the experimental setup and data collection, including the datasets, the stereo rig and calibration, the test benches, and the measurement procedures.
- Chapter 6 presents and discusses the results: the training dynamics, the ablation studies, the in-domain and zero-shot accuracy, the rectification robustness, the efficiency and deployment measurements, and the three-dimensional reconstruction.
- Chapter 7 considers the societal, health, and cultural impact of the work, its safety and ethical dimensions, and its alignment with the Sustainable Development Goals, including its environmental and sustainability case.
- Chapter 8 records how the thesis was managed, its time frame, its overall and component-level budget, its resource planning, and the division of work between the authors.

- Chapter 9 concludes the report, states its limitations, and outlines future work.

The appendices open with the modern tools used in the work (Appendix A) and the program-outcome mapping (Appendix B), then address the knowledge profiles, the complex engineering problems, and the complex engineering activities engaged by the thesis (Appendices C to E), and close with the technical specifications, the plagiarism report, and the AI writing report (Appendices F to H).

## Chapter 2

### Literature Review

#### 2.1 Introduction

This chapter surveys the literature that shapes the design of this thesis. Section 2.2 traces deep stereo matching chronologically, from the classical four-step pipeline and the first learned matching costs through three-dimensional cost-volume networks, iterative refinement, and the foundation-model systems that currently hold the accuracy ceiling. Because this thesis targets deployment on an edge accelerator rather than accuracy alone, that section then states the hardware requirements edge deployment imposes and classifies the efficient and edge-deployable literature by the technique family each uses to meet them. Section 2.3 draws the survey together into the specific gap this thesis targets. Section 2.4 then compares the families directly: it reviews cross-domain generalization and the benchmarks used to measure it, then sets published parameter counts, accuracy, and latency figures for the surveyed methods side by side, alongside our own model. Section 2.5 closes the chapter with the design lessons this survey establishes.

#### 2.2 Existing Research

##### 2.2.1 Stereo Matching Fundamentals

Stereo matching is the problem of finding, for each pixel in one image of a stereo pair, the pixel in the other image that captures the same scene point [2]. This subsection sets out the concepts that the rest of the thesis relies on: the epipolar geometry that reduces the search to one dimension, rectification, the disparity that encodes depth, the matching costs that measure image agreement, the cost volume that organizes those costs, the local and global families that solve for disparity, and the refinement that cleans the result. These ideas were formalized in the classical era [2] and survive, in learned form, in the deep networks reviewed later in this chapter [3, 4].

**Epipolar geometry.** The search for a match is constrained by geometry rather than left open over the whole image. Given the relative pose of the two cameras, a point seen at position  $p$  in the left image must lie, in the right image, on a single straight line called the epipolar line, and every epipolar line passes through a fixed point called the epipole, the image of the other camera’s center. Figure 2.1 illustrates this epipolar constraint: it reduces the two-dimensional problem of searching the entire right image to a one-dimensional problem of searching along a known line.

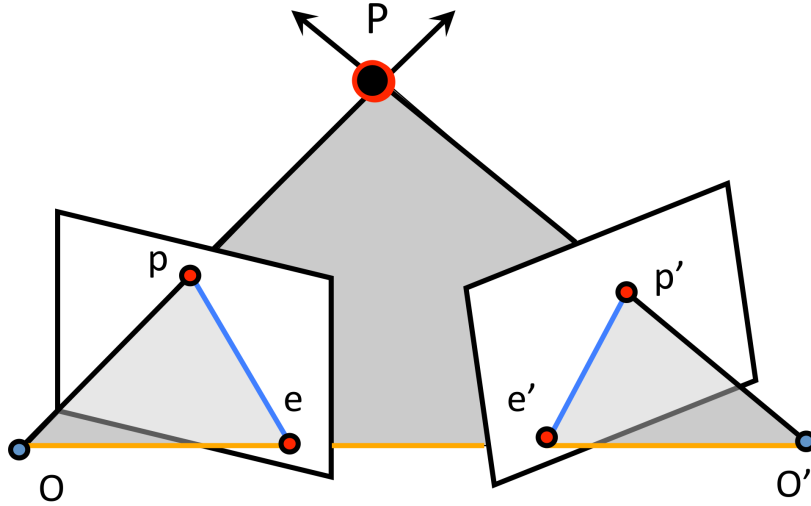


Figure 2.1: Epipolar geometry constraining stereo correspondence search to one dimension (Fei-Fei Li, CS231A).

**Rectification.** The epipolar lines of a general camera pair are slanted and vary from row to row, which makes them difficult to search directly. Rectification addresses this by reprojecting both images onto a common image plane parallel to the line joining the two camera centers, as Figure 2.2 shows. After this warp every epipolar line coincides with an image row, so corresponding points share the same vertical coordinate and the match for a left pixel  $(x, y)$  is found by sliding horizontally [2]:

$$x' = x - d, \quad y' = y \quad (2.1)$$

where  $(x', y')$  is the matching pixel in the right image and  $d \geq 0$  is the disparity. We assume rectified input throughout, so the correspondence search always reduces to a horizontal scan, and the accuracy of the rectification itself becomes a source of error worth examining directly.

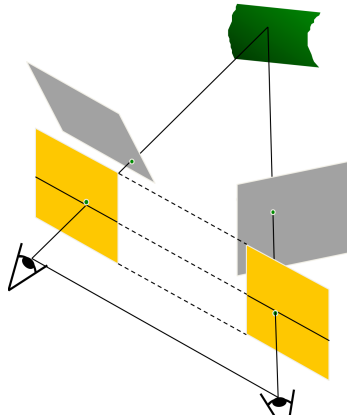


Figure 2.2: Stereo rectification reprojecting tilted image planes onto a common plane (Fei-Fei Li, CS231A).

**Disparity and depth.** The horizontal shift  $d$  between a point and its match is the disparity, and it fixes the depth of the scene point through the calibrated pinhole relation [2]

$$Z = \frac{f B}{d} \quad (2.2)$$

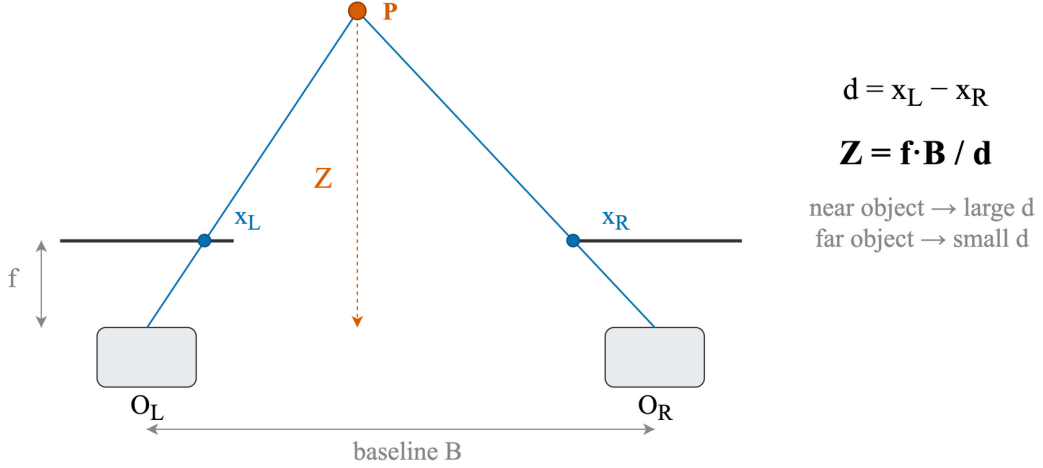


Figure 2.3: Two-view stereo geometry relating disparity to depth.

where  $Z$  is the depth of a scene point,  $f$  the camera focal length in pixels,  $B$  the baseline separating the two cameras, and  $d$  the disparity, the horizontal shift of the point between the two rectified views (Figure 2.3). Because the relation is inverse, disparity resolution matters most for distant surfaces: a fixed sub-pixel error in disparity corresponds to an increasingly large depth interval as the scene recedes, and this is why the field reports accuracy in fractions of a pixel.

**The matching cost.** To decide which candidate along the scanline is the true match, a stereo method needs a numerical measure of how well two image locations agree, called the matching cost. The classical costs trade robustness against simplicity [5]. The squared and absolute intensity differences are cheap but assume the two cameras record the same brightness, so they fail under gain and exposure differences. Normalized cross-correlation over a window removes a constant gain and bias and is therefore more stable. The census transform [6] is the most robust of the classical costs: it replaces each pixel by a binary string that records only whether each neighbor is brighter or darker than the center, and it compares two such strings by their Hamming distance,

$$C_{\text{cen}}(p, d) = \text{Ham}(\mathcal{T}(I_L, p), \mathcal{T}(I_R, p - d)), \quad \mathcal{T}(I, p) = \bigotimes_{q \in N_p} [I(q) \geq I(p)] \quad (2.3)$$

where  $\mathcal{T}$  is the census string built by concatenating (the operator  $\bigotimes$ ) the comparison bit  $[I(q) \geq I(p)]$  over the neighborhood  $N_p$ , Ham counts the positions at which two strings

differ, and  $[\cdot]$  is the indicator function. Because it keeps only the ordering of intensities and discards their magnitudes, the census cost is insensitive to illumination changes between the two views, as Figure 2.4 illustrates. This step is exactly what the first deep methods replaced with a learned network: MC-CNN [7] swapped the hand-crafted cost for a convolutional similarity score while leaving the rest of the classical pipeline in place, and the feature encoders of every modern network build on this same idea.

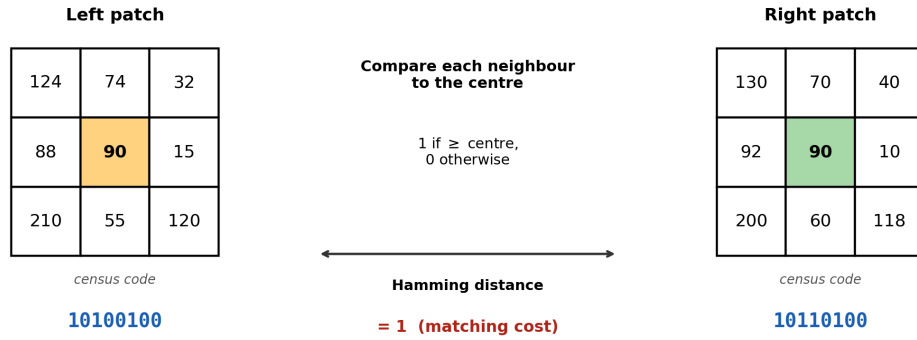


Figure 2.4: The census transform for illumination-robust matching cost computation.

**The disparity space image and cost volume.** Evaluating the matching cost for every pixel and every candidate disparity produces a three-dimensional structure  $C(x, y, d)$  called the disparity space image, or cost volume: for each pixel it records how well the two views agree at each possible shift, as visualized in Figure 2.5. Reading the volume along the disparity axis at one pixel gives a cost profile whose minimum is the winning disparity; slicing it at a fixed disparity shows which regions of the scene sit at that depth. This structure is the direct precursor of the learned cost volumes used throughout this thesis: modern networks build the same volume from learned features instead of raw intensities, but the object being reasoned over is still  $C(x, y, d)$ .

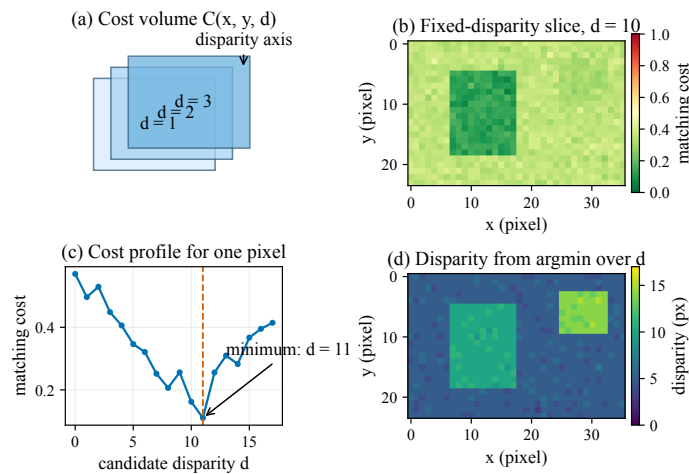


Figure 2.5: The disparity space image, or cost volume, for stereo matching.

**The four-step pipeline.** Scharstein and Szeliski [2] showed that every stereo algorithm, classical or deep, decomposes into the same four steps, shown in Figure 2.6: matching cost computation, which fills the cost volume; cost aggregation, which smooths it by pooling evidence from neighboring pixels; disparity optimization, which selects a disparity at each pixel; and disparity refinement, which post-processes the result. The rest of the chapter follows this decomposition, because deep learning progressively replaced each of the four steps with a learned module while keeping the overall structure intact.

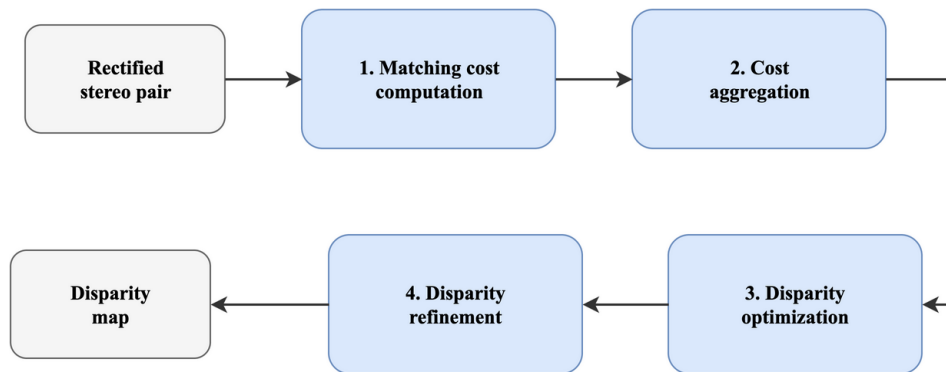


Figure 2.6: The four-step classical stereo matching pipeline [2].

**Local and global methods.** Classical algorithms split into two families by where they place their effort. Local methods concentrate on aggregation: they average the matching cost over a support window and then pick, at each pixel independently, the disparity of lowest cost. This is fast and simple, but it enforces smoothness only implicitly through the window, so it blurs depth edges (the window straddles two depths at a boundary) and fails in textureless regions where the window contains no distinctive signal. Global methods instead concentrate on optimization: they define an energy over the whole disparity map that adds a data term, rewarding low matching cost, to a smoothness term, penalizing disparity jumps between neighbors, and they search for the disparity map of least energy. This yields sharp, consistent results because the smoothness prior is explicit and can be made discontinuity-preserving, but the joint optimization is far more expensive than per-pixel selection. The trade-off between the two families, aggregation speed against optimization quality, reappears in the efficient deep methods surveyed later in this chapter.

**Disparity optimization and semi-global matching.** Semi-global matching [8] became the standard classical algorithm by approximating the expensive two-dimensional global optimization as a sum of independent one-dimensional dynamic-programming problems solved along several path directions, which recovers most of the quality of a full global method at a fraction of the cost. It minimizes an energy that combines a per-pixel matching cost with

two smoothness penalties over neighboring pixels,

$$E(D) = \sum_p C(p, D_p) + \sum_p \sum_{q \in N_p} \left[ P_1 \mathbb{I}[|D_p - D_q| = 1] + P_2 \mathbb{I}[|D_p - D_q| > 1] \right] \quad (2.4)$$

where  $D$  is the disparity map,  $C(p, D_p)$  the matching cost of assigning disparity  $D_p$  to pixel  $p$ ,  $N_p$  the neighbors of  $p$ ,  $P_1$  a small penalty for one-pixel disparity changes that preserves slanted surfaces,  $P_2$  a larger penalty for jumps beyond one pixel that permits true discontinuities, and  $\mathbb{I}[\cdot]$  the indicator function. Semi-global matching links directly to deep stereo: the first learned method kept it as an optimization backend, and later networks replaced its fixed aggregation with learnable layers.

**Disparity refinement.** The disparity selected from a discrete cost volume is an integer, and it still carries errors in occluded and textureless regions, so a final refinement step improves both its precision and its reliability. Sub-pixel precision is recovered by fitting a parabola [2] to the three cost values around the winning disparity  $d$  and taking the location of its minimum,

$$d_{\text{sub}} = d - \frac{C(d+1) - C(d-1)}{2[C(d+1) - 2C(d) + C(d-1)]} \quad (2.5)$$

where  $C(d)$  is the aggregated cost at the winning disparity and  $C(d \pm 1)$  its two neighbors, which yields a fractional disparity without enlarging the search range. A left-right consistency check computes the disparity from both images and flags as occluded any pixel whose two estimates disagree by more than a pixel, and the flagged pixels are then filled from nearby reliable values and cleaned by median filtering. Each of these operations reappears in learned form in the modern networks reviewed below, where refinement is carried out by trained convolutional or recurrent modules rather than fixed rules.

### 2.2.2 The Deep Learning Transition

Deep learning entered stereo matching first through the matching-cost step. MC-CNN [7] replaced only the cost computation with a Siamese convolutional network that scores the similarity of two patches, and fed those scores into an otherwise classical pipeline of cross-based aggregation, semi-global optimization, and refinement. It established a trick that later networks kept: compute each image’s features once and reuse them across all disparities. The method itself was not end-to-end. DispNetC [9] removed the classical stages entirely and regressed disparity directly with an encoder-decoder network. Its central component was a correlation layer, which forms a compact cost volume by correlating the two feature maps along the scanline, as in (2.6); it ran roughly a thousand times faster than MC-CNN at comparable accuracy. The same work introduced the SceneFlow synthetic dataset, which remains the standard pretraining set for stereo networks. GC-Net [10] completed the transition to a fully learned cost-volume network. It builds a four-dimensional cost volume by

concatenating the left and right feature vectors at each disparity, regularizes it with a three-dimensional convolutional encoder-decoder, and regresses a continuous disparity with the differentiable soft-argmin of (2.7), an operator still used across cost-volume networks today.

The correlation layer [9] forms a scalar matching cost by taking the inner product of the two feature maps at each disparity along the scanline,

$$c(x, y, d) = \langle f_L(x, y), f_R(x - d, y) \rangle = \sum_k f_L^{(k)}(x, y) f_R^{(k)}(x - d, y) \quad (2.6)$$

where  $c(x, y, d)$  is the matching cost for reference pixel  $(x, y)$  at disparity  $d$ ,  $f_L$  and  $f_R$  the shared feature maps, and the inner product runs over the feature channels  $k$ ; the search covers  $d \in [0, D_{\max}]$ .

The soft-argmin [10] turns the regularized cost volume into a continuous disparity by taking a softmax-weighted expectation,

$$\hat{d}(x, y) = \sum_{d=0}^{D-1} d \cdot \text{softmax}_d(-c_d(x, y)) \quad (2.7)$$

where  $\hat{d}(x, y)$  is the regressed sub-pixel disparity and  $c_d(x, y)$  the aggregated cost at disparity  $d$ ; the negative sign turns low cost into high probability. The operator assumes a single-peaked cost distribution and returns an intermediate value where that assumption fails, such as at a depth discontinuity.

### 2.2.3 The 3D Cost-Volume Era

The years that followed refined the three-dimensional cost-volume paradigm. PSMNet [11], shown in Figure 2.7, added spatial pyramid pooling to inject global context at the feature level, along with three stacked hourglass modules under intermediate supervision, and became the de-facto baseline for the era.

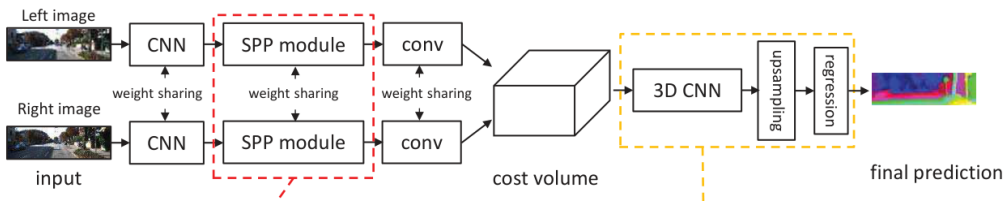


Figure 2.7: PSMNet architecture, representing the three-dimensional cost-volume paradigm [11].

GwcNet [12] introduced the group-wise correlation volume, which splits the features into channel groups and computes an inner-product correlation per group, as in (2.8). This form sits between the information-poor full correlation and the expensive concatenation, and it degrades far more gracefully than concatenation under a reduced aggregation budget,

which is why nearly every later efficient method adopts it, including the tile initialization we use. GA-Net [13] addressed the cost of three-dimensional convolution directly: it replaced the convolution with a differentiable semi-global aggregation layer that approximates recursive aggregation along a few directions and a local guided aggregation layer for thin structures, and showed that a network with only two such layers can outperform GC-Net with nineteen three-dimensional convolutions. Later work pushed efficiency further: AANet [14] removed three-dimensional convolution entirely in favor of deformable intra-scale and cross-scale aggregation, ACVNet [15] used an attention map derived from correlation to filter a concatenation volume, and CFNet [16] shifted the objective toward cross-domain generalization through a fused multi-scale volume and an uncertainty-driven cascade that adaptively narrows the disparity search.

The group-wise correlation [12] splits the  $N_c$  feature channels into  $N_g$  groups and computes a normalized inner-product correlation within each group,

$$C_{\text{gwc}}(g, d, x, y) = \frac{N_g}{N_c} \langle f_L^{(g)}(x, y), f_R^{(g)}(x - d, y) \rangle \quad (2.8)$$

where  $g$  indexes the group,  $d$  the disparity, and  $f_L^{(g)}, f_R^{(g)}$  the channel slice of each feature map belonging to group  $g$ ; the resulting volume carries one channel per group. We use this operator at a single coarse scale in our tile initialization.

#### 2.2.4 Iterative Refinement

RAFT-Stereo [17], shown in Figure 2.8, introduced a different paradigm: it adapted the iterative refinement of RAFT optical flow to stereo. It builds an all-pairs one-dimensional correlation volume, uses a separate context encoder to seed a multi-level convolutional gated recurrent unit, and refines the disparity through many lightweight update iterations, each of which looks up correlation values around the current estimate. It restores full resolution with a learned convex upsampling that predicts a weighted combination of a coarse neighborhood.

Three of these mechanisms, the group-wise cost lookup, the recurrent update, and convex upsampling, reappear in modified form in our own design. IGEV-Stereo [18] improved the paradigm by building a geometry encoding volume, a lightweight three-dimensional-regularized cost volume that both provides a warm-start disparity through soft-argmin and is looked up each iteration alongside the all-pairs correlation, as in (2.10); with this warm start it matches RAFT-Stereo’s thirty-two iterations in three. The geometry encoding volume is the direct ancestor of the geometry branch we use. The warm start matters because an iterative method spends its early iterations recovering what a good initialization provides for free; with it, IGEV-Stereo at a single iteration already outperforms RAFT-Stereo by a wide margin, and the auxiliary loss on the initial estimate forces the geometry volume to be a genuine disparity predictor rather than a mere feature bank. Two later works refined the

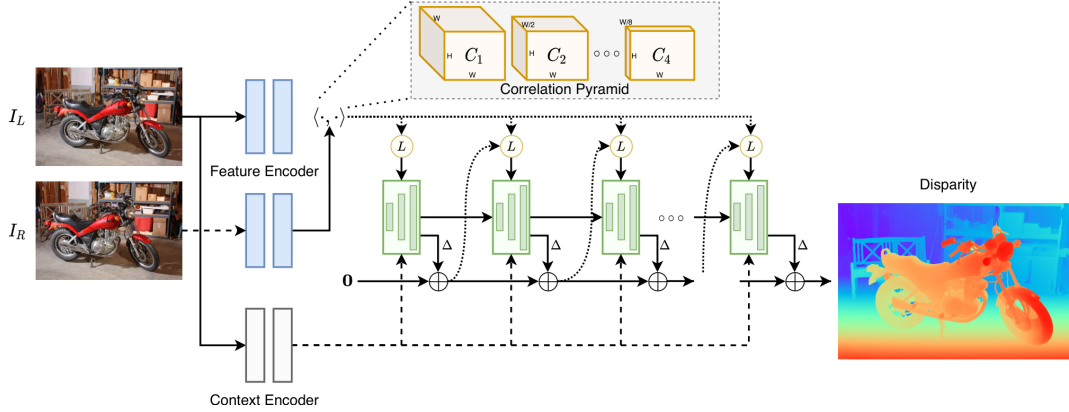


Figure 2.8: RAFT-Stereo architecture, representing the iterative refinement paradigm [17].

recurrent core in directions relevant to this thesis. CREStereo [19] introduced a cascaded recurrent refinement whose adaptive group correlation layer searches a small two-dimensional window rather than assuming a perfectly horizontal epipolar line, and its ablation showed local correlation outperforming the all-pairs form outright; both findings matter here, the first because a two-dimensional local search is precisely what confers tolerance to imperfect rectification, the second because it justifies replacing the all-pairs volume with a small local lookup in a compact model. Selective-Stereo [20] added a selective recurrent unit that runs two gate branches with different kernel sizes and fuses them per pixel through a contextual attention map, reaching the quality of many more iterations in a few; when we tried the same unit in our ablation campaign it did not transfer to the compact tile setting.

The recurrent update is a convolutional gated recurrent unit [17] that updates a hidden state from a per-iteration input,

$$\begin{aligned} z_n &= \sigma(\text{Conv}([h_{n-1}, x_n])), & r_n &= \sigma(\text{Conv}([h_{n-1}, x_n])), \\ \tilde{h}_n &= \tanh(\text{Conv}([r_n \odot h_{n-1}, x_n])), & h_n &= (1 - z_n) \odot h_{n-1} + z_n \odot \tilde{h}_n \end{aligned} \quad (2.9)$$

where  $x_n$  assembles the correlation lookup, the encoded disparity, and the context,  $h_{n-1}$  is the previous hidden state,  $z_n$  and  $r_n$  the update and reset gates,  $\tilde{h}_n$  the candidate state,  $\sigma$  the sigmoid, and  $\odot$  the element-wise product; a small head then predicts the disparity residual from  $h_n$ .

The geometry encoding volume of IGEV-Stereo [18] regularizes a group-wise correlation volume and combines it with the all-pairs correlation into a lookup structure,

$$C_G = R(C_{\text{gwc}}(f_L, f_R)), \quad d_0 = \sum_{d=0}^{D-1} d \cdot \text{softmax}_d(C_G(d)) \quad (2.10)$$

where  $C_{\text{gwc}}$  is the group-wise correlation of the quarter-resolution features,  $R$  a lightweight

three-dimensional network,  $C_G$  the regularized geometry volume, and  $d_0$  the warm-start disparity; each iteration then looks up values from both  $C_G$  and the all-pairs correlation.

A parallel line replaced the cost volume itself with attention. STTR [21] treats matching as dense sequence-to-sequence correspondence along the epipolar lines, alternating self-attention within each image and cross-attention between the views, with an optimal-transport assignment that handles occlusion explicitly and removes the fixed maximum-disparity assumption altogether. The transformer family improved occlusion reasoning and freed the disparity range, but its dense attention has remained costly at the resolutions and budgets that edge deployment demands, which is why we draw on the recurrent family instead.

### 2.2.5 Foundation-Model Era

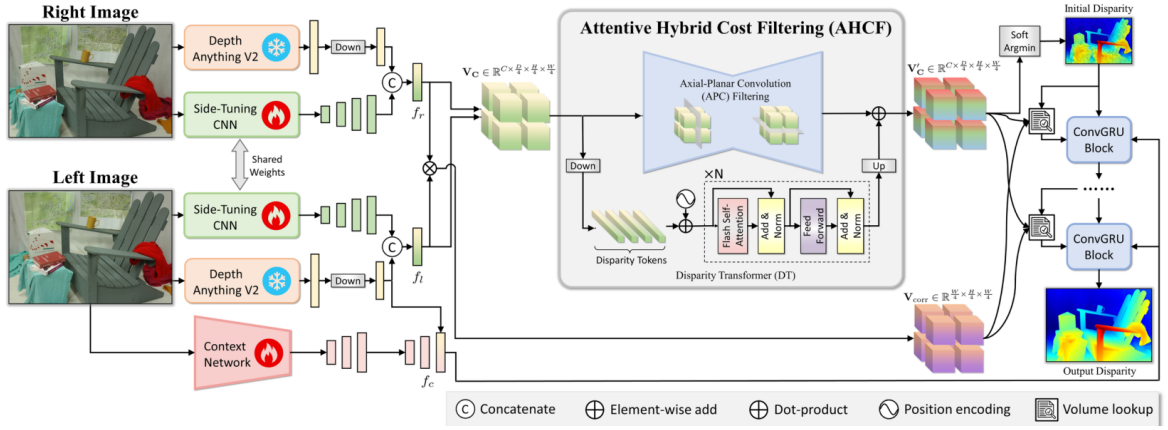


Figure 2.9: FoundationStereo architecture, representing the foundation-model-era paradigm [1].

The most recent and most accurate systems follow a similar approach: they graft a frozen monocular vision foundation model onto an iterative stereo core and train on very large data. FoundationStereo [1] fuses a frozen DepthAnythingV2 transformer with a light convolutional encoder through a side-tuning adapter, adds an attentive hybrid cost filter, and trains on a million synthetic pairs, reaching 0.34 pixel end-point error on SceneFlow; its architecture is shown in Figure 2.9. DEFOM-Stereo [22] adds a monocular depth foundation model to a RAFT-Stereo core and splits each iteration into a scale update that corrects the monocular prior and a standard refinement. MonSter [23] couples an IGEV stereo branch with a monocular branch through bidirectional refinement, and Stereo-Anywhere [24] uses a monocular prior to truncate the stereo cost volume and suppress the false matches that mirrors and glass produce. The prior enters each method differently. DEFOM-Stereo splits each recurrent iteration into a scale update, which corrects the monocular prior’s per-pixel scale through a dedicated lookup over candidate scale factors, and a standard residual update; its ablation shows the monocular initialization alone hurts and only becomes useful with the scale correction. MonSter runs the two branches symmetrically, each guiding the

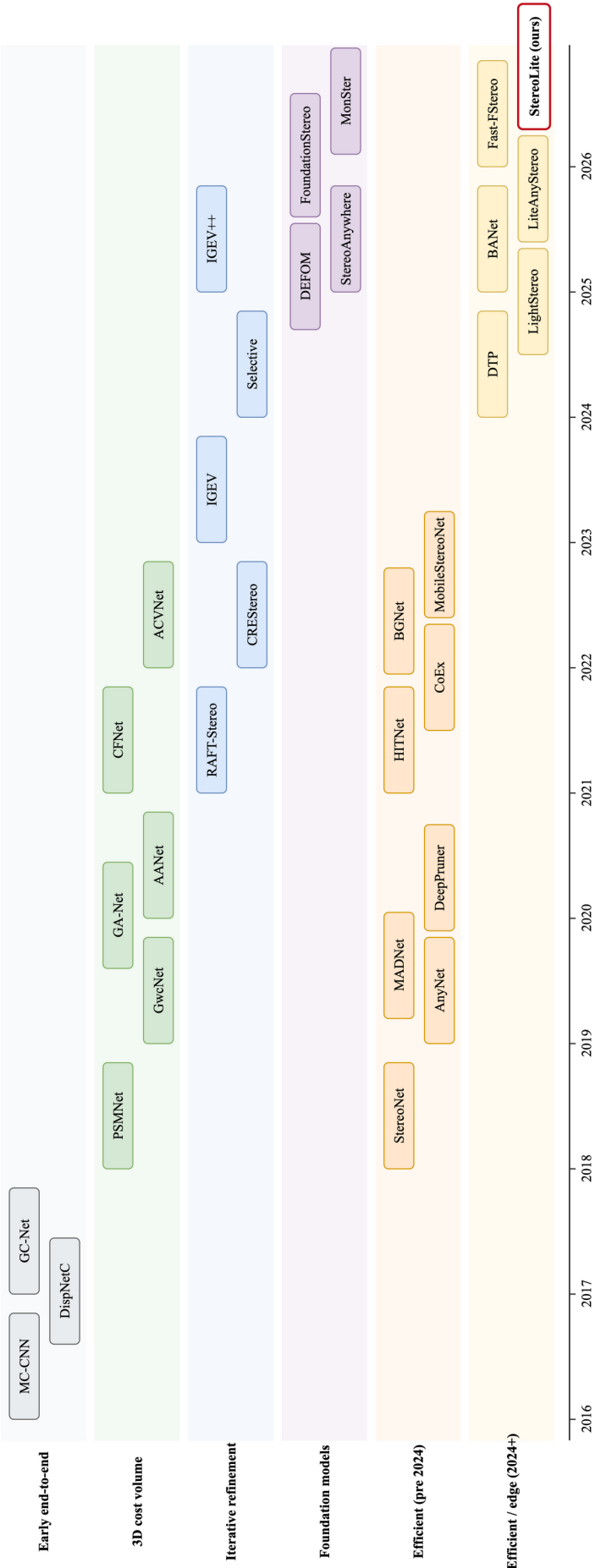


Figure 2.10: Evolution of deep stereo matching, 2016 to 2026.

other after a global scale and shift alignment, and its ablation shows four iterations with the prior matching what the plain stereo core needs thirty-two iterations to reach. The accuracy gain therefore comes from two sources: the prior’s knowledge of monocular structure and the iterations it saves. These systems set the accuracy ceiling, but they do so with hundreds of millions of parameters and hundreds of milliseconds of latency, a cost that motivates the edge problem we address rather than solving it. Figure 2.10 traces this progression across the six paradigm eras and marks where the present work sits.

### 2.2.6 Requirements of Edge-Deployable Stereo Matching

Deployment on an edge accelerator cannot be solved by choosing the most accurate published network: models at the accuracy ceiling carry hundreds of millions of parameters and hundreds of milliseconds of latency, and mid-size iterative networks run their update loop tens of times per frame on desktop graphics hardware. Edge accelerators on mobile robots and drones, exemplified by the Jetson Orin Nano class of device, impose four requirements that bound every design choice we make in this thesis.

First, the network must be compact enough for on-device memory: accelerators share memory with the robot’s other software, so the parameter budget must stay within a few million parameters, the region occupied by the efficient family surveyed next. Second, the network must run in real time on a device of roughly four to six tera-operations per second; a perception stage that cannot keep up with the camera forces the robot to act on stale depth, so latency is a hard requirement that rules out many-iteration update loops and dense attention. Third, the network must keep a small inference working set. Peak activation memory, dominated by the cost volume in most stereo networks, must fit alongside the model weights and the application; this is why cost-volume compression forms its own technique family below. Fourth, the network must use operators that quantize cleanly to eight-bit integers, since accelerators reach rated throughput only under integer quantization; operators that resist quantization, such as trilinear interpolation in three-dimensional volumes, forfeit that throughput regardless of parameter count. These observations frame the classification below.

### 2.2.7 Classification of Efficient Stereo Methods

The efficient and edge-deployable methods are the closest relatives of the model proposed in this thesis, and they are best understood by the technique family each uses to reduce cost. Figure 2.12 organizes these families, and the subsections below treat each in turn.

Figure 2.11 makes the connection explicit: each deployment requirement is answered primarily by one or two of the technique families, and a family that answers no requirement of the target platform can be skipped without loss.

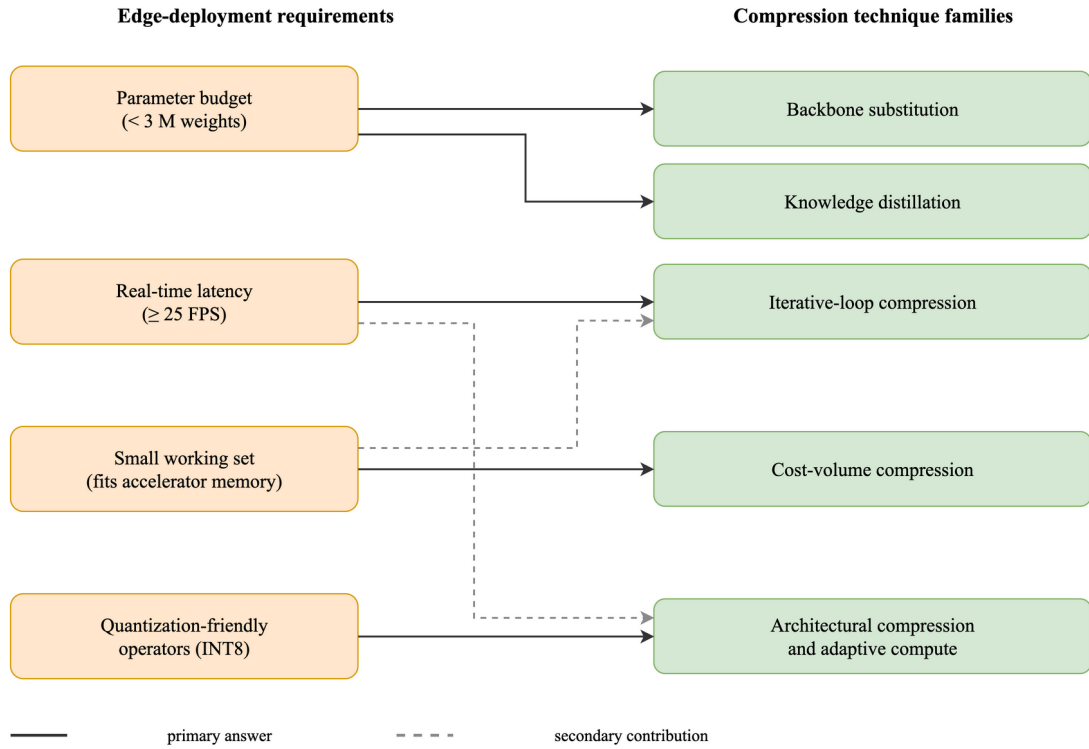


Figure 2.11: Edge-deployment requirements mapped to compression technique families.

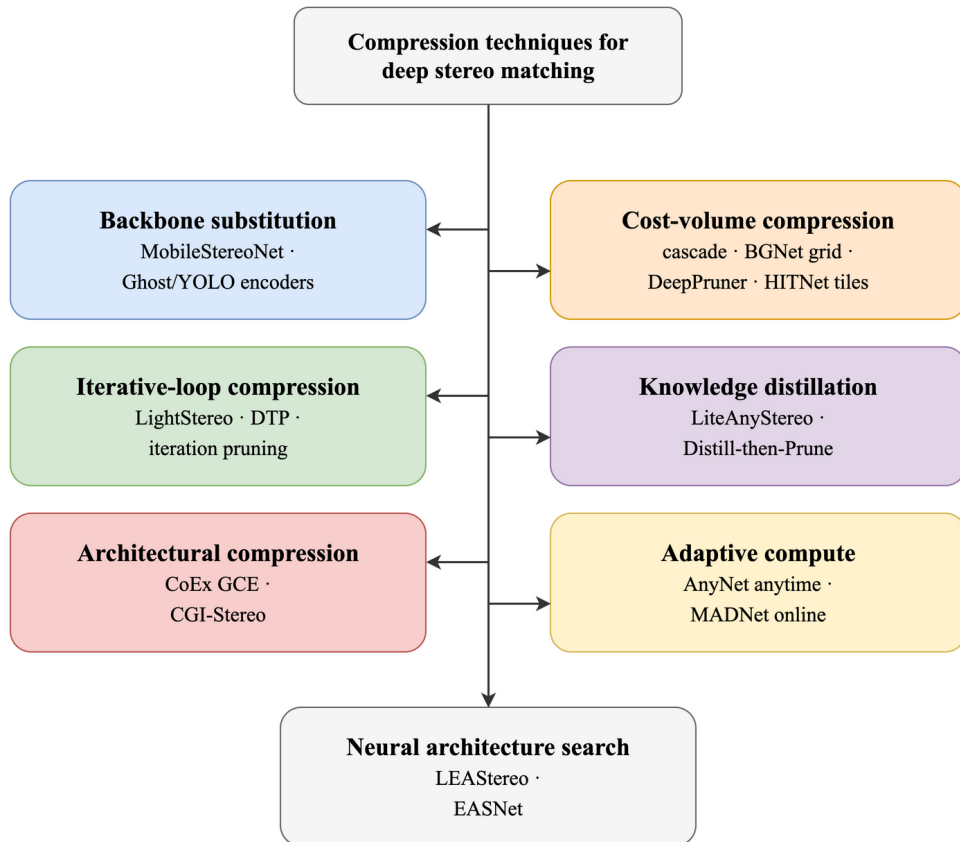


Figure 2.12: Taxonomy of compression techniques for deep stereo matching.

### 2.2.8 Backbone Substitution

The simplest way to reduce cost is to replace the heavy feature encoder with a mobile backbone. MobileStereoNet [25] extended MobileNet’s separable blocks into the three-dimensional cost-volume regularization itself and reported two design points: a two-dimensional variant at 1.14 pixel end-point error with 2.23 million parameters, and a three-dimensional variant at 0.80 pixels with 1.77 million, an order of magnitude smaller than the contemporaneous volumetric baselines at comparable accuracy. The lesson generalizes beyond MobileNet: any well-pretrained compact backbone can serve, and a detection backbone carries an added advantage, since its features are already trained to localize objects across scales. We follow this route and use a truncated detection backbone as our shared encoder.

### 2.2.9 Cost-Volume Compression

The three-dimensional cost volume is the dominant cost in most stereo networks, and several families compress it. Cascade methods [26] build a coarse volume over the full disparity range and then progressively narrow the range at finer scales; on the cascade paper’s reproduced PSMNet baseline the two-stage cascade cut memory substantially while also improving the end-point error from 0.887 to 0.721 pixels (a reproduced baseline, distinct from the 1.09 pixels PSMNet reports in Table 2.2), so narrowing the range turned out to be cheaper and more accurate at once. BGNet [27] learns a bilateral grid that upsamples a one-eighth-resolution cost volume in an edge-preserving way, with a parameter-free slicing step, at 25 milliseconds per frame. DeepPruner [28] predicts a per-pixel disparity range with a differentiable PatchMatch and a confidence-range predictor, so the expensive aggregation runs only on a pruned set of hypotheses; its fast variant reaches 0.97 pixel end-point error at 61 milliseconds. HITNet [29], shown in Figure 2.13, removes the three-dimensional cost volume altogether: it represents the scene as a grid of small tiles, each carrying a slanted-plane hypothesis with a disparity, two slopes, and a descriptor, and refines these hypotheses through two-dimensional propagation and warping across scales, reaching 0.43 pixel end-point error at 0.97 million parameters and 54 milliseconds. The slanted plane lets a tile represent sub-pixel structure that a fronto-parallel hypothesis cannot, and this is what lets the network stay accurate without a dense volume. This tile-hypothesis representation, including its plane slopes, is the paradigm from which we build our model.

### 2.2.10 Iterative-Loop Compression

The update loop is where iterative methods spend most of their cost, and the efficient variants target it directly. LightStereo [30] replaces three-dimensional aggregation with a two-dimensional network of inverted-residual blocks that expand the disparity channels, reaching 0.73 pixel end-point error at 3.44 million parameters and 17 milliseconds; its authors observe that feature extraction, not aggregation, dominates the remaining runtime. Distill-then-Prune

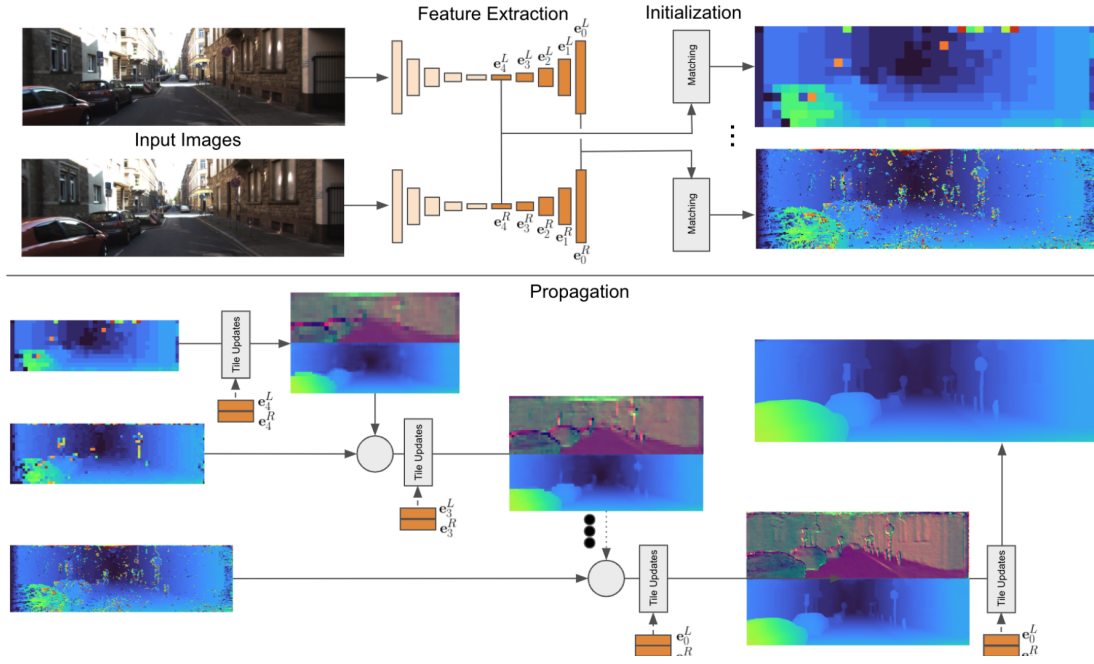


Figure 2.13: HITNet architecture, representing the tile-hypothesis paradigm [29].

[31] designs a backbone that is compatible with deployment toolchains by construction: it avoids three-dimensional convolution and trilinear interpolation, and reaches 1.56 pixel end-point error at 0.26 million parameters with 16 milliseconds of latency on an embedded board.

### 2.2.11 Knowledge Distillation

A distinct family reduces the accuracy cost of a small network by training it against a large teacher. LiteAnyStereo [32], whose network is shown in Figure 2.14, pairs a 7.6 million parameter non-iterative network with a three-stage training pipeline: supervised training on synthetic data, self-distillation under input perturbation, and distillation from a frozen FoundationStereo teacher on unlabeled real pairs. The result is strong zero-shot accuracy that the same network cannot reach by supervised training alone.

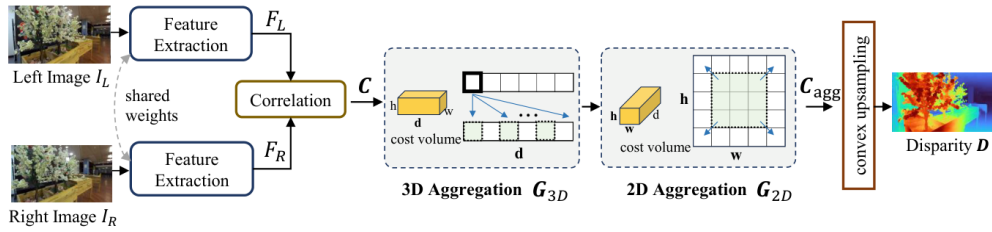


Figure 2.14: LiteAnyStereo architecture, representing the distillation-trained efficient paradigm [32].

Distill-then-Prune [31] shows that distilling before pruning, and using the teacher output alone rather than the teacher combined with ground truth, gives the best student. Written

generically, rather than in any one paper’s notation, the distillation term of such pipelines is a masked regression between the student and teacher disparity,

$$\mathcal{L}_{\text{KD}} = \frac{1}{|\Omega|} \sum_{i \in \Omega} m_i \cdot \text{smooth}_{L_1}(\hat{d}_S(i) - \hat{d}_T(i)) \quad (2.11)$$

where  $\hat{d}_S$  and  $\hat{d}_T$  are the student and teacher disparities,  $\Omega$  the image domain, and  $m_i$  an optional teacher-confidence mask. This three-stage pipeline, illustrated in Figure 2.15, has not yet been applied to our model, although it may offer the largest accuracy gain among the surveyed mechanisms and remains promising future work.

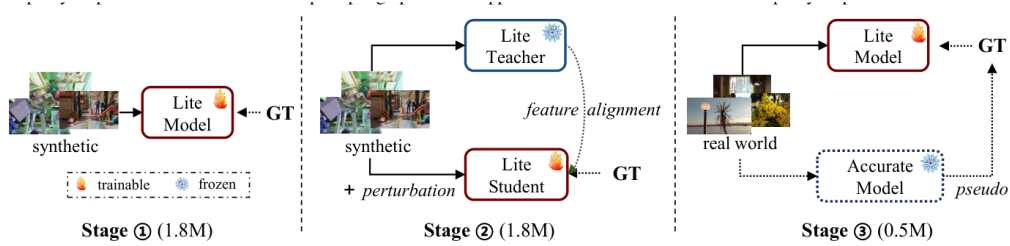


Figure 4. Overview of the proposed three-stage training strategy. **Stage ①**: The lite model is trained using a standard supervised setup

Figure 2.15: The three-stage knowledge-distillation pipeline of LiteAnyStereo [32].

### 2.2.12 Architectural Compression and Adaptive Compute

CoEx [33] re-enriches a cheap, information-poor correlation volume from the image side: its guided cost-volume excitation derives per-channel attention weights from the left features with cheap convolutions and applies them at every aggregation scale, recovering the accuracy of a far costlier concatenation volume at 2.72 million parameters and 27 milliseconds. It also swaps the full soft-argmin for a regression over the top disparity candidates, avoiding the between-modes averaging of (2.7). CGI-Stereo [34] fuses context into geometry at every decoder scale with bidirectional gradient flow, beating far larger networks zero-shot at a fraction of their cost. The same trade, cheap image-derived guidance for volumetric capacity, recurs in our guided excitation and mask-based upsampling. LightStereo [30] adds a complementary finding: expanding the disparity channels in its inverted-residual aggregation blocks helps, but expanding the spatial kernels does not, so the useful capacity lies along the disparity axis.

Adaptive-compute methods change the cost at run time. AnyNet [35] produces a usable disparity from a first coarse stage and refines it through residual cost volumes with a narrow search range, letting an embedded board trade 29 milliseconds of coarse depth against 97 milliseconds of refined depth as the budget allows. MADNet [36] instead adapts online to a new environment, updating one module per frame under a photometric self-supervised loss via a bandit strategy for module choice, and recovers most of the accuracy a zero-shot deployment loses while sustaining 25 frames per second. AnyNet’s narrow-range search and

both methods’ coarse-to-fine budgeting anticipate the local lookup and iteration schedule we use. Neural architecture search reaches the same pipeline in LEAStereo [37], which jointly searches the feature and matching networks within the stereo pipeline and reaches 0.78 pixel end-point error at 1.81 million parameters, showing a well-allocated small budget can rival hand designs an order of magnitude larger.

Three lessons from the efficient family shape our design. First, the cost volume is where the budget goes: every successful compact method shrinks, prunes, or replaces it with a sparser hypothesis structure, and the tile representation we adopt is the most aggressive. Second, image-derived guidance is cheap and effective: excitation, bilateral-grid, and mask-based upsampling all steer a small volume or coarse prediction from the image, a pattern we follow with our convex upsampling and context stream. Third, the training recipe is not an afterthought: the distillation results show the same small architecture can land at very different accuracy purely from how it is trained, so we treat and ablate our training protocol with the same rigor as the architecture. These families are largely complementary; our model combines several, pairing a substituted backbone with a compressed cost volume and a compressed recurrent loop.

## 2.3 Research Gap

The survey above points to a specific opening rather than a general call for smaller networks. Three observations, taken together, define it.

First, accuracy and deployability sit at opposite ends of the surveyed work. The foundation-model and iterative systems hold the accuracy ceiling but carry tens to hundreds of millions of parameters and hundreds of milliseconds of latency, which no edge accelerator of the target class can afford. The efficient family fits the budget, but each of its members attacks a single source of cost in isolation: backbone substitution replaces the encoder, cost-volume compression shrinks or removes the volume, iterative-loop compression trims the update schedule, and knowledge distillation reduces the accuracy penalty through training rather than architecture. What is missing is a single compact design that combines several of these reductions at once, a substituted detection backbone, a tile-hypothesis representation in place of a dense volume, a short recurrent loop, and a narrow-band geometry volume, and shows that the combination still lands inside the edge budget. Building and justifying that combination is the task of Chapter 3.

Second, the efficient literature is reported almost entirely on in-domain accuracy, parameters, and latency, and says far less about how these compact models behave when the input geometry is imperfect. The benchmarks that drive the field are near-perfectly rectified, yet the low-cost stereo rigs these models are meant for leave a residual vertical misalignment of up to about a pixel. A method that is only ever measured on flawless input leaves its field robustness untested, and the survey found no compact model that reports its accuracy

as a function of controlled rectification error. Closing that measurement gap is the second objective of this thesis.

Third, the survey left one methodological question unsettled: how much of a compact network’s cross-domain accuracy comes from its architecture and how much from its training recipe. The distillation results show the same small network reaching very different zero-shot accuracy purely from how it is trained, while other efficient methods credit their recurrent core. The literature does not resolve this with controlled evidence, which is why this thesis treats the training protocol as a first-class design variable and ablates it under the same single-knob discipline it applies to the architecture.

These three gaps, an unbuilt combination, an untested robustness, and an unsettled attribution, are what the comparison in the next section quantifies and what the design chapter sets out to close.

## **2.4 Comparative Analysis**

### **2.4.1 Cross-Domain Generalization**

A network trained on synthetic data often fails on real domains: it learns shortcut features tied to the synthetic statistics, such as consistent local color distributions, instead of domain-invariant structure, and it breaks down in textureless, reflective, repetitive, and large-disparity regions. The scale of the failure is substantial: GA-Net trained on synthetic data and evaluated zero-shot reaches 11.7 percent outliers on KITTI 2015 and 32.2 percent on Middlebury, and PSMNet reaches 16.3 percent on KITTI, several times their fine-tuned rates [38]. Two families of remedy exist. Architecture-level methods make the features themselves domain-invariant. DSMNet [38] adds a domain normalization that removes image-level and pixel-level statistic shifts together with a structure-preserving graph filter, and improves the zero-shot KITTI rate from 11.7 to 6.5 percent and the Middlebury rate from 32.2 to 21.8 percent. HVT [39] applies hierarchical visual transformations at global, local, and pixel level during training only, at zero inference cost, and pushes the zero-shot KITTI rate further to 4.9 percent on the same backbone. Training-recipe methods change the data and the supervision instead: StereoAnything [40] mixes labeled stereo with tens of millions of monocular-to-stereo pseudo-pairs and shows that dataset diversity matters more than dataset size, and LiteAnyStereo [32] reaches strong zero-shot accuracy with a non-iterative network trained by three-stage distillation. The relative importance of iteration and training pipeline for generalization is not settled in the literature: some efficient methods argue that iteration is necessary for cross-domain accuracy, while LiteAnyStereo reaches competitive zero-shot numbers without any iteration, and the cleanest controlled evidence attributes its accuracy to the training pipeline rather than to any recurrent module. Our own non-iterative tile network follows this second line of evidence.

## 2.4.2 Benchmarks and Datasets

Table 2.1 summarizes the benchmarks used in this thesis. SceneFlow is a large synthetic dataset of 35,454 training pairs across three subsets with dense rendered ground truth, and it is the standard pretraining set; only its FlyingThings3D subset provides an official test split, which is used here for in-domain evaluation. Its value lies in the density and exactness of its labels, which real sensors cannot provide, at the price of a domain gap discussed in the preceding subsection. KITTI 2012 and 2015 provide real driving scenes with sparse LiDAR ground truth, and their D1-all outlier rate has become the field’s standard metric for driving. ETH3D covers mixed indoor and outdoor scenes at gray-scale with laser-scanned ground truth. Middlebury 2014 provides high-resolution indoor scenes with dense structured-light ground truth, and it is the hardest of the four in two respects: its images reach several megapixels, so most methods evaluate at half or quarter resolution, and its disparities extend to several hundred pixels, far beyond the search range most networks are built for. These two properties make it the standard zero-shot stress test, and it is the cross-domain benchmark we use. Its evaluation protocol splits the scenes into a training set with public ground truth and a test set whose ground truth is withheld on a server. The zero-shot convention we follow, matching the reference methods we compare against, evaluates on the ground-truth-public scenes without any training or fine-tuning on Middlebury. The metrics also carry conventions worth noting: end-point error rewards sub-pixel accuracy on smooth surfaces and can look healthy while boundaries are poor, whereas the bad- $t$  and D1-all rates count outlier pixels and expose exactly the boundary and textureless failures that end-point error averages away. For this reason, we report both families of metric together.

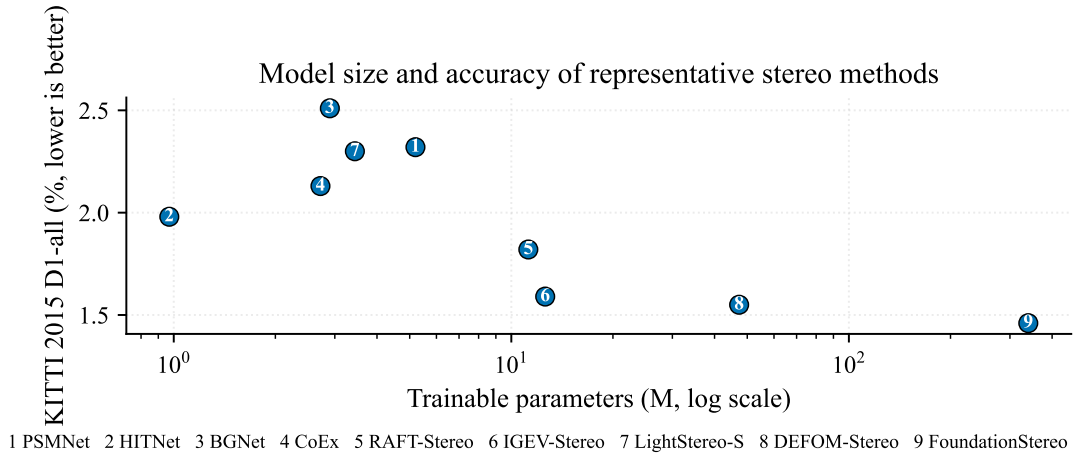


Figure 2.16: Accuracy against parameters and latency for the surveyed methods: (a) accuracy against trainable parameters.

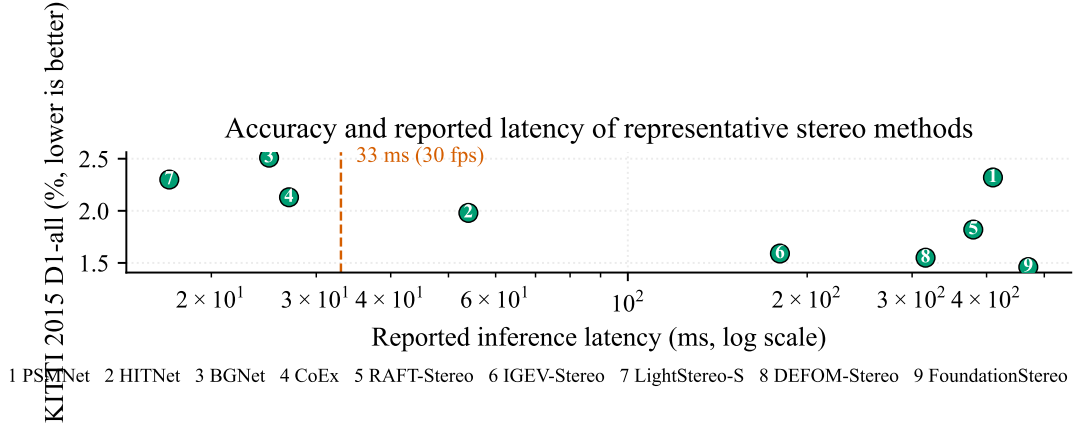


Figure 2.16: Accuracy against parameters and latency for the surveyed methods (continued): (b) KITTI 2015 D1-all against reported latency.

Table 2.1: Stereo benchmark datasets with train/test splits, resolutions, and ground-truth types.

| Dataset         | Domain    | Train/Test     | Resolution       | Ground truth       | Ref. |
|-----------------|-----------|----------------|------------------|--------------------|------|
| SceneFlow       | synthetic | 35,454 / 4,370 | 960×540          | dense, rendered    | [9]  |
| KITTI 2012      | driving   | 194 / 195      | 1242×375         | sparse LiDAR       | [41] |
| KITTI 2015      | driving   | 200 / 200      | 1242×375         | sparse LiDAR + CAD | [42] |
| Middlebury 2014 | indoor    | 23 / 10        | up to ~3000×2000 | structured light   | [43] |
| ETH3D           | mixed     | 27 / 20        | ~940×490         | laser scan         | [44] |

### 2.4.3 Accuracy, Parameters, and Latency Across Method Families

Table 2.2 and Figure 2.16 place every method surveyed in this chapter, plus our own model, on a common footing of parameters, SceneFlow end-point error, KITTI 2015 D1-all, and reported latency. Read together with the taxonomy of Section 2.2.7, they turn the family-by-family survey above into a direct comparison. Foundation-scale and iterative systems hold the accuracy frontier, at hundreds of millions and tens of millions of parameters respectively, while the efficient family occupies the low-parameter region at some cost in accuracy: FoundationStereo reaches 0.34 pixel end-point error at roughly 340 million parameters and 470 milliseconds, whereas HITNet reaches 0.43 pixels, a difference of only 0.09 pixels, at 0.97 million parameters and 54 milliseconds, under a hundredth of the latency. Parameter count alone does not order the methods either: PSMNet, at 5.2 million parameters, trails HITNet’s 0.97 million on both accuracy and latency, which shows that the architecture a budget buys matters as much as the size of the budget. CoEx and LightStereo-S sit in the same low-parameter, low-latency region as HITNet, at 2.72 and 3.44 million parameters and 27 and 17 milliseconds, and land within 0.3 pixels of it, so several distinct efficient architectures reach comparable operating points from different technique families. Our own model, StereoLite,

sits at 2.96 million parameters and 49.8 milliseconds with 0.78 pixel end-point error, inside the same efficient cluster but with room to close against HITNet on accuracy.

The accuracy-parameter-latency comparison alone is not sufficient, however, because it says nothing about cross-domain behavior. The review in Section 2.2 showed that two networks of similar synthetic-data accuracy can differ by an order of magnitude in zero-shot outlier rate depending on their training pipeline rather than their parameter count, so a fair comparison across families has to weigh generalization alongside the raw numbers in Table 2.2. Our own design targets this gap directly, sizing the architecture against the frontier established here.

Table 2.2: Parameters, SceneFlow error, KITTI D1-all, and latency for representative methods (values verified against cited papers).

| Method               | Year | Params (M) | SF EPE (px) | KITTI D1 (%) | Latency (ms) |
|----------------------|------|------------|-------------|--------------|--------------|
| PSMNet [11]          | 2018 | 5.2        | 1.09        | 2.32         | 410          |
| HITNet (L) [29]      | 2021 | 0.97       | 0.43        | 1.98         | 54           |
| BGNet [27]           | 2021 | ~2.9       | 1.17        | 2.51         | 25           |
| CoEx [33]            | 2021 | 2.72       | 0.69        | 2.13         | 27           |
| RAFT-Stereo [17]     | 2021 | 11.23      | 0.61        | 1.82         | 380          |
| IGEV-Stereo [18]     | 2023 | 12.60      | 0.47        | 1.59         | 180          |
| LightStereo-S [30]   | 2025 | 3.44       | 0.73        | 2.30         | 17           |
| DEFOM-Stereo [22]    | 2025 | 47.3       | 0.42        | 1.55         | 316          |
| FoundationStereo [1] | 2025 | ~340       | 0.34        | 1.46         | ~470         |
| StereoLite (ours)    | 2026 | 2.96       | 0.78        | n/a          | 49.8         |

## 2.5 Summary

This chapter surveyed the literature that bears on an edge-deployable stereo network and then compared that literature directly rather than treating it as a list of isolated methods. We traced deep stereo matching from the classical four-step pipeline through learned matching costs, three-dimensional cost-volume networks, iterative refinement, and the foundation-model systems that now hold the accuracy ceiling at a computational cost most edge devices cannot meet. From that survey we stated four requirements for edge deployment (compact parameters, real-time latency on roughly four to six tera-operations per second, a small inference working set, and integer-friendly operators) and used them to classify the efficient literature into backbone substitution, cost-volume compression, iterative-loop compression, knowledge distillation, and architectural and adaptive-compute techniques. We compared these families directly: the review of cross-domain generalization showed that both architecture-level and training-recipe remedies narrow the synthetic-to-real gap, and that the relative contribution of iteration versus training pipeline to zero-shot accuracy remains unsettled in the literature; the benchmark review fixed the dual-metric, zero-shot evaluation convention the

rest of this thesis follows; and the combined parameter-accuracy-latency comparison placed our own model on the same footing as every method it is compared against.

## **Chapter 3**

### **Engineering Design and Methodology**

#### **3.1 Introduction**

This chapter presents the engineering design of StereoLite, the compact stereo matching network developed in this thesis, together with the methodology that produced it. We open with the evidence-gated research methodology that governed every design decision, then state the hardware envelope, the design criteria, and the controlled steps by which each candidate was accepted or rejected. From there the chapter builds the system model, traces the conceptual design to its literature ancestry, records the preliminary design decisions that shaped the final network, and works block by block through the detailed design with all governing equations. The chapter closes with the performance indices used to assess the design and the review process behind it. The training objective and the edge-inference optimization belong to the implementation chapter, and the experimental results to the results chapter.

#### **3.2 Research Methodology**

The work follows an evidence-gated design methodology. We admit an architectural choice only after it wins a controlled comparison, and we judge every trained model on accuracy, latency, and memory together rather than on any single metric. Design proceeds by single-knob ablation on a fixed harness of one hundred training pairs, so that competing variants are compared under identical data, seed, and compute. The surviving configuration is then trained on the full dataset on cloud GPUs and evaluated in three ways: in domain, zero-shot on an unseen real benchmark, and under controlled rectification perturbation. It is finally optimized and measured on the target edge device. The steps are:

1. Study of the stereo matching literature and its efficient and edge-deployable families;
2. Definition of the design requirements and the edge deployment envelope;
3. Architecture design by controlled single-knob ablation on a fixed 100-pair harness;
4. Full-dataset training of the selected configuration on cloud GPUs;
5. Evaluation in domain, zero-shot on an unseen real domain, and under rectification perturbation;
6. Optimization of the network for edge inference and on-device latency measurement.

The architecture itself was not invented from a blank page. From the surveyed accuracy frontier we adopted a small set of mechanisms, each chosen because the surveyed evidence

shows it delivers a disproportionate share of its parent method’s accuracy at a cost the edge envelope can absorb. The conceptual design later in this chapter details how each mechanism was adapted; the point here is that adoption was itself gated on evidence, not on preference. This division of labor, evidence for selection and controlled comparison for tuning, is what keeps the design auditable decision by decision.

Figure 3.1 summarizes this pipeline end to end, including the two feedback paths: a rejected ablation arm returns to the harness, and a failed evaluation reopens the design itself.

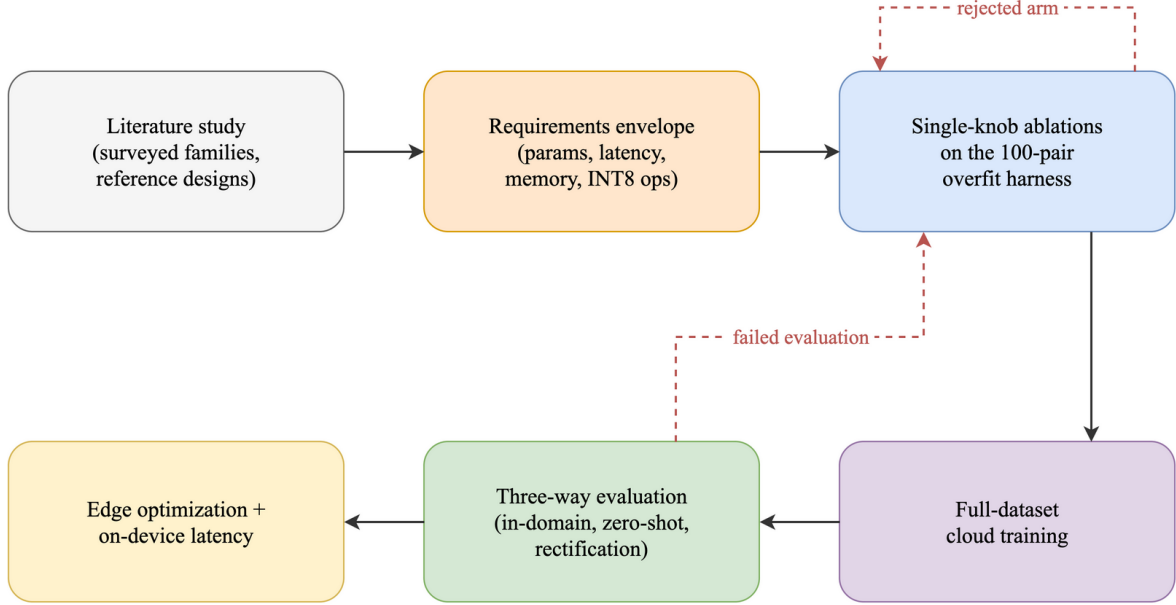


Figure 3.1: Evidence-gated design methodology from literature through ablation, training, and evaluation to edge deployment.

### 3.3 Design Requirements and Requisites

The design requirements follow from the class of edge accelerators found on mobile robots and drones. Together they form a hard envelope, and we reject a candidate that violates any one of them regardless of accuracy. The network must stay under three million parameters, so that it is compact enough for on-device memory and storage; parameter count is also the first-order proxy for a deployed model’s download and update footprint. It must run in real time on the target edge accelerator, an NVIDIA Jetson Orin Nano, which delivers 20 to 40 tera-operations per second of eight-bit throughput [45], so a forward pass must fit inside a live camera stream’s frame budget on that device. Its inference working set at the deployment precision must stay under 200 megabytes, small enough to coexist with the rest of a robot’s software stack on a shared-memory edge board, though a higher-precision development GPU may use more. Finally, every operator on the inference path must quantize cleanly to eight-bit integer arithmetic, since the accelerators in this class reach their rated throughput only under integer execution.

### 3.4 Design Criteria

Within the envelope above, we judge candidate designs on balance rather than on any single number. Every design decision is evaluated on the full eight-metric stereo set: end-point error, root-mean-square error, median absolute error, the bad-0.5, bad-1, bad-2, and bad-3 rates, and the D1-all rate, together with inference latency and peak memory. These quantities are defined formally in Section 3.10. A variant that improves one metric while degrading the others, or that buys accuracy with latency the envelope cannot afford, is not accepted. Because in-domain gains can mask a loss of generalization, we require a cross-dataset zero-shot evaluation after every training run. All comparisons are made under equal conditions, the same training pairs, random seed, and step budget, so that a measured difference is attributable to the design change alone.

### 3.5 Design Steps

The architectural choices in this chapter were made under a controlled matched-overfit protocol rather than intuition. We train each candidate on a fixed set of SceneFlow pairs with a fixed seed, deterministic convolution kernels, and an identical step budget and evaluation protocol, and we report all eight stereo metrics plus an inference latency benchmark for every arm. Single knobs are varied one at a time, the winning criterion is registered before a run so the outcome cannot be rationalized after the fact, and every candidate is checked against a held-out cross-dataset evaluation, so that an in-domain gain which fails to transfer is not mistaken for progress. This section states the protocol; the runs it produced appear in the results chapter, and the mapping from each decision to its supporting run is collected in the preliminary design below.

### 3.6 Modeling of Proposed System

Figure 3.2 gives the complete pipeline. StereoLite is a coarse-to-fine tile-hypothesis network with light recurrent refinement. A shared feature encoder processes both views and a separate left-image context encoder feeds the recurrent state; a group-wise cost volume at the coarsest scale initializes a slanted tile state; convolutional GRU updates refine the state across three scales with plane-aware propagation between them; a narrow-band geometry encoding volume corrects the estimate through a learned fail-soft gate; and plane rendering with learned convex upsampling recovers the full-resolution disparity.

Table 3.1 summarizes the spatial sizes and the main operation at each pipeline scale for the default 384 by 640 input.

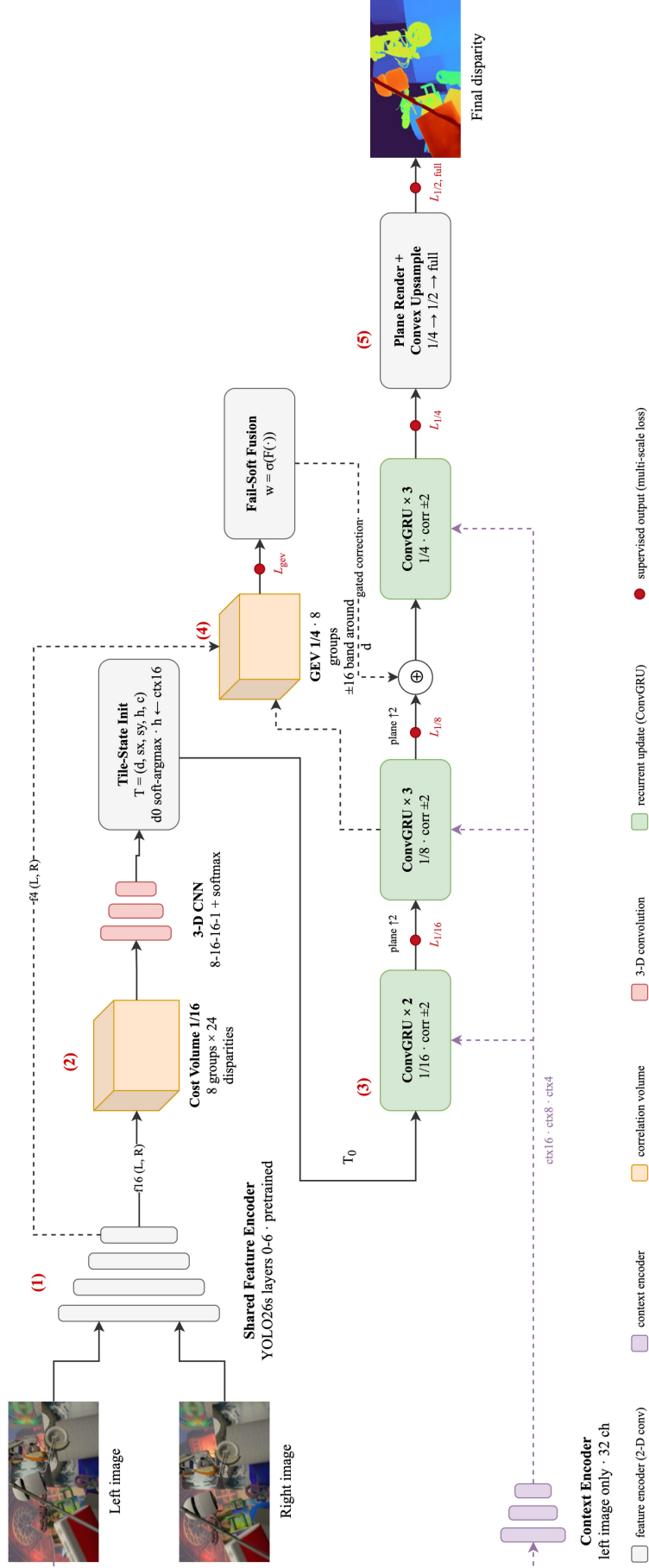


Figure 3.2: StereoLite architecture: five numbered stages with supervised outputs marked by red dots.

Table 3.1: Spatial size, channel count, and main operation at each pipeline scale.

| Scale | Spatial size     | Channels | Main operation                        |
|-------|------------------|----------|---------------------------------------|
| Full  | $384 \times 640$ | 3        | input and output                      |
| 1/2   | $192 \times 320$ | 32       | convex-upsample guidance              |
| 1/4   | $96 \times 160$  | 128      | GEV fusion and 3 refinements          |
| 1/8   | $48 \times 80$   | 256      | 3 refinements                         |
| 1/16  | $24 \times 40$   | 256      | cost initialization and 2 refinements |

### 3.7 Modeling of Conceptual Design

Conceptually, the pipeline is an assembly of five stages, each of which adapts a mechanism from the literature review to the edge budget. We did not adopt these mechanisms because they are recent, but because the surveyed evidence shows each one carries a large share of its parent method’s accuracy at a cost our envelope can afford. The detailed design of each stage follows in Section 3.9; here we set out the reasoning behind each choice.

The matching cost is the group-wise correlation volume of GwcNet [12], given in (2.8). It carries more matching evidence than a plain correlation yet costs a small fraction of a concatenation volume, and it degrades gracefully when the three-dimensional aggregation behind it is cut to a few layers, which is exactly the regime a compact network operates in. We restrict it to a single coarse scale, which keeps the peak activation memory within the small inference working set the envelope demands.

The refinement operator and the final upsampling come as a pair from RAFT-Stereo [17], given in (2.9): a convolutional gated recurrent unit reused every iteration, and a learned convex combination that restores full resolution from a coarse estimate with image-derived weights instead of a dense fine-scale volume. The recurrent update concentrates capacity in a small module, so accuracy scales with compute rather than with parameters. We run the pair for only a handful of iterations at coarse resolution, which fits the real-time budget that rules out the many-iteration desktop operating points.

The third mechanism is the geometry encoding volume of IGEV-Stereo [18], given in (2.10). Its warm-start disparity saves the early iterations that a cold-started recurrent loop wastes, which is the single largest lever for cutting iteration count, and the surveyed ablations show a handful of warm-started iterations matching dozens of cold ones. In our design a learned gate blends this global proposal into the recurrent estimate, so the volume corrects rather than competes.

The fourth mechanism is the slanted tile-plane representation of HITNet [29]. A grid of tiles, each carrying a disparity, two plane slopes, and a descriptor, replaces the dense cost volume as the working state of the network, and the slopes preserve sub-pixel structure across scales without rebuilding a volume at every level. This is the most aggressive of the surveyed cost-volume compressions, and its published operating point, under one million

parameters at real-time latency, sits squarely inside our envelope.

The fifth choice is structural rather than borrowed from a single method: the matching path and the recurrent context are kept in separate streams. A shared feature encoder feeds the matching cost, and a dedicated left-image context encoder feeds the recurrent state. If the matching features also served as context, the recurrent unit could not separate appearance from correspondence; the RAFT-Stereo lineage [17] established that a dedicated context path improves refinement, and we adopt the same separation. What the surveyed literature does not directly address is the specific combination these mechanisms form, a tile-hypothesis network of a few million parameters that fuses a geometry encoding volume through a fail-soft gate, measured for near real-time inference on the Jetson Orin Nano class of hardware and characterized for robustness to imperfect rectification. That gap is the design problem this thesis takes up.

### 3.8 Preliminary Design

The final architecture was reached through a sequence of preliminary variants, each accepted or rejected under the design-steps protocol above. Every load-bearing decision in this chapter is tied to a controlled study in the results chapter or to published evidence in the literature review. Table 3.2 collects this mapping so the design can be audited decision by decision; it is the record of preliminary designs, including the rejected ones, that shaped the final system.

Table 3.2: Architectural and training decisions with their supporting evidence.

| Decision                            | Evidence                                                                           |
|-------------------------------------|------------------------------------------------------------------------------------|
| YOLO26s shared encoder              | encoder ablation: best EPE of three encoders at +2 ms                              |
| Separate left context encoder       | RAFT-Stereo lineage [17]; context-gated variants win the family ablation           |
| Group-wise cost at one coarse scale | GwcNet [12] graceful degradation under small aggregation budgets                   |
| Iterated coarse-to-fine refinement  | refinement ablation: single-pass collapses, finer scales add cost without gain     |
| Local $\pm 2$ correlation lookup    | CREStereo [19]: local search beats all-pairs; also confers rectification tolerance |
| Slanted tile planes                 | HITNet [29]; plane rendering wins the blur study                                   |
| Narrow GEV band + fail-soft gate    | family ablation best EPE; narrowing accuracy-neutral                               |
| Rejected: selective recurrent unit  | family ablation: EPE worsens on the compact chassis                                |
| Rejected: fresh 1/4 initialization  | family ablation: EPE and bad-1 worsen                                              |
| Rejected: fix compositions          | blur study: worse than plane rendering alone                                       |
| Threshold-stack + D1 objective      | loss ablation: wins EPE, bad-2, D1 together                                        |
| Native-crop input protocol          | input-protocol study: native axis EPE 6.67 to 2.87 at no resized-axis cost         |
| Augmentation triplet                | matched comparison: best val EPE 2.778 to 1.921                                    |
| OneCycle 60k schedule               | training dynamics: validation floor reached before the budget ends                 |

### 3.9 Detailed Design

This section works through each pipeline block in the order data flows through it, together with the governing equations for each stage. Read at a high level before the detail, the pipeline does five things in sequence. It first turns the two raw images into features a network can compare (Section 3.9.1). It then makes a coarse, low-resolution guess at the disparity of every image tile by testing a range of candidate horizontal shifts and keeping the one whose left and right features agree best (3.9.2). That guess is usually wrong in the details, so a recurrent loop repeatedly looks at how well the current guess lines up the two views and nudges it closer, scale by scale, carrying the estimate from coarse tiles to progressively finer ones as it goes (3.9.3-3.9.4). Because that loop only ever looks at a small neighborhood around each tile, a second, wider-reaching check is added purely to correct the tiles the loop is still struggling with, without disturbing the ones it has already resolved well (3.9.5). Finally the coarse tile grid is rendered and sharpened up to the full image resolution (3.9.6), and the resulting disparity map is converted into an actual three-dimensional point cloud (3.9.7). Figure 3.2 already showed this same flow as a numbered block diagram; the subsections below give each block’s mechanism and governing equations in turn.

Figure 3.3 makes the same process concrete by running the trained network on a real stereo pair and showing the disparity map as it actually leaves each stage. The coarse initialization at one sixteenth resolution is blocky and only roughly right; each later stage sharpens it, until the full-resolution output resolves the motorcycle’s wheels, frame, and background boxes cleanly. The panel labelled *GEV proposal* and the fail-soft gate beside the inputs preview the correction step of Section 3.9.5, where a second, independent estimate is blended in only where the gate opens. Reading the subsections below alongside this figure shows both what each block does and what it contributes to the final disparity.

#### 3.9.1 Shared Feature Encoder and Context Encoder

Before the two views can be compared, each has to be turned into features a network can match, and the left view alone has to supply the running context the recurrent refinement in Section 3.9.3 will need. This subsection builds those two encoders: one shared between the left and right image so both are described in the same feature space, and one private to the left image alone.

The matching network is a truncated YOLO26s backbone whose first seven blocks are retained, producing features at one quarter, one eighth, and one sixteenth resolution with 128, 256, and 256 channels. The choice of a detection backbone is deliberate: its features are pretrained to localize objects across scales, which is close to what stereo matching needs from an encoder, and the encoder ablation confirms that this backbone beats both a hand-built GhostConv stack and its own smaller sibling on every accuracy metric at a two-millisecond cost. We remove the detection neck and head above the one-sixteenth stage

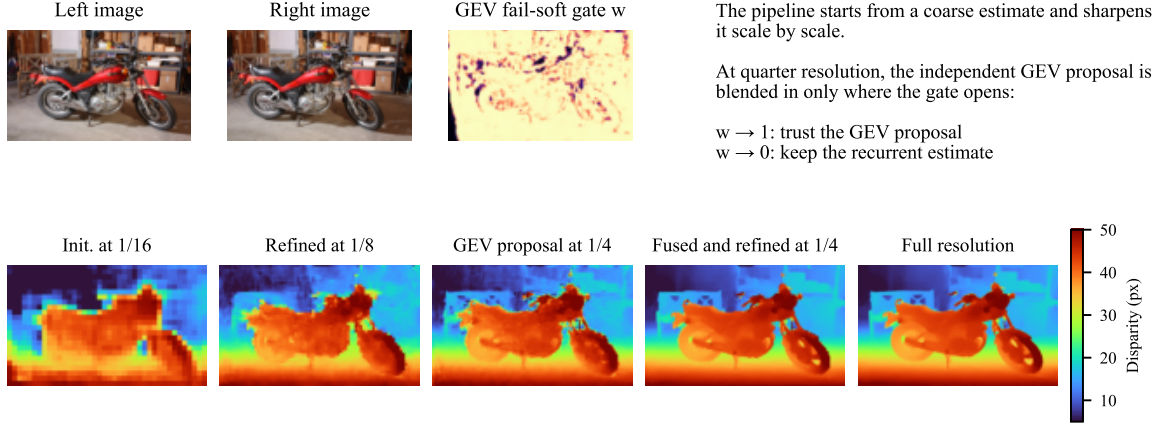


Figure 3.3: Disparity as it leaves each stage of the trained network, run on a real Middlebury 2014 pair (Motorcycle [43]). Every panel is an actual model output, upsampled to full resolution and shown on one shared disparity scale; the estimate sharpens from the coarse one-sixteenth initialization to the full-resolution result. The gate  $w$  (top right) shows where the GEV proposal is trusted.

at construction rather than leaving them unused, because dead blocks would waste parameters and forward compute. The two views are concatenated along the batch dimension and pass through a single shared invocation, so both images are processed with identical weights. A separate context encoder processes the left image alone: a strided convolution is followed by two GhostConv blocks with squeeze-and-excitation, and the result is projected to a 32-channel quarter-resolution context map that is pooled to the coarser scales. We keep matching evidence and recurrent context in separate streams for a specific reason: if the matching features also served as context, the recurrent unit could not separate appearance from correspondence. The RAFT-Stereo lineage [17] established that a dedicated context path improves refinement, and we adopt the same separation.

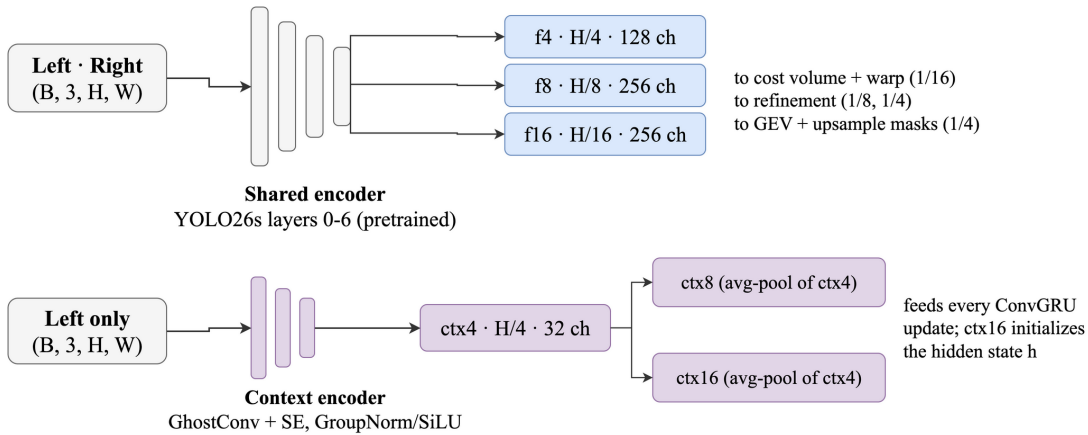


Figure 3.4: Shared YOLO26s matching encoder and separate left-image context encoder.

### 3.9.2 Tile Initialization and Cost Volume

This subsection produces the network’s first, coarse guess at disparity. The idea is the same one every stereo method relies on: for a small patch of the left image, try a range of candidate horizontal shifts, compare each shifted patch of the right image against it, and keep the shift whose features agree best. Doing this once, cheaply, at a coarse resolution gives every tile a starting point that later stages only need to correct rather than discover from nothing.

The tile initialization module builds a group-wise correlation volume, following GwcNet [12], at one sixteenth resolution. For each of eight feature groups and each integer disparity, the matching score is the mean channel-wise product between the left features and the horizontally shifted right features, as given in (3.1). A small three-dimensional convolutional network with an eight, sixteen, sixteen, one channel schedule regularizes the volume, and a softmax along the disparity axis yields a probability distribution. The initial disparity and confidence come from the GC-Net-style soft-argmin [10] and the distribution peak, as in (3.2). The plane slopes are initialized to zero and the hidden feature is replaced by the coarsest context map. The 24 hypotheses at this scale span roughly 0 to 368 full-resolution pixels.

$$C(g, d, y, x) = \frac{1}{C/G} \sum_{c=1}^{C/G} f_L^{g,c}(y, x) f_R^{g,c}(y, x - d) \quad (3.1)$$

where  $G = 8$  is the number of feature groups,  $C$  the channel count,  $d \in \{0, \dots, 23\}$  the integer disparity hypotheses at one sixteenth resolution, and  $f_L^{g,c}, f_R^{g,c}$  the  $c$ -th channel of group  $g$  of the left and right feature maps.

$$d_0 = \sum_d p(d) d, \quad c_0 = \max_d p(d) \quad (3.2)$$

where  $p(d)$  is the softmax of the regularized cost volume along the disparity dimension,  $d_0$  the expected disparity, and  $c_0$  the peak probability that serves as an initial confidence.

### 3.9.3 Recurrent Tile Refinement

The initial guess from Section 3.9.2 is coarse and often wrong in the details, so this subsection corrects it iteratively rather than in one shot. At each step, the network checks how well the current disparity estimate lines up the two views by warping one onto the other and looking at the local mismatch, then nudges the estimate, the local surface tilt, and its own confidence a little closer to correct. Repeating this update, and carrying a persistent memory of what has already been tried between repetitions, lets the network spend more computation on tiles that are hard to match and less on tiles it already got right, rather than paying the same fixed cost everywhere.

Separate refinement modules refine the tile state at one sixteenth, one eighth, and one

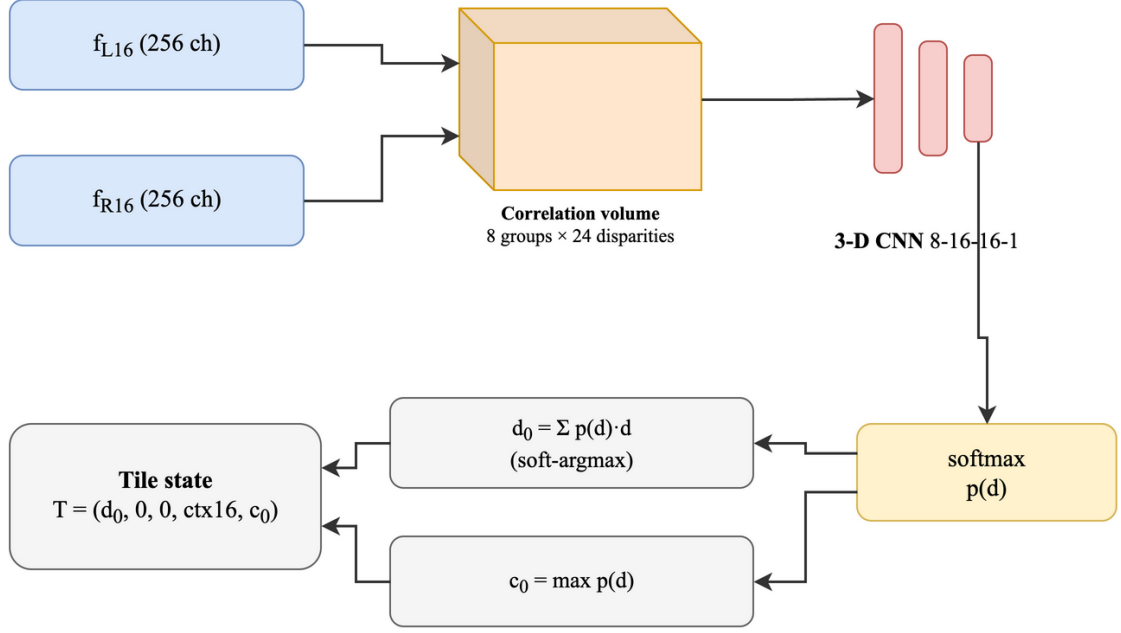


Figure 3.5: Tile-state initialization: group-wise correlation volume regularized by 3-D CNN with soft-argmax outputs. The 24 hypotheses at 1/16 scale cover roughly 0 to 368 full-resolution pixels.

quarter resolution; their weights are not shared, because the matching-feature channel counts differ across scales. At each iteration the right feature map is warped toward the left view by the current disparity, and a differentiable local correlation, in the manner of CREStereo’s local search [19], is evaluated over five offsets within a half-range of two pixels. The recurrent input assembles the left features, the warped right features, the current state, the local correlation, and the context, following the context-conditioned input RAFT-Stereo feeds to its recurrent unit [17], as in (3.3). A convolutional gated recurrent unit, following RAFT-Stereo [17], then updates the persistent hidden state through the gate equations of (3.4). A two-layer head with 48 hidden channels predicts residuals for disparity, both plane slopes, and confidence, and a second learned gate emits four independent sigmoid weights that scale each residual. A softplus constrains disparity to remain non-negative. The schedule runs two iterations at one sixteenth, three at one eighth, and three at one quarter, for eight learned updates in total.

$$x = [f_L, \mathcal{W}(f_R; d), d, s_x, s_y, c, \text{corr}, \text{ctx}] \quad (3.3)$$

where  $\mathcal{W}(f_R; d)$  denotes the right features warped by the current disparity, corr the five-offset local correlation, and ctx the scale-matched context map.

$$\begin{aligned} z &= \sigma(\text{Conv}([h, x])), & r &= \sigma(\text{Conv}([h, x])), \\ q &= \tanh(\text{Conv}([r \odot h, x])), & h' &= (1 - z) \odot h + z \odot q, \end{aligned} \quad (3.4)$$

where  $h$  is the 32-channel persistent hidden state,  $z$  and  $r$  the update and reset gates,  $q$  the

candidate state, and  $\odot$  the element-wise product.

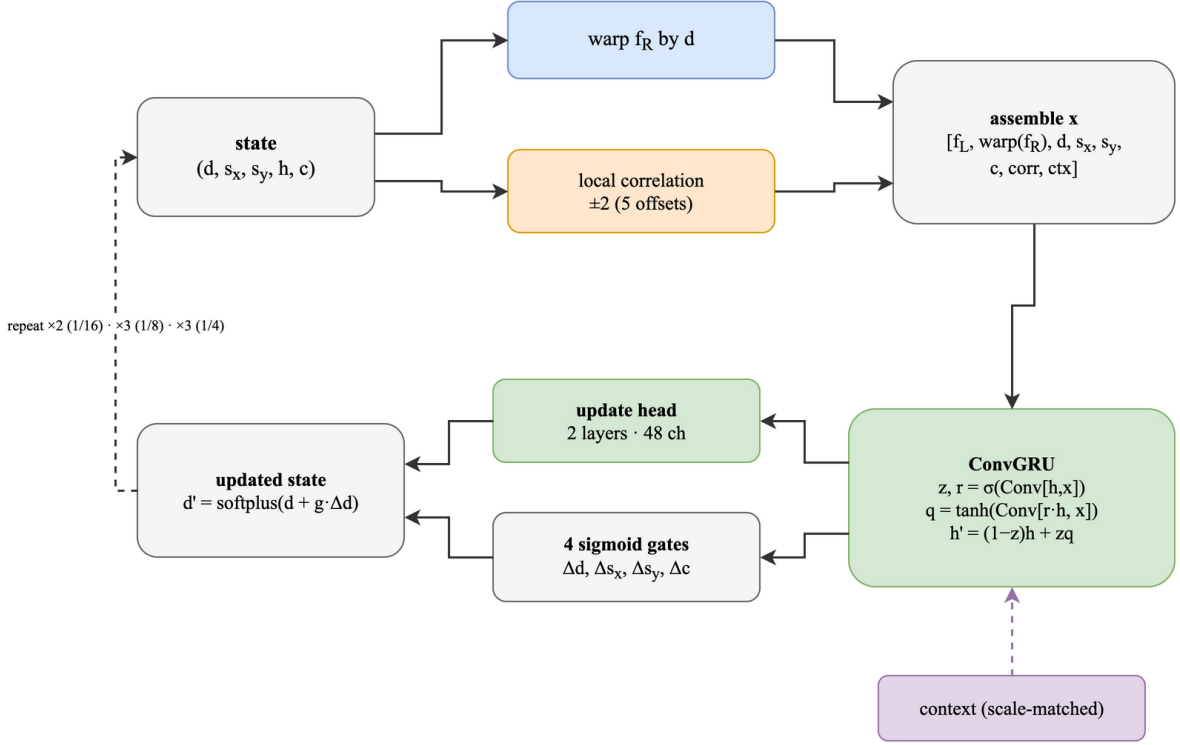


Figure 3.6: One recurrent refinement iteration with warping, local correlation lookup, and gated residual updates.

### 3.9.4 Plane-Aware Cross-Scale Propagation

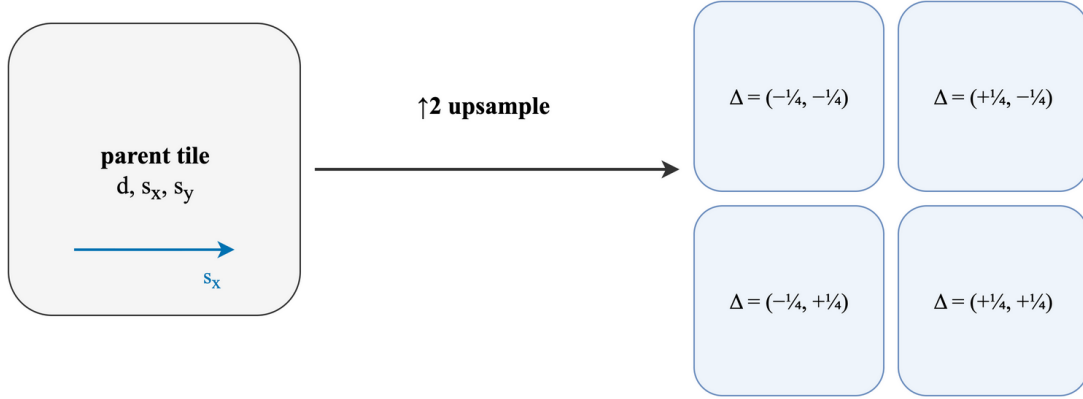
Refinement at one resolution has to hand its result up to the next, finer resolution before refinement can continue there, and a generic upscaling would just blur the coarse tile boundaries across the new, smaller tiles. This subsection is the hand-over step, and it does better than plain upscaling because it also carries forward the local surface tilt each tile already estimated, so a slanted surface is placed correctly at the finer grid instead of being flattened and then re-discovered from scratch.

Between adjacent scales, the propagation module resamples the disparity, slopes, hidden feature, and confidence to the finer grid. Disparity is multiplied by two because its pixel unit changes with resolution. Unlike plain bilinear interpolation, the propagated disparity also uses the local plane slopes to place the four child pixels around each parent tile, as given in (3.5). This tile-plane component, in the manner of HITNet [29], preserves a local slanted-surface hypothesis while moving to a finer grid; sub-pixel structure is retained without rebuilding a cost volume at every level.

$$d_{\text{child}} = 2 \mathcal{B}(d_{\text{parent}}) + 2 s_x \Delta x + 2 s_y \Delta y, \quad \Delta x, \Delta y \in \{-0.25, +0.25\} \quad (3.5)$$

where  $\mathcal{B}$  is bilinear resampling of the parent disparity,  $s_x$  and  $s_y$  the local plane slopes, and

$\Delta x, \Delta y \in \{-0.25, +0.25\}$  the child offsets; the factor of two rescales the disparity to the finer grid.



$$d_{\text{child}} = 2 \cdot \text{bilinear}(d) + 2 s_x \Delta x + 2 s_y \Delta y$$

Figure 3.7: Plane-aware cross-scale propagation using local slopes instead of bilinear interpolation. The slopes shift each child off the parent plane, and the factor of two rescales the disparity to the finer grid.

### 3.9.5 Geometry Encoding Volume and Fail-Soft Fusion

The recurrent loop in Section 3.9.3 is a local process: every update only looks at a small neighborhood around the current estimate, so a tile whose initial guess started far from the truth can stay stuck near that bad guess no matter how many iterations run. This subsection adds a second, independent estimate with a wider reach, used only to pull those stuck tiles toward a better value. It is deliberately not allowed to overrule tiles the recurrent loop has already resolved well: a learned gate decides, tile by tile, how much weight this second opinion gets, and it learns to give it almost none where the recurrent estimate is already trustworthy.

The geometry encoding volume adds an independent global proposal at one quarter resolution, in the spirit of IGEV-Stereo [18]. In the trained configuration it is narrow: rather than the full 64 disparity planes of the base design, it samples 33 hypotheses in a band of plus or minus 16 around the current tile disparity. The efficiency study validates this change as accuracy-neutral, and it also removes roughly a quarter of the model’s multiply-accumulate operations. The band is less restrictive than it sounds: 16 quarter-resolution planes correspond to 64 full-resolution pixels on either side of the current estimate, and since the coarse initialization has already localized the disparity, the band’s job is correction, not search. The full-range volume spends most of its computation scoring hypotheses the tile state has already ruled out, so removing them costs nothing. The volume is regularized by three three-dimensional convolutions with 16 hidden channels, and the softmax distribution yields the

expected disparity, a confidence, and a projected geometry feature, following IGEV-Stereo’s warm-start soft-argmin [18], as in (3.6). The proposal is not substituted unconditionally: a learned gate takes the context, the geometry feature, both confidences, the absolute disagreement, and the current disparity, and produces a scalar weight through (3.7). We initialize the gate’s final bias to minus four, so the initial weight is approximately 0.018 and training begins close to the stable recurrent baseline; the network then learns where the geometry proposal helps. This fail-soft design stops the global branch from corrupting regions the recurrent path already resolves well.

$$d_{\text{gev}} = \sum_d p(d) d, \quad c_{\text{gev}} = \max_d p(d), \quad g_{\text{gev}} = \phi\left(\sum_d G(d) p(d)\right) \quad (3.6)$$

where  $p(d)$  is the softmax over the regularized quarter-resolution volume,  $G(d)$  the regularized three-dimensional geometry feature, and  $\phi$  a projection to a 16-channel two-dimensional expected-geometry map.

$$w = \sigma\left(F([\text{ctx}_4, g_{\text{gev}}, c, c_{\text{gev}}, |d_{\text{gev}} - d|, d])\right), \quad (3.7)$$

$$d_{\text{fused}} = \text{softplus}(d + w(d_{\text{gev}} - d)),$$

where  $F$  is the fusion network whose final-layer bias is initialized to  $-4$ ,  $c$  and  $c_{\text{gev}}$  the tile and geometry confidences, and  $w$  the blend weight; the slopes are attenuated by  $(1 - w)$  and the confidence becomes the maximum of the two estimates.

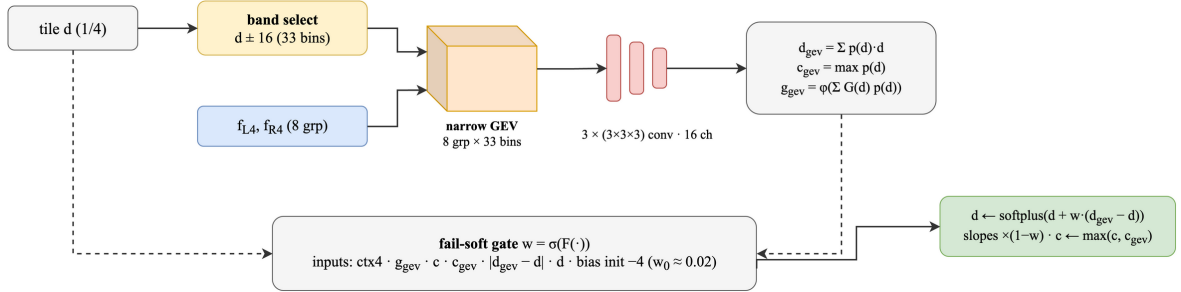


Figure 3.8: Narrow-band geometry encoding volume fused into tile estimates via learned fail-soft gate.

### 3.9.6 Plane Rendering and Learned Convex Upsampling

Every stage so far has worked on a tile grid coarser than the final output, at one quarter of the image resolution at best, so the last step is to turn that coarse grid into a full-resolution disparity map. A naive upsampling would blur object edges across tile boundaries; this subsection instead uses the slanted-plane estimate each tile already carries to reconstruct a sharp surface within it, and a learned weighting scheme, rather than fixed bilinear interpolation, to combine neighboring tiles so that boundaries stay crisp where the scene actually has a depth discontinuity.

The final one-quarter disparity is upsampled to full resolution in two learned two-times stages. The first stage predicts masks from the one-quarter left features and the second from the one-half left features. For every output subpixel, a mask predicts nine weights over the three-by-three coarse neighborhood, and a softmax makes the weights non-negative and sum to one, so the upsampled disparity is a convex combination of nearby coarse values. This is the same convex upsampling used in RAFT-Stereo [17]. Before upsampling, a plane-rendering step reconstructs each tile from its disparity and slopes and applies an edge gate; this was the decisive fix in the boundary-sharpness study, where it reduced the bad-0.5 rate by roughly 22 percent relative to the control. Convex upsampling can select different source neighbors near a boundary, but it cannot invent a disparity hypothesis absent from the one-quarter prediction, so thin and distant structures remain a limitation.

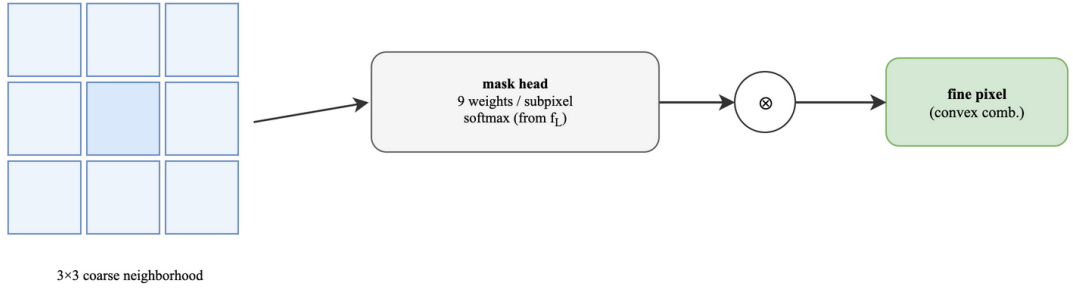


Figure 3.9: Learned convex upsampling via softmax-weighted 3x3 neighborhoods to full resolution. Two 2 $\times$  stages (1/4 to 1/2 to full) each multiply disparity by two, with non-negative weights that sum to one.

### 3.9.7 From Disparity to 3D Reconstruction

The disparity map itself is an intermediate result; the goal of the pipeline is the three-dimensional structure of the scene that the disparity encodes. Once the disparity is known, every valid pixel back-projects to a metric three-dimensional point through the pinhole relations

$$Z = \frac{fB}{d}, \quad X = \frac{(u - c_x)Z}{f}, \quad Y = \frac{(v - c_y)Z}{f} \quad (3.8)$$

where  $(u, v)$  are the pixel coordinates,  $(c_x, c_y)$  the principal point,  $f$  the focal length in pixels,  $B$  the stereo baseline, and  $(X, Y, Z)$  the recovered point in the camera frame. Each point is colored with the corresponding left-image pixel, which yields a dense colored point cloud of the scene. For the reconstructions reported in the results chapter, the cloud is post-processed with two standard operations: a statistical outlier filter removes the isolated flying points that appear along depth discontinuities, and a voxel downsampling makes the point density uniform. The stereo rig’s calibrated values, a focal length of approximately 1005 pixels at the native 1280-pixel width and a baseline of 52 millimeters, scale to the inference resolution by the width ratio. This reconstruction path is what the thesis title refers to: the network pro-

vides the dense disparity, and the calibrated geometry turns it into a metric three-dimensional model of the scene.

### 3.10 Design Assessment and Performance Indices

We report accuracy with the standard stereo metrics in (3.9): end-point error is the mean absolute disparity error over valid pixels, bad- $t$  is the percentage of valid pixels whose absolute error exceeds  $t$  pixels, and D1-all is the KITTI outlier definition [42]. We also report root-mean-square error and median absolute error: the first is sensitive to large outliers, while the second gives an outlier-robust summary. Each image contributes its own averaged metric, so scores are averaged per image rather than pooled across every pixel. Latency is measured at batch one, 384 by 640 resolution, with ten warm-up and a hundred timed passes; memory is the peak inference working set, and the device used is stated throughout.

$$\begin{aligned} \text{EPE} &= \frac{1}{|V|} \sum_{p \in V} |\hat{d}(p) - d^*(p)|, & \text{bad-}t &= \frac{100}{|V|} \left| \{p \in V : |\hat{d}(p) - d^*(p)| > t\} \right|, \\ \text{D1} &= \frac{100}{|V|} \left| \{p \in V : |\hat{d}(p) - d^*(p)| > 3 \wedge |\hat{d}(p) - d^*(p)| > 0.05 d^*(p)\} \right| \end{aligned} \quad (3.9)$$

where  $V$  is the set of valid pixels,  $\hat{d}$  the predicted disparity,  $d^*$  the ground truth, and  $t \in \{0.5, 1, 2, 3\}$  pixels the bad-rate thresholds.

Alongside accuracy, latency, and memory, the parameter budget is itself a performance index, bounded by the envelope in Section 3.3. Figure 3.10 reports the module-wise split of the trained model; the split is computed directly from the checkpoint’s state dictionary.

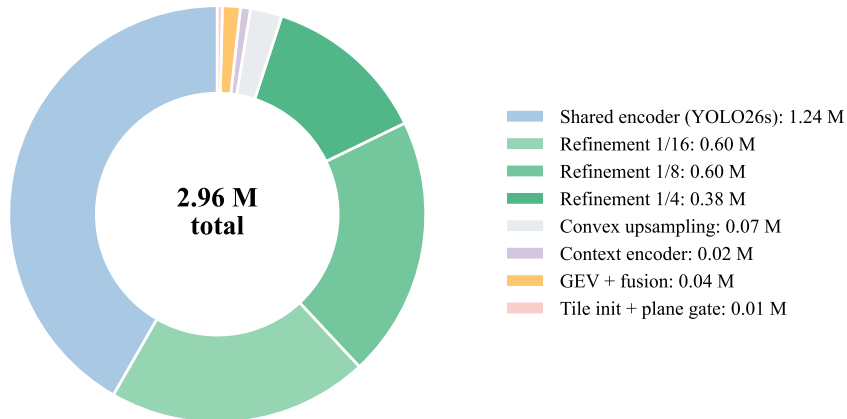


Figure 3.10: Module-wise parameter distribution: encoder and refinement stages hold 95% of 2.96 M parameters.

These indices did not merely score the finished model; they governed the design as it was built. We followed a review-and-feedback process rather than unchecked iteration: every proposed change, whether architectural, a loss term, or an input protocol, had to win a controlled ablation on the matched harness described above before entering the production configuration, and a change that lost its comparison was not adopted no matter how plausible it looked. Every run, win or lose, went into a master experiments log with its full configuration and metrics, so the evidence behind each decision is auditable and no comparison depends on memory. We reviewed training curves, qualitative panels, and ablation tables with the supervisor at regular intervals, and that feedback redirected the work more than once, most visibly toward the cross-dataset evaluation gate that now follows every training run. Failed variants were documented rather than discarded: the rejected mechanisms and anti-synergistic compositions recorded in the results chapter constrain future work as firmly as the adopted ones justify the present design.

### 3.11 Summary

This chapter set out the engineering design of StereoLite and the methodology that produced it. We work under an evidence-gated methodology: an architectural choice is admitted only after it wins a controlled single-knob comparison on a fixed harness, and every model is judged on accuracy, latency, and memory together with a mandatory cross-dataset zero-shot evaluation. The design obeys a hard envelope: under 3 million parameters, real-time operation on a 4 to 6 TOPS accelerator, under 200 megabytes of inference memory, and INT8-exportable operators. Within that envelope we assembled StereoLite from five mechanisms adapted from the surveyed literature, a group-wise cost volume, recurrent tile refinement with plane-aware cross-scale propagation, a narrow-band geometry encoding volume with a fail-soft gate, and plane rendering with learned convex upsampling, fed by a shared matching encoder and a separate context encoder. Each block was specified with its governing equations, and each design decision is auditable through the evidence table and the master experiments log.

## Chapter 4

### System Development and Implementation

#### 4.1 Introduction

A design on paper only becomes useful once it is written as code, trained until it stops improving, and made fast enough to run on the hardware it was meant for. This chapter is about that step. It turns StereoLite from a fixed architecture into a working system and records, in one place, everything needed to rebuild it: the software that trains and evaluates the network, the exact settings the final model uses, the work done to bring one forward pass inside the target speed and memory budget, the computing and camera hardware involved, and what actually happens inside the network when a rectified image pair goes in and a disparity map comes out.

The emphasis here is on what was built, not on the numbers it produced; those belong with the results. Each value that matters, how the network is sized, how it is trained, and what is changed to run it in eight-bit on an edge board, is pinned down once so that every result reported afterwards rests on a configuration a reader can see in full and reproduce.

#### 4.2 Software Development

The network is implemented in PyTorch 2.11 with CUDA 12.8 on Python 3.10. We organized the codebase around scripted drivers rather than notebooks: a separate script trains the production model, separate harnesses run the encoder, refinement, and loss ablations under a matched protocol, and separate drivers run the SceneFlow held-out evaluation and the zero-shot Middlebury 2014 cross-domain evaluation. Each driver writes its full configuration and its metrics to a metadata file next to its outputs, so a reported number can be traced back to the exact run that produced it.

Two cloud platforms carry the compute load. Full-data training and the production run described in Section 4.3 ran on Modal-orchestrated cloud A100 jobs, while ablations and cross-domain evaluations ran on cloud T4 instances, which are cheap enough to run after every design change without straining the project budget. An earlier development pass on the edge-tier chassis used a self-contained Kaggle notebook with data-parallel training across two T4 GPUs; the production run reported in this thesis moved to the single-A100 Modal path once the architecture and loss were fixed, since a single larger GPU trains the full SceneFlow corpus faster than two smaller ones split across processes. Keeping training, ablation, and evaluation as separate scripted drivers, each logging its own metadata, is what let every reported number trace back to a specific run rather than to a notebook cell that may since have changed.

### 4.3 Network Implementation

The design chapter argued for each architectural family; this section records the exact values the production model uses. The backbone is a YOLO26s encoder truncated after its first seven blocks. The recurrent hidden state carries 32 channels, and the refinement head that reads it carries 48. The tile initialization module builds its correlation volume from eight groups over 24 disparity hypotheses at one sixteenth resolution, and the per-iteration local correlation looks up a half-range of two pixels around the current disparity estimate. The refinement schedule runs two iterations at one sixteenth resolution, three at one eighth, and three at one quarter, and the geometry encoding volume is narrowed to a band of plus or minus 16 disparities around the tile estimate rather than spanning the full disparity range. Table F.1 of Appendix F gives the complete configuration record of the trained checkpoint, verbatim, alongside these headline values.

Table 4.1 lists the training-side configuration that Chapter 3 selected and justified: the optimizer, schedule, batch size, input protocol, augmentation set, training data, validation protocol, and training hardware. Every value in the table is the result of a controlled ablation, and the design-evidence audit of Chapter 3 traces each choice to its justification. Section 4.5 below fixes the training objective and the protocol narrative behind this table in full.

Table 4.1: Training configuration of the production run (run 20260704).

| Element                | Value                                                   |
|------------------------|---------------------------------------------------------|
| Optimizer              | AdamW, weight decay $10^{-5}$                           |
| Schedule               | OneCycle, 60,000 steps, peak LR $8 \times 10^{-4}$      |
| Batch size / precision | 32 / bfloat16 mixed                                     |
| Input protocol         | random native $384 \times 640$ co-located crops         |
| Augmentation           | asymmetric color jitter, right-only erase, random scale |
| Training data          | full SceneFlow finalpass, 35,454 pairs                  |
| Validation             | fixed 400-pair FlyingThings3D subset, every 1k steps    |
| Hardware               | one NVIDIA A100 80 GB (cloud)                           |

### 4.4 Optimization and Configuration Selection

Reaching the latency target required optimizing the network’s inference graph, not just training it. We applied six changes: we rewrote the cost-volume construction loops to use padded zero-copy views instead of allocating and zeroing a feature map for every disparity bin; we reduced the grid-sample calls from 48 to eight by batching the lookups and caching the sampling grids; we hoisted the static channels of the recurrent input, which are re-convolved every iteration, to run once per scale, following the precomputed-context trick of RAFT-Stereo [17]; we fused the gate convolutions and the four output heads, which share inputs; we removed dead inference-path code; and we narrowed the geometry volume to a band

around the tile disparity. We proved each change equivalent before adopting it, either bitwise or through a matched A/B comparison, and together they cut the full-precision latency by a factor of 1.74.

Figure 4.1 lays the deployment path out: the four graph optimizations act on the full-precision model before export, while the three operator swaps prepare the graph for eight-bit calibration.

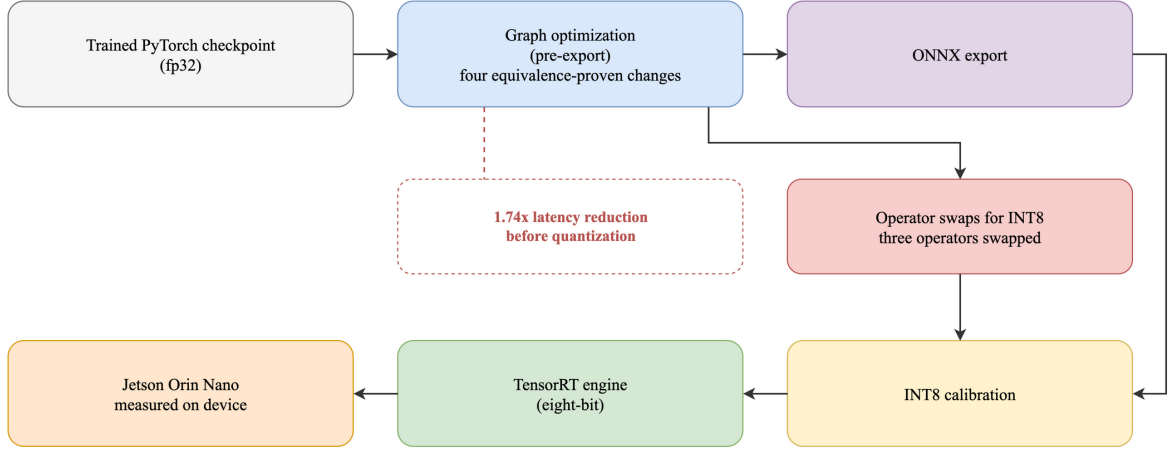


Figure 4.1: Export pipeline from trained checkpoint to INT8 TensorRT engine.

The same pass also had to clear three operators that block eight-bit deployment: the unfold operation in convex upsampling, group normalization on some accelerator toolchains, and three-dimensional convolution on neural processing units. We swapped each for a quantization-friendly equivalent, then exported and calibrated the resulting graph to an eight-bit TensorRT engine for the Jetson Orin Nano. Table 4.2 summarizes the six changes and their equivalence status.

Table 4.2: Inference optimizations and equivalence status measured at  $384 \times 640$  on RTX 3050.

| Optimization                                 | Saving                                         | Equivalence    |
|----------------------------------------------|------------------------------------------------|----------------|
| Zero-copy padded views in cost-volume loops  | 5 to 15% latency                               | bitwise        |
| Batched correlation lookups, cached grids    | 3 to 8% latency                                | identical math |
| Static GRU input channels hoisted per scale  | 8 to 10% of MACs                               | exact          |
| Fused gate convolutions and output heads     | 1 to 3% latency                                | bitwise        |
| Dead inference-path code removed             | memory + launches                              | exact          |
| Narrow GEV band ( $\pm 16$ around tile $d$ ) | 25 to 30% of MACs                              | matched A/B    |
| Combined                                     | fp32 61.4 ms vs 106.7 ms base (1.74 $\times$ ) |                |

## 4.5 Training Objective and Protocol

The optimizations above act on a network that already has to be trained correctly in the first place. This section fixes what the network minimizes during training and the data protocol under which that objective is applied, the two pieces of the design methodology that sit upstream of the parameter values already listed in Table 4.1.

### 4.5.1 Training Objective

The network trains under the multi-scale objective of (4.1). A pixel counts as valid when its ground-truth disparity is finite and lies between 0 and 320 pixels. Each lower-scale prediction is resized to full resolution and multiplied by its scale factor before comparison. The objective weights an L1 term at every scale, heaviest at full resolution, adds a matching L1 term on the geometry-volume output, and augments the full-resolution prediction with a Sobel gradient-consistency term, a threshold hinge stack, a KITTI-style D1-relative hinge [42], an edge-aware smoothness term, and a gated slant-supervision term that trains the plane slopes only where they are trustworthy. The slant term pairs with plane rendering: it makes the slopes accurate enough for the rendering step to sharpen boundaries.

We chose this composition by ablation rather than guesswork. The loss study tested nine formulations: plain L1 under-serves the outlier metrics, hinge terms on a single threshold overfit that threshold, and the threshold stack combined with the D1 hinge was the only formulation to win both the end-point error and the outlier rates. The production objective descends from that winner, keeping the gradient term for its sub-pixel benefit and holding the smoothness term to a small weight so it regularizes without blurring.

$$\begin{aligned} \mathcal{L} = & 1.00 \mathcal{L}_1(d_{\text{full}}) + 0.50 \mathcal{L}_1(d_{1/2}) + 0.30 \mathcal{L}_1(d_{1/4}) + 0.20 \mathcal{L}_1(d_{1/8}) + 0.10 \mathcal{L}_1(d_{1/16}) \\ & + 0.15 \mathcal{L}_1(d_{\text{gev}}) + 0.50 \mathcal{L}_{\text{grad}} + 0.20 \mathcal{L}_{\text{thr}} + 0.20 \mathcal{L}_{\text{D1}} + 0.02 \mathcal{L}_{\text{smooth}} + 0.30 \mathcal{L}_{\text{slant}} \end{aligned} \quad (4.1)$$

where  $\mathcal{L}_1$  is the masked mean absolute disparity error,  $\mathcal{L}_{\text{grad}}$  a Sobel gradient-consistency term,  $\mathcal{L}_{\text{thr}}$  a hinge over the error thresholds  $\{0.5, 1, 2, 3\}$  pixels,  $\mathcal{L}_{\text{D1}}$  the D1-relative hinge,  $\mathcal{L}_{\text{smooth}}$  an edge-aware smoothness term, and  $\mathcal{L}_{\text{slant}}$  the gated slant supervision.

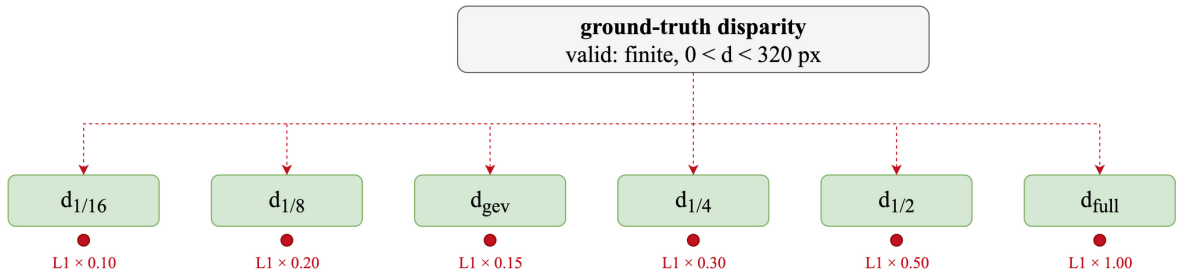


Figure 4.2: Six disparity scales supervised with L1, gradient consistency, and threshold terms.

### 4.5.2 Datasets and Training Protocol

This subsection fixes the protocol applied on top of the SceneFlow finalpass corpus, the thesis’s training and in-domain test set, and the project’s own rig captures used only for qualification. Optimization uses AdamW with a OneCycle schedule over 60,000 steps at a peak learning rate of  $8 \times 10^{-4}$ , a batch of 32, and bfloat16 mixed precision, on a single A100 GPU. Input is normalized to the unit interval at the model boundary. Table 4.1 lists this configuration in full; the paragraphs below detail the two parts a reader is most likely to want to reproduce, the data split and the crop-and-augment pipeline.

**Data split.** Table 4.3 gives the exact split. SceneFlow finalpass is the union of three rendered subsets, FlyingThings3D, Monkaa, and Driving, whose training halves sum to the 35,454 pairs the production model trains on. Only FlyingThings3D carries an official held-out test split, 4,370 pairs, which serves as the in-domain benchmark and is never seen during training. In-training validation is not a fourth pool of data: it is a fixed 400-pair subset drawn from that test split, evaluated every thousand steps at the native  $960 \times 540$  axis so that progress is tracked against the same distribution the final number is reported on. The project’s own stereo rig contributes 997 real indoor pairs, and its split is deliberately degenerate: none of them train or fine-tune the network at any point, so all 997 sit on the test side as a zero-shot real-camera qualification set, of which three scenes are carried through to the metric reconstruction of Chapter 6.

Table 4.3: Dataset split used for training, validation, and evaluation.

| Source              | Subset          | Train  | Val              | Test  |
|---------------------|-----------------|--------|------------------|-------|
| SceneFlow finalpass | FlyingThings3D  | 22,390 | 400 <sup>†</sup> | 4,370 |
| SceneFlow finalpass | Monkaa          | 8,664  |                  |       |
| SceneFlow finalpass | Driving         | 4,400  |                  |       |
| SceneFlow finalpass | total           | 35,454 | 400 <sup>†</sup> | 4,370 |
| Stereo rig          | indoor captures | 0      | 0                | 997   |

<sup>†</sup>The 400-pair validation set is a fixed subset drawn from the FlyingThings3D test split, evaluated every 1,000 steps; it is not additional data.

**Cropping.** Training samples a random  $384 \times 640$  window, co-located at the same coordinates in both the left and right view, from each native  $960 \times 540$  frame. Cropping at the native resolution rather than downscaling the whole frame first is a deliberate choice: a disparity is a distance in pixels, so shrinking the image also shrinks every disparity, and a network trained on downscaled frames learns to predict a compressed disparity range that no longer matches full-resolution input. The native crop keeps disparity magnitudes at their true scale while still fitting the memory budget, and the ablation of Chapter 3 found this native-crop

protocol substantially ahead of the legacy downscale-then-crop protocol; Figure 4.3 contrasts the two. Cropping the same window from both views is what preserves the epipolar correspondence: shifting the crops relative to each other would silently offset every disparity by the shift, so the window is always identical in the two images.

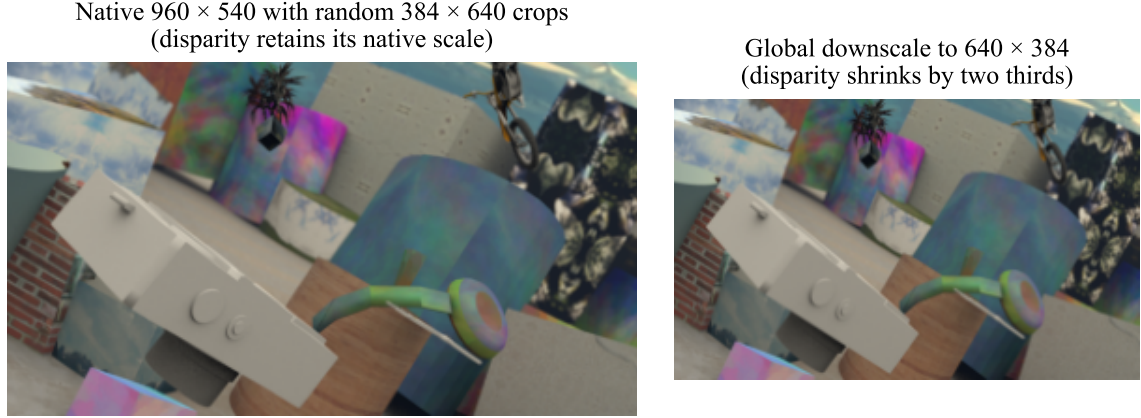


Figure 4.3: Native-crop sampling (left) vs downscale-then-crop (right) training protocols.

**Augmentation.** Three augmentations, drawn from the OpenStereo recipe, are applied to the training crop only; Figure 4.4 shows each acting on a real pair, and each answers a specific failure mode of a synthetic-only model on real cameras. *Asymmetric color jitter* perturbs brightness, contrast, and saturation each within  $[0.6, 1.4]$ , and one time in five it draws different parameters for the left and right image rather than the same ones. A real low-cost stereo rig has two separate sensors that never expose or white-balance identically, so the two views differ photometrically; jittering them asymmetrically forces the matcher to rely on structure rather than on an assumption that corresponding pixels share an exact color, which is what lets the SceneFlow-trained model survive on the rig. *Right-image random erasing* blanks one or two rectangles of 50 to 100 pixels in the right view only, filling them with the image mean. Because the erased content has no match in the left view, this manufactures occlusion on demand and trains the network to infer disparity from context where a direct match is unavailable, exactly the situation at object boundaries and image borders. *Random scaling with anisotropic stretch* resizes the crop by a factor of  $2^{U(-0.2, 0.4)}$ , with an extra horizontal stretch of  $2^{U(-0.2, 0.2)}$ , and rescales the disparity by the horizontal factor before recropping to  $384 \times 640$ . Scaling widens the range of object sizes and, crucially, of disparity magnitudes the network sees, so it does not overfit to the specific scale statistics of the synthetic set; the horizontal stretch changes disparity directly, which is why the disparity target is multiplied by the same horizontal factor. Evaluation applies none of these, so every reported number measures the network on unperturbed data.

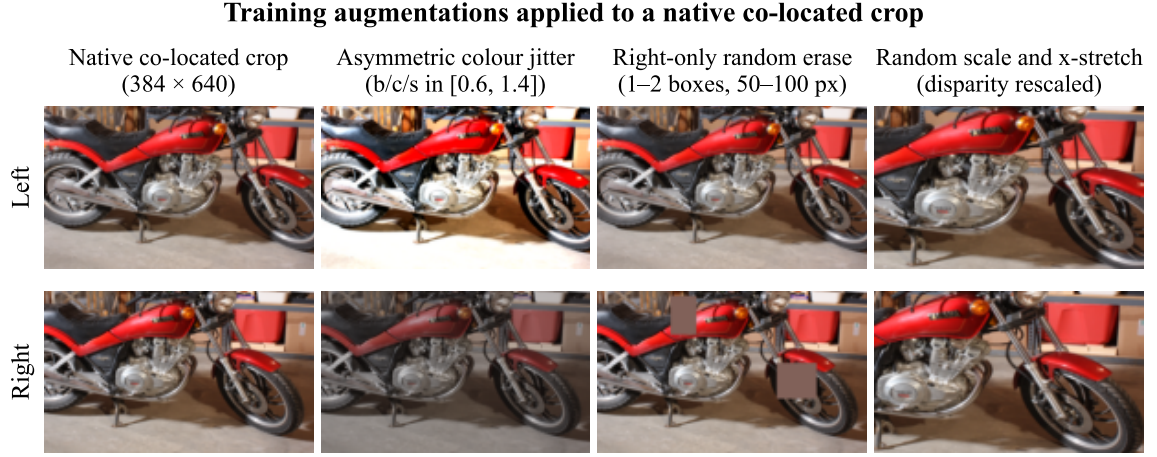


Figure 4.4: Training augmentations applied to a native  $384 \times 640$  co-located crop of a real rectified pair (Middlebury 2014 Motorcycle [43]): asymmetric color jitter, right-only random erasing (boxes outlined), and random scale with horizontal stretch. The eraser and the two views’ color jitter act independently on the right image; scaling rescales the disparity target by the horizontal factor.

## 4.6 Hardware and Embedded Deployment

The apparatus divides into compute and sensing. On the compute side, training runs on a single cloud A100 GPU with 80 gigabytes of memory. Ablations and evaluations run on a cloud T4, chosen for its low cost, which keeps a controlled study practical after every design change. The local latency benchmark runs on an RTX 3050 laptop GPU with roughly 4 gigabytes of memory, representing the low end of the deployment memory budget. The intended deployment target is the Jetson Orin Nano class of edge accelerator, on which we measured the optimized eight-bit model directly.

On the sensing side, real-camera pairs come from a calibrated low-cost binocular rig that streams a 2560 by 720 side-by-side image, split into a rectified 1280 by 720 pair per frame. This rig supplies the real-data qualification of the trained model: the zero-shot qualitative pairs, the real-camera agreement protocol, and the three-dimensional reconstructions it supports. Housing both the training-side compute and the sensing-side rig in the same section keeps the apparatus that produced every downstream number in one place, ahead of the experimental protocols built on top of it.

## 4.7 Principle of Operation

In deployment the system runs as a fixed feed-forward pipeline. A rectified left and right pair enters at the inference resolution of 384 by 640 pixels, normalized to the unit interval, and passes through the blocks fixed in Chapter 3. The shared encoder processes both views in a single batched pass and emits matching features at one quarter, one eighth, and one sixteenth

resolution, while a separate context encoder reads the left image alone and produces the context maps that feed the recurrent state.

Matching begins at the coarsest scale. The tile initialization module builds the group-wise correlation volume of (3.1) at one sixteenth resolution, regularizes it with a small three-dimensional network, and reads off the initial disparity and confidence through the soft-argmax of (3.2); the plane slopes start at zero, and the hidden state starts from the coarsest context map. The recurrent refinement then runs its fixed schedule: two iterations at one sixteenth, three at one eighth, and three at one quarter resolution. At each iteration the right features are warped by the current disparity, a local correlation is evaluated over five offsets, and the convolutional gated recurrent unit of (3.4) updates the hidden state; gated residuals adjust the disparity, the plane slopes, and the confidence. Between scales, the plane-aware propagation of (3.5) carries the tile state to the finer grid along its local slanted-surface hypothesis rather than by plain interpolation.

At one quarter resolution, the narrow-band geometry encoding volume produces its independent proposal per (3.6), and the fail-soft gate of (3.7) blends that proposal into the recurrent estimate only where the learned weight favors it. This corrects regions the tile state resolved poorly, without disturbing the ones it resolved well. The refined quarter-resolution state is rendered from its tile planes with an edge gate and upsampled to full resolution by two learned convex-combination stages, each output subpixel a softmax-weighted mixture of its three-by-three coarse neighborhood. The result is a dense full-resolution disparity map.

Disparity becomes depth through the calibrated relations of (3.8): depth follows  $Z = fB/d$ , and each valid pixel optionally back-projects to a metric three-dimensional point in the camera frame, colored by its left-image pixel. On the project's rig, the calibrated focal length and baseline scale to the inference resolution by the width ratio, so the point cloud is metric without further tuning. This single forward pass, from image pair to disparity map and optional colored point cloud, is the operation this thesis measures throughout.

## 4.8 Summary

This chapter traced the path from the network architecture to a trained, deployable system. The codebase separates training, ablation, and evaluation into scripted drivers that each log their own configuration and metrics, running on Modal-orchestrated cloud A100 and T4 GPUs. The production model fixes a truncated YOLO26s backbone, a 32-channel recurrent state, an eight-group correlation volume, and a two, three, three iteration schedule, and this configuration is recorded in full. A six-change, equivalence-proven optimization pass cut inference latency by a factor of 1.74 and cleared the operators that block eight-bit deployment, producing a calibrated TensorRT engine for the Jetson Orin Nano. Training itself runs under an eleven-term multi-scale objective built around a threshold-stack and D1-relative hinge, applied to random native-resolution crops of the SceneFlow corpus with a three-part aug-

mentation recipe and a sixty-thousand-step OneCycle schedule. The compute and sensing apparatus spans a cloud A100 for training, a cloud T4 for ablations, an RTX 3050 for local latency benchmarking, the Jetson Orin Nano deployment target, and a calibrated low-cost stereo rig for real-camera qualification. The principle of operation section walked through the resulting forward pass, from a rectified image pair to a full-resolution disparity map and an optional metric point cloud.

## **Chapter 5**

### **Experimental Setup and Data Collection**

#### **5.1 Introduction**

A measurement is only as trustworthy as the conditions that produced it, and this chapter sets those conditions down before any result is reported. It answers a plain set of questions: what machines the work ran on, what data the network learned from and what data it was only ever tested on, where the real camera images came from and how the rig behind them was calibrated, how every one of those images is prepared before it reaches the network, and exactly what procedure turns a single forward pass into a number that can be quoted.

The chapter is arranged from apparatus to data to procedure. It first describes the test bench, the computing platforms and the stereo rig, and the calibration that makes the rig’s geometry usable for depth. It then covers the three sources of data the work draws on and the single preparation path that carries all of them to the network’s input: bringing each image to a common resolution, normalising it, and, for training only, augmenting it. It closes with the four measurement procedures that every reported accuracy, latency, and robustness figure follows, each stated once so those figures can later be quoted without re-explaining how they were obtained.

#### **5.2 Experimental Setup and Test Bench**

The test bench spans four compute platforms and one piece of physical sensing hardware, chosen so that each stage of the work runs on the platform best suited to it. The production model is implemented in PyTorch 2.11 with CUDA 12.8 on Python 3.10, and training runs on a single cloud GPU with 80 gigabytes of memory. Full runs are expensive enough that they are reserved for the final configuration; the many controlled ablations behind that configuration, and the cross-domain evaluation that follows every training run, instead run on a smaller cloud GPU, chosen for its low cost so that a controlled comparison after every design change stays practical. Local latency benchmarking runs on a laptop-class GPU with roughly 4 gigabytes of memory, which represents the low end of the memory budget the network must fit inside rather than a generous development machine. The intended deployment target is a Jetson Orin Nano class of edge accelerator, on which the optimized model was measured directly, and Figure 5.1 shows the development board used for that measurement.

On the sensing side, the test bench includes a calibrated low-cost binocular rig, a Wave-share AR0144 stereo USB camera [46] described fully in Section 5.3, which supplies every real-camera pair used in this thesis. Together, the four compute platforms and the rig give a training platform sized for the full dataset, an inexpensive platform for iterating on the

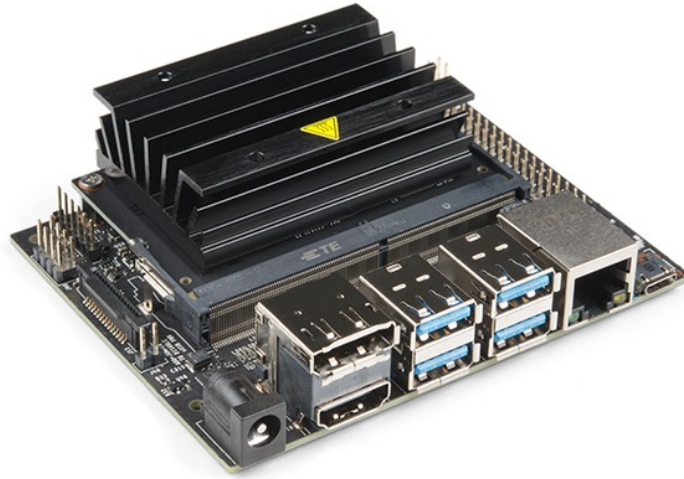


Figure 5.1: Jetson Orin Nano edge accelerator, deployment target for inference.

design, a worst-case memory reference for the deployment budget, the intended deployment hardware itself, and a physical source of real, imperfectly rectified data that no synthetic benchmark can substitute for.

### 5.3 Calibration and Rectification

Real-camera pairs come from a calibrated binocular rig, a Waveshare AR0144 stereo USB camera whose dual global-shutter sensors are driven synchronously so both views land in the same frame [46]. Figure 5.2 shows the module. The rig streams a single 2560 by 720 side-by-side image, which is split into a rectified 1280 by 720 pair per frame; Figure 5.3 sketches the stream and the calibrated geometry, and Table 5.1 lists the manufacturer specifications.

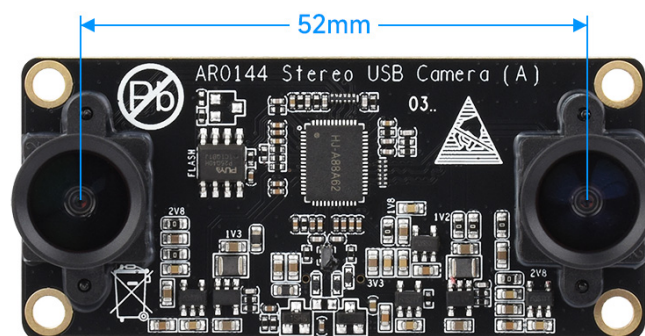
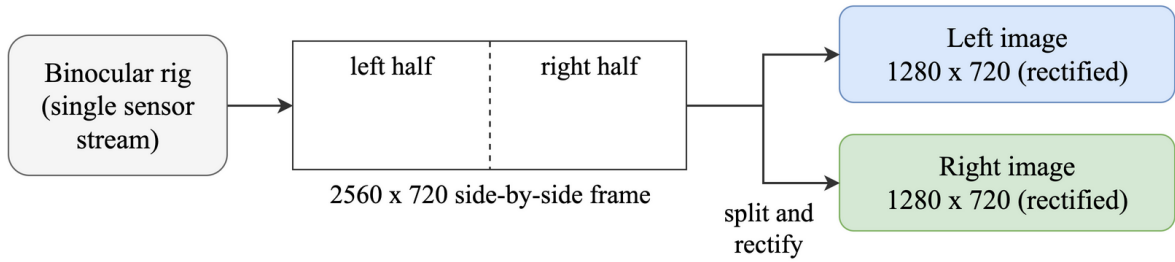


Figure 5.2: Waveshare AR0144 stereo camera module with its 52 mm baseline (product image: Waveshare [46]).

Table 5.1: Manufacturer specifications of the AR0144 stereo camera [46].

| Item          | Value                                                 |
|---------------|-------------------------------------------------------|
| Sensors       | 2 × onsemi AR0144, 1/4 inch, global shutter           |
| Stereo output | 2560 × 720 side by side (1280 × 720 per view), 30 FPS |
| Baseline      | 52 mm                                                 |
| Lens          | 2.88 mm EFL, F/2.2, fixed focus, distortion < 0.2%    |
| Field of view | 74° diagonal, 65° horizontal, 43° vertical            |
| Interface     | USB 2.0 Type-C, 5 V                                   |
| Board size    | 66.00 × 30.00 × 17.72 mm                              |

### Capture and split



### Rig geometry

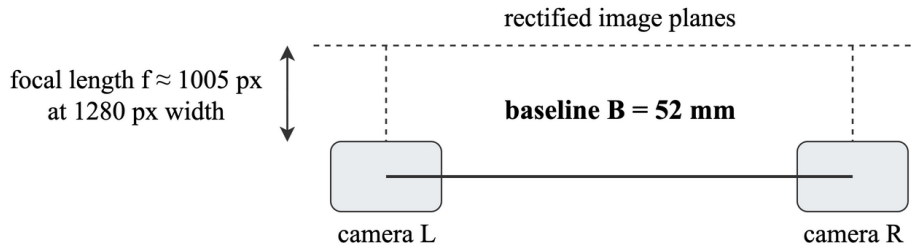


Figure 5.3: Stereo rig capture and rectification geometry for metric depth.

Calibration recovers a focal length of approximately 1005 pixels at the native 1280-pixel width and a baseline of 52 millimeters between the two camera centers; both values scale to the network’s inference resolution by the width ratio, so the disparity the network predicts converts to metric depth without any additional tuning. The calibrated values agree with the manufacturer data of Table 5.1: the published baseline is exactly 52 millimeters, and the 65 degree horizontal field of view over the 1280-pixel width implies a focal length close to the calibrated 1005 pixels. The rig’s factory rectification is imperfect and leaves a residual vertical misalignment between the two views, on the order of a fraction of a pixel to about a pixel, rather than the perfect epipolar alignment synthetic training data provides. This gap between laboratory-perfect and factory rectification is exactly the condition the second research objective targets, and Section 5.7 defines the two protocols, one controlled and synthetic, one on the physical rig, that measure how well the network tolerates it.

## 5.4 Datasets

This thesis draws on three data sources, summarized in Table 5.2: a synthetic corpus with dense ground truth for training and in-domain testing, a real benchmark the network never trains on for zero-shot cross-domain testing, and the project’s own physical rig for real-camera qualification. This section describes what each source contains.

Table 5.2: Data sources with pair counts, resolution, and ground truth.

| Source              | Role                      | Pairs            | Resolution       | Ground truth             |
|---------------------|---------------------------|------------------|------------------|--------------------------|
| SceneFlow finalpass | train / in-domain test    | 35,454 / 4,370   | 960×540          | dense, rendered          |
| Middlebury 2014     | zero-shot cross-domain    | 23 (perfect set) | up to ~3000×2000 | structured light         |
| Stereo rig capture  | real-camera qualification | 997              | 1280×720         | none (teacher reference) |

### 5.4.1 Synthetic Training and Test Corpus: SceneFlow

SceneFlow finalpass [9] is the synthetic corpus the network trains on. Its canonical split combines FlyingThings3D, Monkaa, and Driving into 35,454 training pairs, each rendered at  $960 \times 540$  with dense, pixel-exact disparity ground truth; only the FlyingThings3D subset provides an official test split, 4,370 pairs, held out and never seen during training. Chapter 4 describes the crop, augmentation, and optimization protocol applied on top of this corpus; this chapter records the corpus itself, since its size and density set the ceiling on how well the network can learn dense correspondence before it is ever asked to generalize. Rendered ground truth of this kind is something no physical sensor can supply at this scale, which is precisely why the field trains on it despite the domain gap it introduces relative to real imagery.

### 5.4.2 Zero-Shot Cross-Domain Benchmark: Middlebury 2014

Middlebury 2014 [43] supplies the cross-domain test. Its 23 perfect-rectification scenes, real indoor imagery captured with structured-light ground truth at up to roughly  $3000 \times 2000$  pixels, carry public ground truth and are the scenes this thesis evaluates on; the benchmark’s remaining 10 scenes withhold their ground truth on a submission server and are not used here. The network never trains on any Middlebury imagery, so every result against this benchmark is a genuine zero-shot measurement. Its disparities extend to several hundred pixels and its scenes include thin structures and repeated texture that the synthetic training distribution under-represents, which is why it functions as a stress test rather than a second in-domain benchmark.

### 5.4.3 Physical Stereo Rig Captures

The third source is the project’s own rig, described in Section 5.3. Unlike the two benchmarks above, it carries no dense ground truth of its own; its role is to confirm that behaviour measured on synthetic and public data survives contact with a real, imperfectly rectified camera. Section 5.5 details how these pairs were captured and how a reference disparity was obtained for them despite the absence of ground truth.

## 5.5 Data Collection

Real pairs were captured with the rig connected over USB to a laptop running a capture script built on OpenCV’s V4L2 backend, which requests the sensor’s native 2560 by 720 side-by-side stream at up to 60 frames per second and reads it as motion JPEG. Each captured frame is split in software at its horizontal midpoint into the rectified 1280 by 720 left and right views described in Section 5.3 and written to disk as a matched pair before the next frame is taken, so every saved pair shares one exposure instant rather than being assembled from two separate captures. Collection proceeded in short indoor sessions across different rooms, lighting conditions, and object arrangements; no frame selection criterion beyond this basic scene variety was applied, since the purpose of these pairs is to qualify the network on unremarkable real data rather than to stress a particular scenario. This yielded 997 rectified pairs, none of which contributed to training or fine-tuning at any stage; the network that produces disparity on these pairs is exactly the SceneFlow-trained checkpoint of Chapter 4, run through the same resize-and-normalize step every other input passes through, detailed in Section 5.6.

Because the rig supplies no dense ground truth, the real-camera agreement protocol of Section 5.7 instead compares the network’s prediction on these 997 pairs against a pseudo-ground-truth reference produced by a frozen FoundationStereo teacher run on the same pairs. The teacher is a large, accurate, foundation-model-based network that is never trained or fine-tuned on this project’s data; it is applied once, off-line, to generate a disparity reference for each of the 997 pairs, and that reference is then treated as a strong stand-in for ground truth in the agreement comparison. Because both the compact network and the teacher observe the same imperfectly rectified input, a close match between them measures agreement under real conditions rather than absolute error against a laboratory-perfect reference, and Section 5.7 states this distinction explicitly wherever the resulting numbers are used.

## 5.6 Image Preprocessing Pipeline

Three different data sources feed the network: rendered SceneFlow pairs, photographed Middlebury pairs, and the rig’s own captures. Each arrives in a different raw format, but every pair, regardless of source, is brought through the same two stages before it reaches

the network: a resolution step that puts the pair on the pixel grid the current use case needs, and then a fixed normalization step at the model boundary. Table 5.3 summarizes how each source fills in that pattern; the paragraphs below explain the reasoning behind each row.

Table 5.3: Preprocessing applied to each data source before the network boundary.

| Source              | Raw format                                                                 | Resolution step                                                                               | Augmentation                                    |
|---------------------|----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------|
| SceneFlow, training | $960 \times 540$ rendered pair, dense PFM disparity                        | random co-located $384 \times 640$ crop, same window in both views                            | color jitter, right-image erasing, random scale |
| SceneFlow, testing  | $960 \times 540$ rendered pair                                             | native full frame (primary axis), or area-resized to $640 \times 384$ (logged for comparison) | none                                            |
| Middlebury 2014     | real photograph up to $\sim 3000 \times 2000$ , structured-light disparity | area-resized to $640 \times 384$ ; disparity resized and rescaled by the width ratio          | none                                            |
| Stereo rig          | $2560 \times 720$ live stream                                              | split at midpoint to $1280 \times 720$ , then area-resized to $640 \times 384$                | none                                            |

SceneFlow pairs render already rectified, with a dense per-pixel disparity map at the same  $960 \times 540$  resolution as the image, so no rectification or interpolation of the ground truth is needed at this stage. During training, a random  $384 \times 640$  window is cropped from matching coordinates in both the left and right image, which keeps disparity magnitudes at their native scale rather than shrinking them the way a whole-frame resize would; Section 4.5 in Chapter 4 gives the reasoning behind this native-crop choice and the three augmentations, asymmetric color jitter, right-image-only random erasing, and random scaling, that are applied to the cropped window only during training. Evaluation never crops or augments: the primary in-domain metric runs the network on the full native  $960 \times 540$  frame, which the architecture’s fully convolutional design accepts directly, and a  $640 \times 384$  resized-axis measurement is logged alongside it purely for comparison with work that reports at the resized scale.

Middlebury pairs arrive at whatever resolution the scene was photographed at, up to roughly  $3000 \times 2000$  pixels, far larger than the network’s working resolution. Both views are downsampled with area interpolation, which averages pixels rather than sampling a single one and so avoids aliasing high-resolution detail, to the fixed  $640 \times 384$  inference size; the structured-light ground truth is resized with linear interpolation to match and its values rescaled by the same width ratio, since a disparity is a distance in pixels and shrinks proportionally to the image. This is the only rescaling the zero-shot protocol applies, and it is applied identically to the trained model and to every reference checkpoint it is compared

against.

The rig contributes the longest chain, because it is the only source that is not already a single rectified frame. Its raw  $2560 \times 720$  stream frame is split at the midpoint into left and right  $1280 \times 720$  views, as described above; each view is then downsampled with the same area interpolation used for Middlebury, to the network’s  $640 \times 384$  working resolution. This resize step is the reason the rig’s calibrated focal length is scaled by the width ratio wherever it is used for metric back-projection (Section 5.3 and Section 3.9.7): the disparity the network predicts is a distance in pixels on the resized grid, and the geometry that converts it to depth has to be expressed on that same grid.

At the model boundary, every source ends up on a common footing regardless of how it got there. The RGB tensor, in the  $[0, 255]$  range after loading, is divided by 255 to the unit interval immediately before the first convolution; no further per-channel standardization is applied, so the same simple boundary serves training, evaluation, and on-device deployment alike. Augmentation is the one stage that is not shared: only the SceneFlow training crop is randomly jittered, erased, and rescaled, precisely because the point of testing on SceneFlow’s held out pairs, on Middlebury, and on the rig is to measure how the network behaves on data it was not artificially perturbed to expect.

## 5.7 Experimental Procedure and Evaluation Metrics

Every reported number follows one of four measurement protocols, defined here so that each has a single stated procedure rather than an implicit one repeated across tables. Accuracy throughout follows the standard stereo metrics of (3.9): end-point error [9], the bad- $t$  rates [2], and the D1-all outlier rate [42], each metric averaged per image and then over images rather than pooled across every pixel in the set.

The first protocol measures latency and memory. Latency is measured at batch one and  $384 \times 640$  resolution after warm-up passes; memory is recorded as the peak inference working set, and every measurement states its device. On the local laptop GPU the model runs in eager PyTorch at full and half precision, without compiler assistance, so those numbers reflect an unoptimized serving path. On the deployment target, the Jetson Orin Nano, the optimized model is exported to an eight-bit engine and measured directly on the device, and that on-device figure is the one taken as evidence for the real-time claim.

The second protocol tests tolerance to imperfect rectification under controlled conditions. The trained model is evaluated at the native axis on a fixed 400-pair FlyingThings3D subset. The right image is shifted vertically by a controlled offset while the disparity ground truth stays unchanged, so the offset emulates a residual vertical misalignment between the views without touching anything else about the pair. The sweep runs from zero through sub-pixel offsets to several pixels, and the standard metrics are reported at every point on the sweep.

The third protocol corroborates that synthetic sweep on physical data, using the collec-

tion described in Section 5.5. The trained model is run on the 997 rectified pairs from the project’s rig, whose factory rectification leaves a residual vertical misalignment of the kind the second protocol emulates, and its disparity is compared against the FoundationStereo teacher’s pseudo-ground-truth on the same pairs. Because both the model and the teacher see the same misaligned input, this protocol measures agreement with a strong reference rather than absolute error, and it corroborates rather than replaces the controlled sweep of the second protocol.

The fourth is the dual-axis evaluation convention. Every in-domain evaluation is reported on the native  $960 \times 540$  axis, which preserves true disparity magnitudes, with a resized-axis measurement at  $640 \times 384$  logged alongside for comparability with the legacy protocol discussed in Chapter 4. Any comparison between input protocols evaluates every candidate on both axes, so a gain on one axis cannot hide a loss on the other. Zero-shot Middlebury 2014 evaluation follows a single fixed protocol applied identically to the trained model and to the public reference checkpoints it is compared against: inference at  $384 \times 640$ , ground truth scaled to the inference width, disparities above 192 pixels masked, and per-scene metrics macro-averaged over scenes.

Each protocol runs as a separate scripted driver on a cloud GPU, and every driver writes its configuration and metrics to a metadata file alongside its outputs so that a reported number can be traced back to the run that produced it; the specific driver names are recorded in the appendix covering technical specifications. A cross-domain evaluation run of this kind costs only minutes and a small fraction of a cloud GPU-hour, which is what keeps the after-every-training-run evaluation gate described in Chapter 3 practical rather than an occasional afterthought.

## 5.8 Summary

This chapter fixed the test bench, the data sources, and the measurement protocols behind every number in the study. The test bench pairs a large cloud GPU for production training with a smaller, cheaper cloud GPU for ablations, a memory-constrained laptop GPU for a conservative latency reference, and the Jetson Orin Nano as the deployment target, alongside a calibrated low-cost stereo rig whose focal length, baseline, and imperfect factory rectification were all stated so that reconstruction and robustness numbers stay traceable to a physical measurement. Three data sources were described: the SceneFlow synthetic corpus that trains and in-domain-tests the network, the Middlebury 2014 benchmark that tests it zero-shot on unseen real imagery, and 997 real pairs collected from the project’s own rig, for which a frozen FoundationStereo teacher supplied a pseudo-ground-truth reference in place of the dense ground truth the rig cannot provide. Four measurement protocols, latency and memory, controlled rectification perturbation, real-rig agreement, and dual-axis evaluation, were each fixed to a single stated procedure, giving every reported number a documented

and reproducible method behind it rather than an implicit one.

# Chapter 6

## Results and Discussion

### 6.1 Introduction

This chapter reports the accuracy, efficiency, and cross-domain behaviour of the trained model. We cover four threads: in-domain accuracy on FlyingThings3D against published methods, zero-shot accuracy on Middlebury 2014 against matched-protocol checkpoints, on-device latency against the real-time target, and the training dynamics and ablation studies behind the architectural design of Chapter 3. Unless stated otherwise, the results come from a single production run trained on the full SceneFlow FlyingThings3D corpus, under the protocol and configuration of Table 4.1 in Chapter 4. We follow the accuracy metrics of (3.9); latency is measured at batch one, 384 by 640 resolution; and cross-domain behaviour is assessed on Middlebury 2014, which the model never saw during training. Chapter 5 (Section 5.4) sets out both datasets, the physical stereo rig, and the evaluation protocol in full; this chapter does not repeat those details.

### 6.2 Experimental Results

#### 6.2.1 Training Dynamics

Figure 6.1 shows the training trajectory. Loss falls sharply during the OneCycle warm-up, then descends steadily as the learning rate anneals toward its floor. Validation end-point error tracks the loss closely, bottoming out at 0.79 pixels near step 53,000 and holding flat between 0.80 and 0.81 pixels for several thousand steps as the rate nears zero. We took the best checkpoint at step 53,000, not the final step 60,000, since further training gave no improvement. Figure 6.2 shows the same pattern: broad surfaces settle early while object boundaries and thin structures keep sharpening. The run survived several cloud preemptions: scheduler and optimizer state are checkpointed and restored exactly, so a preemption costs at most the steps since the last validation. The one-cycle schedule’s high peak rate acts as a regularizer mid-training, visible as the sustained loss variance in Figure 6.1; its long cosine descent then turns that exploration into steady gains, with more than half the final improvement coming after the rate drops below a quarter of its peak. The per-batch loss wobble in the high-rate phase is the schedule working as designed, not instability, and the flat validation tail confirms that 60,000 steps was about right for this model and dataset.

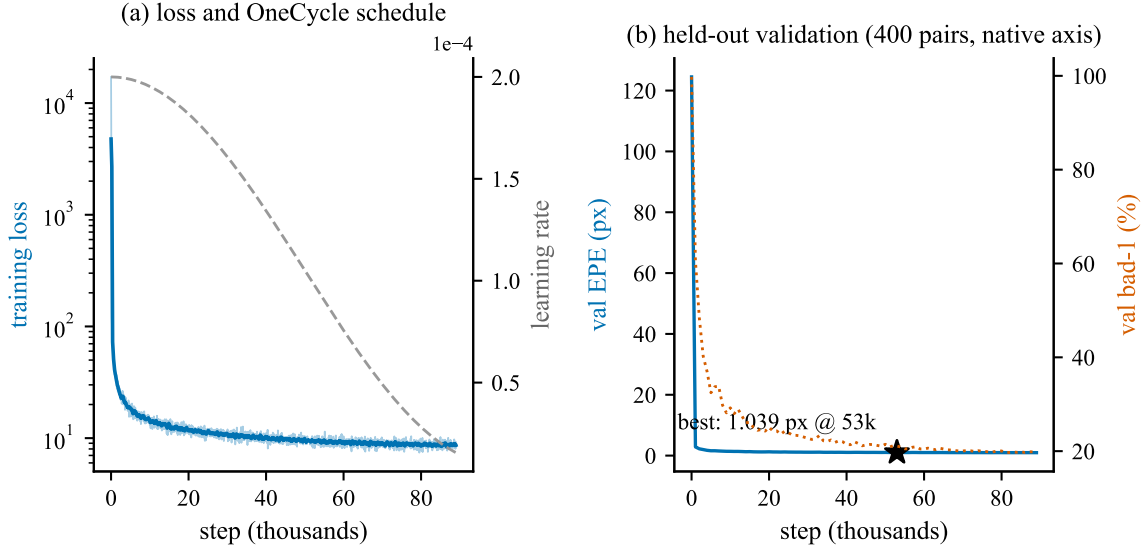


Figure 6.1: Training loss and validation accuracy curves over the training run.

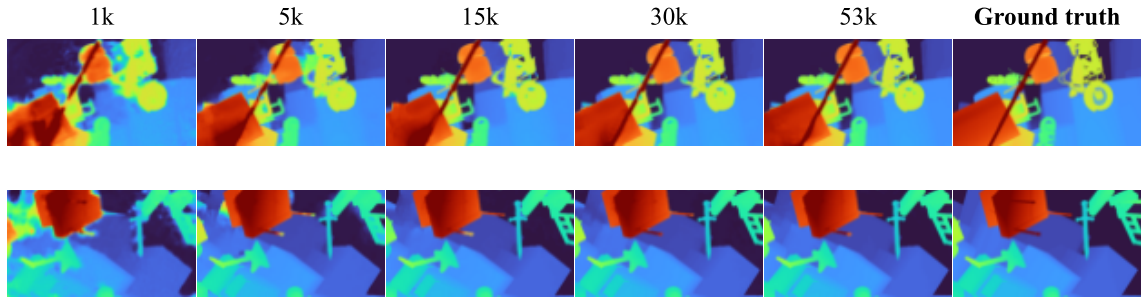


Figure 6.2: Predicted disparity across training checkpoints for two FlyingThings3D scenes.

### 6.2.2 In-Domain Accuracy: SceneFlow Test

We evaluated the trained model on the full FlyingThings3D test set, 4,370 pairs at native 960 by 540 resolution, using the best checkpoint. Table 6.1 reports 0.78 pixel end-point error, 3.40 percent D1-all, and a median error of 0.13 pixels; four frames with no valid ground-truth pixel after the disparity cutoff are excluded from the macro-average. The metric spread tells its own story: the median of 0.13 pixels is six times smaller than the mean of 0.78, and the RMSE of 3.64 is nearly five times larger, so most pixels are highly accurate while a small population of large outliers sets the average. The bad-rate ladder agrees: errors above half a pixel affect 15.3 percent of pixels, but above three pixels only 4.0 percent, so most failures are moderate and catastrophic ones rare. Figure 6.3 shows where those outliers live: occlusion boundaries and thin structures, consistent with recurrent reasoning that runs at quarter resolution and an upsampler that can recombine disparity but cannot invent it.

Table 6.1: Full FlyingThings3D test-set results across all eight evaluation metrics.

| EPE (px) | RMSE | Median | bad-0.5 (%) | bad-1 (%) | bad-2 (%) | bad-3 (%) | D1-all (%) |
|----------|------|--------|-------------|-----------|-----------|-----------|------------|
| 0.781    | 3.64 | 0.130  | 15.32       | 8.92      | 5.34      | 4.00      | 3.40       |

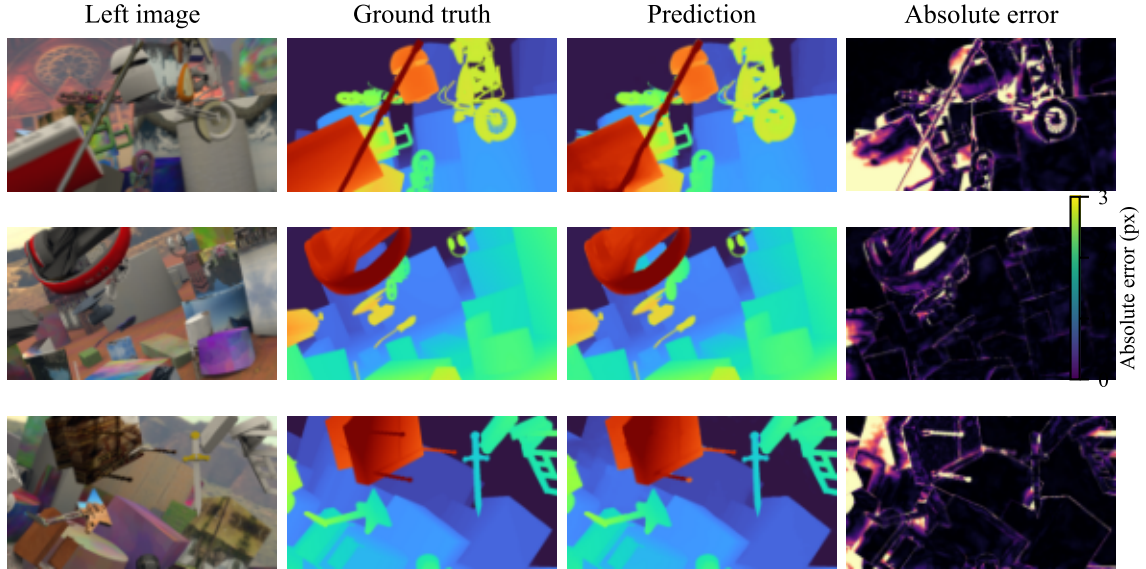


Figure 6.3: Left image, ground truth, prediction, and error map for held-out FlyingThings3D scenes.

## 6.3 Comparison

### 6.3.1 Comparison with Published Methods on SceneFlow

Table 6.2 places our numbers alongside published methods, though the comparison is not strictly protocol-identical: each published EPE follows its own paper’s SceneFlow protocol, while our value comes from the full test set at native resolution. The model falls between the efficient and iterative families: more accurate than the classical 3D-cost-volume baseline PSMNet, close to the efficient LightStereo-S, and a fraction of the parameters of the iterative and foundation-scale systems.

Table 6.2: Comparison with published SceneFlow methods (protocol not directly comparable to ours).

| Method               | Params (M) | SF EPE (px) | Latency (ms) | Hardware        |
|----------------------|------------|-------------|--------------|-----------------|
| PSMNet [11]          | 5.2        | 1.09        | 410          | Titan Xp        |
| HITNet (L) [29]      | 0.97       | 0.43        | 54           | Titan V         |
| CoEx [33]            | 2.72       | 0.69        | 27           | RTX 2080Ti      |
| RAFT-Stereo [17]     | 11.23      | 0.61        | 380          | RTX 6000        |
| IGEV-Stereo [18]     | 12.60      | 0.47        | 180          | RTX 3090        |
| LightStereo-S [30]   | 3.44       | 0.73        | 17           | RTX 3090        |
| FoundationStereo [1] | ~340       | 0.34        | ~470         | RTX 4090        |
| StereoLite (ours)    | 2.96       | 0.78        | 49.8 (fp16)  | RTX 3050 laptop |

### 6.3.2 Zero-Shot Cross-Domain: Middlebury 2014

Cross-domain accuracy is the real test for an edge model: one accurate only on its training domain has little field use. We evaluated the trained model zero-shot on the 23 perfect-set scenes of Middlebury 2014, unseen during training, under a protocol identical for every method in Table 6.3, reaching 1.71 pixel end-point error and 10.9 percent D1-all, the smallest model in the comparison at 2.96 million parameters. A gap remains to LiteAnyStereo (6.9 percent) and sixteen-iteration IGEV-Stereo (5.0 percent), both larger models; LiteAnyStereo also adds a three-stage knowledge-distillation pipeline that our supervised-only recipe skips. Figure 6.4 shows sixteen of the 23 scenes within the reference band, with the aggregate dragged up by a few hard scenes; Table F.2 lists the per-scene split.

Table 6.3: Zero-shot Middlebury 2014 comparison across three models (our reproduction of public checkpoints; T4 fp32 eager latency).

| Model                     | Params (M) | T4 (ms) | EPE         | bad-0.5 | bad-1 | bad-2 | bad-3 | D1-all (%)  |
|---------------------------|------------|---------|-------------|---------|-------|-------|-------|-------------|
| <b>StereoLite (ours)</b>  | 2.96       | 86      | <b>1.71</b> | 39.6    | 23.9  | 14.5  | 11.2  | <b>10.9</b> |
| LiteAnyStereo [32]        | 7.60       | 64      | 1.17        | 35.1    | 19.7  | 10.5  | 7.4   | 6.9         |
| IGEV-Stereo (16 it.) [18] | 12.60      | 305     | 0.86        | 22.2    | 12.4  | 7.2   | 5.2   | 5.0         |

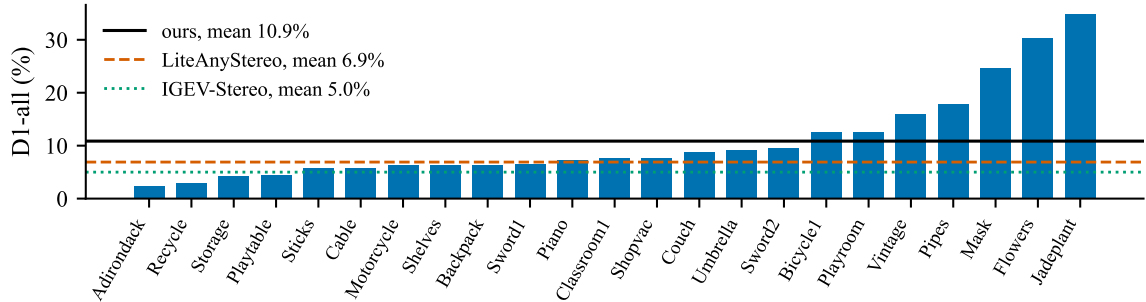


Figure 6.4: Per-scene zero-shot D1-all on Middlebury 2014, sorted by error.

The easiest scenes are ordinary furnished rooms: Adirondack at 2.4 percent D1-all, Recycle at 2.9, and Storage at 4.2, comparable to the reference models. The hardest four alone raise the mean by several points: Jadeplant at 34.9 percent, Flowers at 30.3, Mask at 24.7, and Pipes at 17.9. These scenes share dense thin repeated structures such as leaves and rods, plus true disparities near or beyond the 192-pixel evaluation cap, exactly where a quarter-resolution, bounded-search prediction performs worst. Figure 6.5 shows both ends of that spectrum. The model’s cross-domain failure is not diffuse: it is localized to an identifiable scene type, which a future distillation direction could target directly.

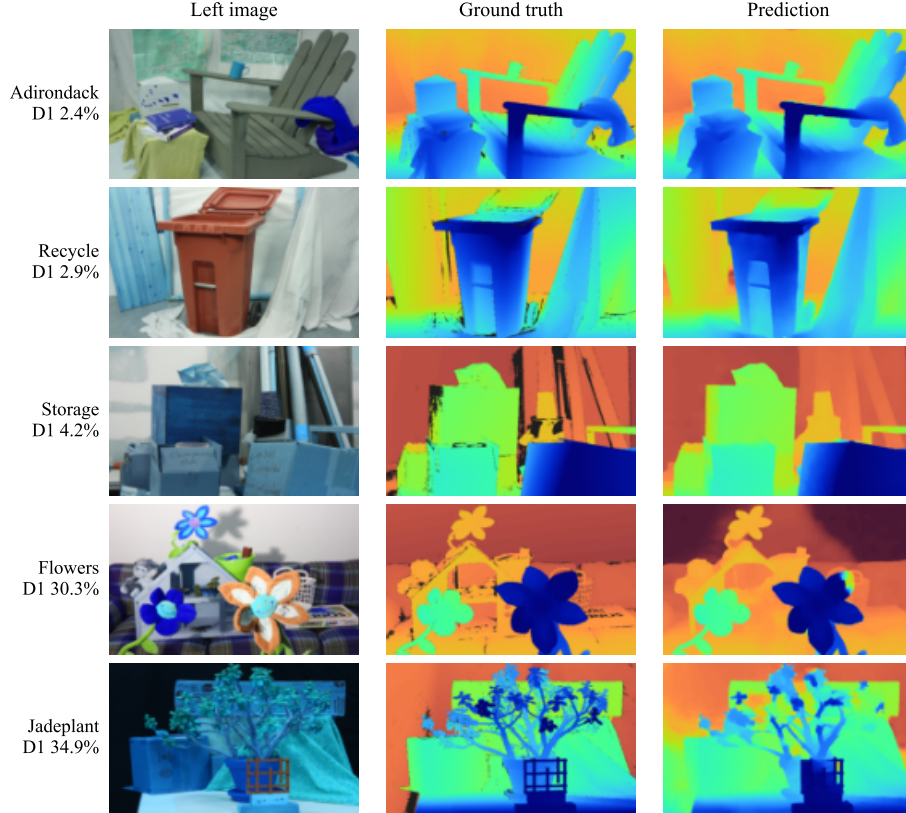


Figure 6.5: Left image, ground truth, and prediction for the easiest and hardest Middlebury 2014 scenes.

## 6.4 Error Analysis

### 6.4.1 Robustness to Rectification Imperfection

A second thesis objective is tolerance to imperfect rectification. We evaluated the trained model on a 400-pair test subset with the right image shifted vertically by a controlled offset and ground truth unchanged, emulating residual vertical misalignment between views. Table 6.4 and Figure 6.6 report the result. The zero-offset baseline of 1.03 pixels is measured on this 400-pair perturbation subset rather than the full 4,370-pair test set, so it sits above the 0.78-pixel headline of Table 6.1; the quantity of interest here is the rise under misalignment, not the absolute level. Accuracy degrades gracefully across the practical range. At a half-pixel offset, end-point error rises only from 1.03 to 1.22 pixels and D1-all from 4.3 to 4.6 percent; at a full pixel, error stays at 1.53 pixels with 6.2 percent D1-all. A calibrated rig typically holds residual vertical error below one pixel, so accuracy survives this range; the tolerance comes from the refinement stage’s local correlation lookup, which checks a small neighborhood around each match instead of assuming a horizontal epipolar line. Beyond this range degradation is steep: a two-pixel offset raises D1-all to 15.8 percent, a four-pixel offset to 41.2 percent, since the network lacks explicit vertical search and large misalignment

pushes matches outside the lookup window. This is measured tolerance to small misalignment, not immunity to arbitrary de-rectification.

Table 6.4: Accuracy on FlyingThings3D under simulated vertical rectification offset.

| Vertical offset (px) | EPE (px) | bad-1 (%) | bad-2 (%) | D1-all (%) |
|----------------------|----------|-----------|-----------|------------|
| 0.0                  | 1.03     | 10.28     | 6.63      | 4.29       |
| 0.5                  | 1.22     | 14.62     | 7.58      | 4.60       |
| 1.0                  | 1.53     | 26.27     | 11.84     | 6.19       |
| 2.0                  | 2.43     | 47.73     | 27.71     | 15.83      |
| 4.0                  | 5.07     | 71.59     | 55.68     | 41.17      |

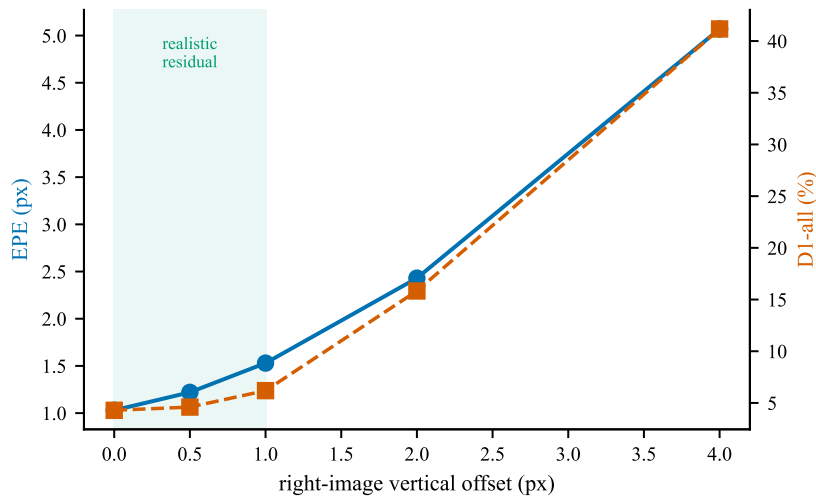


Figure 6.6: End-point error and D1-all against vertical rectification offset.

The synthetic sweep is controlled with known ground truth, but the condition it emulates occurs for real on a low-cost rig. To confirm this on physical data, we ran the model on 997 rectified pairs from the project’s stereo rig, a commodity camera whose factory rectification leaves residual vertical misalignment, and compared it against the FoundationStereo teacher’s pseudo-ground-truth on the same pairs. Agreement is close: mean end-point error is 1.45 pixels and D1-all is 11.1 percent, matching the in-domain synthetic error despite the real rig’s imperfect rectification. Since both see the same misaligned input, this figure measures agreement with a strong reference rather than absolute error, corroborating rather than replacing the controlled sweep. Together, the two results confirm that the model withstands the rectification imperfections of an inexpensive camera.

### 6.4.2 Qualitative Results

Visual inspection reinforces the quantitative results above. On the synthetic test set, Figure 6.3 (shown earlier in this chapter) shows accurate reconstruction of broad surfaces, with error confined to occlusion boundaries. On the unseen Middlebury scenes, Figure 6.5 shows

clean reconstruction of easy scenes, with failure localized to thin repeated structures rather than spread across the image. The plane-rendering step of Chapter 3 shows at the boundaries: predicted disparity follows object edges more closely than a bilinear upsample would. Figure 6.7 extends this inspection to real data: four indoor pairs from the calibrated stereo rig, none used in training or fine-tuning, yield plausible disparity maps that cleanly separate foreground from background, and the model transfers to a real camera without adaptation.



Figure 6.7: Left image and predicted disparity for real indoor stereo pairs.

### 6.4.3 3D Reconstruction of Real Scenes

This subsection presents the promised reconstruction. We back-projected disparity from three real indoor scenes to metric point clouds via the calibrated relations of (3.8), cleaned them with a statistical outlier filter and voxel downsampling per Section 3.9.7, and rendered them from three viewpoints (Figure 6.8). The reconstructions are metrically plausible. In the first scene, two fire extinguishers stand proud of the wall and door in the oblique and top-down views. In the second, the corridor recovers a smooth depth gradient to the bright far end. In the third, the metal cabinet sits in front of the back wall at its correct standoff. The flying points along depth boundaries are the three-dimensional footprint of the same occlusion-edge error visible in the error maps of Figure 6.3; the depth quantization on distant surfaces reflects depth’s inverse relation to disparity, since a fixed sub-pixel disparity step spans a growing depth interval with distance. These pairs come from the same low-cost rig whose imperfect rectification we examined in Section 6.4.1; we did no fine-tuning on the rig’s data, so the clouds are the zero-shot output of the checkpoint evaluated throughout this chapter.



Figure 6.8: Left image and point clouds from three viewpoints for real indoor scenes.

#### 6.4.4 Comparison with LiDAR Point Clouds

It is worth placing these reconstructions next to the sensor they are usually measured against. LiDAR is the accuracy standard for depth: it measures range directly by timing a laser pulse, and its points are metrically exact to within a centimetre or two. But that accuracy comes one point at a time. A scanning unit samples the scene along a fixed handful of rotating beams, so its returns land on a small number of scan rings and the space between them is simply not measured. Figure 6.9 shows this directly: a raw Velodyne scan from KITTI [41], on the left, is accurate everywhere it has a point, yet any single surface in it carries only a thin scatter of returns; our stereo cloud of the fire-extinguisher scene, on the right, covers the same kind of surface continuously, giving the full shape of each object and the layout of the scene rather than a sample of it.

For a mobile robot, that difference in density is not a cosmetic one. A robot planning a path or building a map has to know the shape and extent of the obstacles around it, not just the range to a few isolated points on them, and a thin object such as a table edge, a chair leg, or the very extinguisher nozzles in Figure 6.8 can fall between a LiDAR’s scan lines entirely while a dense stereo surface still resolves it. Density, in other words, buys spatial understanding, and it is exactly what a passive stereo cloud provides where a low-cost LiDAR does not.

Against this, the accuracy gap is a reasonable price. The reconstructions here are not

centimetre-exact the way a LiDAR return is; their error grows with distance, as the inverse disparity-to-depth relation of Section 3.9.7 requires, and their depth boundaries carry the flying points visible in the figure. What the trade buys is substantial: a passive stereo rig costs a small fraction of a scanning LiDAR, draws a few watts instead of tens, emits nothing into the scene, and returns a dense surface instead of a sparse one, all from two commodity image sensors. For the edge robots, drones, and assistive devices this thesis targets, where budget, power, and the shape of nearby obstacles matter more than millimetre-accurate ranging, trading a bounded loss of point accuracy for dense, low-cost, passive coverage is the right exchange, and it is the one the whole design is built around.

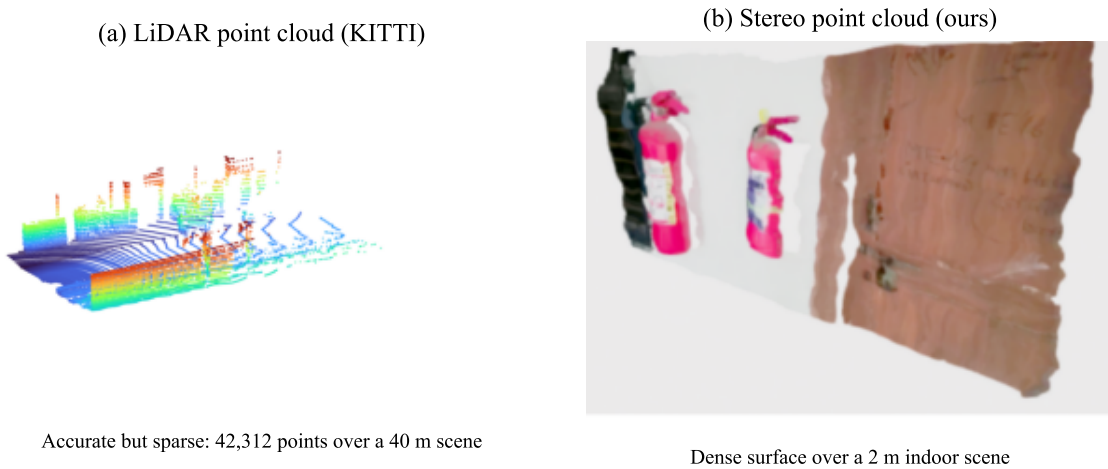


Figure 6.9: A real sparse LiDAR scan (KITTI [41], left) beside our dense stereo reconstruction of the fire-extinguisher scene (right). LiDAR is accurate but samples a surface along a few scan rings; the stereo cloud covers it densely, trading a bounded accuracy loss for dense, low-cost, passive coverage.

## 6.5 Validation

This section validates the design decisions behind the architecture of Chapter 3 through a sequence of controlled ablation studies, run under the matched-harness protocol described there, rather than in a single design pass. We group the studies in three: early exploration that selected the encoder, refinement scheme, and loss on the earlier tile chassis; the ablation of the recurrent family that produced the final model; and training-recipe and efficiency studies on the final model.

### 6.5.1 Design-Space Exploration of the Tile Chassis

An earlier campaign on a smaller tile chassis settled three questions. Table 6.5 compared a GhostConv stack against two truncated detection backbones under a matched 20-pair harness; YOLO26s won on every metric, 0.528 pixel end-point error against 0.625 for Ghost-

Conv and 0.712 for the smaller YOLO26n, at a two-millisecond latency cost, and was selected for the final model. Table 6.6 tested the iterated coarse-to-fine scheme against two alternatives: extending refinement to finer scales gained almost nothing (0.573 against 0.578 pixels) at 37 percent more latency, while HITNet-style single-pass-per-scale propagation collapsed accuracy to 0.844 pixels. Iterated refinement at coarse scales is thus the load-bearing choice, kept in the final model. Table 6.7 compared nine formulations under the same harness; the threshold-stack objective with the D1-relative hinge won on end-point error, bad-2, and D1, and is the ancestor of the production objective of (4.1). The clearest negative result: sequence-weighted multi-scale L1, which imitates the recurrent literature’s iteration-weighted loss without exposing true per-iteration outputs, was the worst of all nine at 1.005 pixels. Faking a sequence loss across scales does not work; the final model instead exposes true multi-scale outputs to the objective.

Table 6.5: Encoder ablation on the tile chassis under the matched 20-pair harness.

| Encoder                    | Params (M) | EPE          | bad-1 (%)    | D1-all (%)  | Latency (ms) |
|----------------------------|------------|--------------|--------------|-------------|--------------|
| GhostConv stack            | 0.538      | 0.625        | 13.31        | 3.54        | 23.5         |
| YOLO26n (truncated)        | 0.808      | 0.712        | 18.07        | 2.90        | 24.1         |
| <b>YOLO26s (truncated)</b> | 2.061      | <b>0.528</b> | <b>10.88</b> | <b>2.50</b> | 25.8         |

Table 6.6: Refinement-scheme ablation on the tile chassis under the matched harness.

| Scheme                         | Params (M) | EPE          | bad-1 (%)    | D1-all (%)  | Latency (ms) |
|--------------------------------|------------|--------------|--------------|-------------|--------------|
| <b>iterated coarse-to-fine</b> | 0.538      | <b>0.578</b> | <b>10.72</b> | 3.44        | <b>23.8</b>  |
| + refinement at 1/2            | 0.576      | 0.573        | 11.12        | <b>3.25</b> | 32.5         |
| single pass per scale          | 0.487      | 0.844        | 19.30        | 4.79        | 25.2         |

Table 6.7: Loss-formulation ablation across nine objectives on the tile chassis.

| Objective                         | EPE          | bad-0.5 (%) | bad-2 (%)   | D1-all (%)  |
|-----------------------------------|--------------|-------------|-------------|-------------|
| plain L1                          | 0.663        | 27.6        | 6.09        | 3.99        |
| sequence-weighted multi-scale L1  | 1.005        | 53.5        | 13.25       | 4.04        |
| L1 + gradient                     | 0.618        | <b>23.9</b> | 5.93        | 3.92        |
| L1 + bad-1 hinge                  | 0.763        | 45.8        | 6.21        | 3.65        |
| cocktail (L1 + grad + bad-1)      | 0.692        | 40.0        | 6.12        | 3.68        |
| cocktail, 0.5-px hinge            | 0.676        | 38.3        | 6.21        | 3.50        |
| threshold stack                   | 0.672        | 38.0        | 5.97        | 3.45        |
| <b>threshold stack + D1 hinge</b> | <b>0.591</b> | 31.5        | <b>5.64</b> | <b>3.35</b> |
| Charbonnier                       | 0.631        | 31.0        | 5.78        | 3.58        |

## 6.5.2 Architecture Ablation of the Recurrent Family

The second campaign built the recurrent context family that became the final model, changing one mechanism at a time on a matched 20-pair harness (Table 6.8). The base design

reached 0.424 pixel end-point error; a learned update gate plus the YOLO26s encoder brought it to 0.319. Rejected additions: a selective recurrent unit (0.474), a fresh quarter-resolution initialization (0.455), and the sharp-tail and high-resolution-refine variants, which gave minor per-metric gains at added latency without improving the headline error. The decisive addition, a quarter-resolution geometry encoding volume with fail-soft fusion, reached 0.261 pixels, the campaign best, at 0.041 million added parameters. Step and batch budgets differ slightly across runs, so the table is an engineering record, not a controlled comparison; the next subsection’s matched studies re-validate these conclusions.

Table 6.8: Architecture ablation of the recurrent family that produced the final model.

| Variant                           | Params (M) | EPE          | Median       | bad-1 (%)   | Latency (ms) |
|-----------------------------------|------------|--------------|--------------|-------------|--------------|
| recurrent context (base)          | 2.831      | 0.424        | 0.167        | 7.62        | 10.7         |
| + geometry-guided state           | 2.919      | 0.459        | 0.218        | 8.16        | 36.7         |
| + update gate                     | 2.921      | 0.473        | 0.246        | 7.75        | 12.2         |
| gate, YOLO26n encoder             | 1.410      | 0.387        | 0.182        | 5.98        | 10.9         |
| gate, YOLO26s encoder             | 2.921      | 0.319        | 0.127        | 5.24        | 11.9         |
| + sharp tail                      | 2.950      | 0.322        | 0.136        | <b>5.21</b> | 14.6         |
| + high-resolution refine          | 3.106      | 0.351        | 0.186        | 5.18        | 16.1         |
| + selective recurrent unit        | 3.398      | 0.474        | 0.255        | 8.20        | 14.8         |
| + raw 1/4 initialization          | 2.948      | 0.455        | 0.195        | 10.64       | 20.7         |
| <b>+ GEV and fail-soft fusion</b> | 2.962      | <b>0.261</b> | <b>0.059</b> | 5.25        | 24.6         |

### 6.5.3 Training-Recipe and Efficiency Studies

Four further controlled studies (Figure 6.10) justify the training recipe and efficiency pass of the final model. The training recipe, not any architectural block, was the largest accuracy lever: in a matched nine-arm comparison, the OpenStereo augmentation triplet lowered the best validation end-point error from 2.778 to 1.921 pixels, a 31 percent reduction well above the noise floor. The edge-inference optimization pass (Table 6.9) held accuracy while cutting latency by roughly a third: exact-equivalence fixes plus the narrow geometry volume brought latency from 46.6 to 30.2 milliseconds, with end-point error unchanged within noise (2.908 to 2.906 pixels). An exact-equivalence class (zero-copy views, batched lookups, hoisted static channels, fused heads) is bitwise or mathematically identical, so its accuracy deltas are pure noise; the narrow geometry band, the one accuracy-relevant change, was validated separately as neutral. The boundary-sharpness study (Table 6.10) selected plane rendering, which cut the bad-0.5 rate from 55.36 to 42.91 percent, the sharpest and fastest result of the study, since rendering from planes replaces a costlier path. Adding the bimodal head or top-k initialization on top of plane rendering worsened results (44.40 and 46.58 percent bad-0.5): the fixes anti-synergize, so we adopted only plane rendering, a negative result that constrains future work. Table 6.11 reports the input-protocol study on both axes: training on native crops cut the native-axis end-point error from 6.672 to 2.869 pixels versus the legacy

global-resize protocol, at no cost on the resized axis, so we adopted the native-crop protocol for the production run.

Table 6.9: Inference-efficiency ablation on the 100-pair harness, 9k steps (RTX 3050 fp32: 106.7 to 61.4 ms, Table 4.2).

| Variant                   | Params (M) | EPE   | bad-1 (%) | D1-all (%) | Latency (ms) |
|---------------------------|------------|-------|-----------|------------|--------------|
| base (gev4)               | 2.962      | 2.908 | 30.32     | 16.31      | 46.6         |
| + exact-equivalence fixes | 2.962      | 3.021 | 32.09     | 17.79      | 35.8         |
| + narrow GEV band         | 2.962      | 2.906 | 32.29     | 16.88      | 30.2         |

Table 6.10: Boundary-sharpness fixes and their composition on a 500-pair windowed split.

| Variant                            | EPE   | bad-0.5 (%)  | D1-all (%) | Latency (ms) |
|------------------------------------|-------|--------------|------------|--------------|
| control (narrow)                   | 2.006 | 55.36        | 12.57      | 22.2         |
| + bundle1 (top-k init)             | 1.982 | 49.95        | 12.14      | 21.9         |
| + bimodal aux head                 | 1.779 | 43.26        | 11.53      | 25.2         |
| <b>+ plane rendering (adopted)</b> | 1.826 | <b>42.91</b> | 12.19      | 15.0         |
| plane + bimodal                    | 1.847 | 44.40        | 11.62      | 25.5         |
| plane + bimodal + bundle1          | 1.853 | 46.58        | 11.93      | 17.2         |

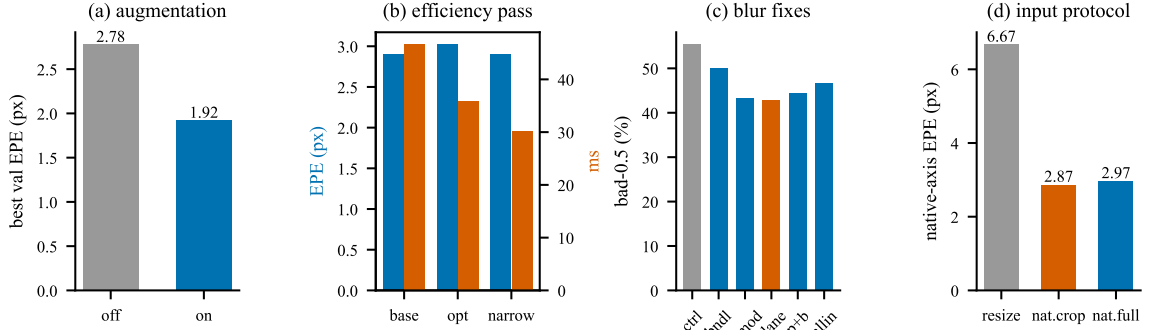


Figure 6.10: Ablation results for augmentation, inference optimization, boundary blur fixes, and input protocol.

Table 6.11: Input-protocol ablation on the dual-axis evaluation (50 validation frames).

| Arm                          | Native axis (960 × 540) |              |              | Resized axis (640 × 384) |              |              |
|------------------------------|-------------------------|--------------|--------------|--------------------------|--------------|--------------|
|                              | EPE                     | bad-1        | D1-all       | EPE                      | bad-1        | D1-all       |
| global resize (legacy)       | 6.672                   | 58.69        | 35.81        | 1.783                    | 29.91        | 11.44        |
| <b>native crop (adopted)</b> | <b>2.869</b>            | <b>36.75</b> | <b>16.47</b> | 1.783                    | <b>29.30</b> | <b>10.71</b> |
| native full frame            | 2.967                   | 38.68        | 16.74        | 2.557                    | 36.45        | 16.36        |

## 6.6 Performance Evaluation

### 6.6.1 Efficiency and Deployment Measurements

Table 6.12 reports the measured efficiency. On the RTX 3050 laptop GPU at batch one and 384 by 640 resolution, the optimized model ran in 49.8 milliseconds at half precision and 61.4 milliseconds at full precision, with inference working sets of 0.26 and 0.35 gigabytes. On the target device, a Jetson Orin Nano, the optimized eight-bit model measured 36.3 milliseconds per frame on-device, or 27.5 frames per second at the same resolution. This nears the 30 frames per second real-time target, confirming interactive rates on the intended edge accelerator, not just the laptop GPU. The on-device working set stays well within the platform’s memory budget.

Table 6.12: Latency and memory across RTX 3050 and Jetson Orin Nano configurations (Jetson memory is estimated).

| Configuration                   | Latency (ms) | FPS  | Inference memory (GB) |
|---------------------------------|--------------|------|-----------------------|
| RTX 3050, fp32                  | 61.4         | 16.3 | 0.35                  |
| RTX 3050, fp16                  | 49.8         | 20.1 | 0.26                  |
| Jetson Orin Nano, INT8 TensorRT | 36.3         | 27.5 | ~0.15                 |

## 6.7 Discussion

These results answer the four research questions posed in Chapter 1. The model reaches 0.78 pixel end-point error within a few million parameters, answering the first. It generalizes zero-shot to an unseen domain at 10.9 percent D1-all, short of the distillation-trained references, answering the second. It runs interactively on a laptop GPU and reached 27.5 frames per second on the Jetson Orin Nano, near the real-time target, answering the third. The fourth question, tolerance to imperfect rectification, is addressed by the perturbation study of Section 6.4.1. Figure 6.11 places the model on the zero-shot accuracy-efficiency front, at the low-parameter end below the larger distillation-trained and iterative references. This comparison bears on an open question about cross-domain generalization: LiteAnyStereo, the strongest efficient reference, is itself non-iterative and owes its accuracy to a three-stage distillation recipe rather than to iteration, which suggests that the training pipeline carries more of the cross-domain gain than architectural iteration does, though our study does not isolate the two factors directly. A few limitations bound these conclusions: the results come from a single run, so run-to-run variance is unknown; the model was not submitted to the KITTI leaderboard; and it lacks left-right consistency or occlusion reasoning. These limits are taken up as future directions in the concluding chapter.

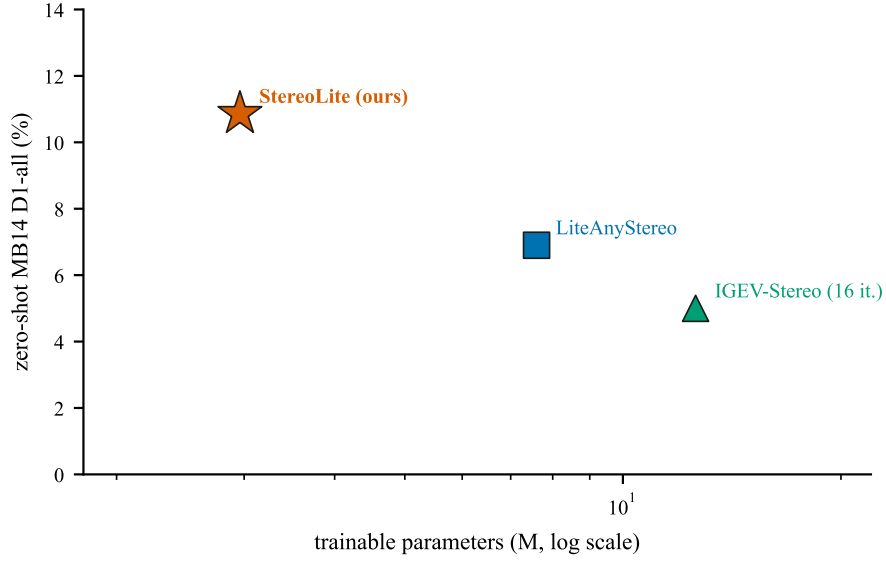


Figure 6.11: Middlebury 2014 D1-all against trainable parameters for three zero-shot models.

## 6.8 Summary

This chapter evaluated the model on every objective axis. In domain, it reached 0.78 pixel end-point error and 3.40 percent D1-all on the FlyingThings3D test set. Zero-shot on unseen Middlebury 2014 scenes, it reached 10.9 percent D1-all under a protocol identical to the reference models, the smallest network in that comparison; remaining failures cluster in scenes with thin repeated structures and disparities near the evaluation cap. Under controlled vertical rectification error, end-point error rose from 1.03 to 1.53 pixels across the sub-pixel to one-pixel range that a calibrated rig produces, and the real-rig agreement study confirmed that tolerance on physical data. On the Jetson Orin Nano, the optimized eight-bit model reached 36.3 milliseconds per frame, near the 30 frames per second real-time goal; real-camera disparity maps and metric point-cloud reconstructions confirmed that the checkpoint transfers to physical data without adaptation.

## Chapter 7

### Societal, Environmental Impact and Sustainability

#### 7.1 Introduction

The value of this work is not exhausted by its benchmark numbers. A depth sensor that runs its whole computation on the device, emits nothing into the scene, and is built from a handful of commodity parts carries consequences for the people around it, for the environment it operates in, and for the wider goals that engineering is meant to serve. This chapter draws those consequences out. Section 7.2 examines the societal, health, and cultural effects that follow from where the computation happens and how the scene is sensed. Section 7.3 turns to safety and to the ethical questions a camera-carrying, autonomy-enabling system raises. Section 7.4 maps the work onto three of the United Nations Sustainable Development Goals and, in doing so, sets out its environmental and sustainability case. Section 7.5 draws the chapter together.

#### 7.2 Impact of the Thesis on Societal, Health, and Cultural Issues

The most direct societal consequence of this work follows from where the computation happens. Because the network runs the depth computation on the device rather than in the cloud, the video stream stays local: no imagery of the people or spaces that a robot, drone, or headset observes ever needs to leave the sensor. This matters for privacy in a way that a cloud-processed depth pipeline cannot match, since a system that transmits continuous video to a remote server creates, by construction, a stream of personal imagery that must then be secured, retained, and governed. Keeping the raw imagery on the device removes that stream at the source. The same property eases the legal and cultural concerns that accompany camera-carrying machines. Regulations on personal-data transfer, and the reasonable discomfort of people who share a space with an observing device, both attach primarily to imagery that is recorded or transmitted; an embedded sensor that consumes its own video internally and emits only geometry is easier to deploy responsibly in homes, workplaces, and public settings.

The choice of sensing modality carries a health implication that is easy to overlook. A passive stereo rig emits no laser and no structured infrared light; it only observes. Bystanders are therefore not exposed to any emitted radiation, a real distinction from active depth sensors, whose emitters must be engineered and certified for eye safety, particularly around children and in crowded indoor spaces. A depth sensor that adds no new emission to a shared room is, by that measure, gentler on the people in it.

Finally, the work has an accessibility dimension with a cultural reach. A compact, low-

cost, passive depth sensor lowers the barrier to robotics and assistive technology in settings where specialized depth hardware is unaffordable. Depth perception built from two commodity image sensors and an embedded accelerator puts spatial sensing within reach of student projects, small workshops, assistive devices, and products aimed at price-sensitive markets, communities that a sensor costing several times as much would simply exclude. Widening who can build and afford spatially aware machines is itself a societal outcome, and it follows directly from the design constraints this thesis set out to satisfy.

### **7.3 Safety and Ethical Considerations**

Safety is the first concern for any system that senses depth so that a machine can move. A mobile robot or drone acts safely in three-dimensional space only to the extent that its depth perception is reliable, and the evaluation of Chapter 6 was built around exactly the failure modes that make depth unreliable in the field: the model was tested not only in its training domain but zero-shot on unseen real scenes, under controlled rectification error, and on the physical stereo rig itself. On-device inference also removes the network round trip from the control loop, so the depth estimate a robot acts on does not depend on the availability or latency of a wireless link; a platform whose obstacle avoidance survives a lost connection is categorically safer than one whose perception stalls with the network. The passive rig reinforces this: with no laser and no projected pattern, the sensor poses no eye-safety hazard to the people it works alongside.

Reliability, though, has limits that honesty requires stating. The network is trained on synthetic data and is not perfect on real scenes; its errors concentrate at depth boundaries and on thin, repeated, or reflective structures, and its accuracy degrades gracefully rather than failing silently, but it degrades. A system that acts on this depth should therefore treat it as one cue among several rather than as ground truth, and this thesis reports the error rates plainly, including where they are worst, so that a downstream integrator can design for them instead of being surprised by them.

The ethical questions extend past safety. A device that perceives three-dimensional space is inherently dual-use: the same depth stream that lets a robot avoid a person can, in another deployment, help track one. The design choices here lean against the harmful use, on-device processing keeps imagery from being collected centrally, and passive sensing avoids the covert illumination that active depth can use, but no architecture can prevent misuse on its own, and it would be dishonest to claim otherwise. Two further points of engineering ethics bear on this work directly. First, the data: the real pairs used for qualification were captured by the authors in their own institutional spaces, with no third parties as subjects, and no personal imagery was collected, retained, or distributed. Second, the reporting: every accuracy, latency, and efficiency figure in this thesis is tied to a specific scripted run whose configuration is logged, comparisons against published methods use each paper’s own re-

ported numbers, and the model’s shortcomings are stated alongside its strengths, so that the record is one a reader can trust and reproduce rather than one tuned to flatter.

## 7.4 Alignment with Sustainable Development Goals (SDGs)

The work aligns with three of the United Nations Sustainable Development Goals. Each connection is concrete rather than aspirational, and the environmental case for the design is made here, under the goal it serves most directly.

**SDG 9: Industry, Innovation and Infrastructure.** This goal calls for resilient infrastructure and for broad, affordable access to the fruits of innovation. A stereo network that delivers usable depth within the memory, latency, and power budget of a low-cost edge board moves a capability that until recently required either expensive sensors or a datacenter onto hardware that small manufacturers, researchers, and startups can actually afford. By showing that a design methodology, controlled ablation under a hard budget, can produce a competitive model at a few million parameters, the thesis also contributes a repeatable way of building efficient perception rather than a single artifact, which is the kind of transferable innovation this goal seeks.

**SDG 11: Sustainable Cities and Communities.** This goal includes safe, accessible, and sustainable mobility and living spaces. Reliable, low-cost, on-device depth perception is an enabling technology for exactly the machines that operate in shared human environments: service and delivery robots, inspection drones, and assistive devices for people with limited mobility or vision. Because the sensor keeps imagery local and emits nothing, it can be deployed in homes, sidewalks, and public buildings with a lighter privacy and safety footprint than a cloud-connected or actively illuminating alternative, which is what makes such deployments socially sustainable rather than merely technically possible.

**SDG 12: Responsible Consumption and Production.** This goal is where the environmental case sits, and it applies to the hardware, the running energy, and the development of the model alike. On the hardware side, a stereo rig uses two commodity image sensors rather than a specialized time-of-flight or LiDAR unit and carries no emitter hardware at all: no laser diode, no pattern projector, and none of the associated drive electronics. Those commodity sensors are manufactured at enormous scale for the phone and webcam markets, so a depth sensor assembled from them draws on an established supply chain rather than specialized parts, and its fewer, more common components make it easier to source, repair, and replace over its service life. On the energy side, the deployed network runs within a few watts to a few tens of watts and computes each frame locally, avoiding the recurring per-frame cost of transmitting video to a datacenter and back; because that transport cost is

paid for every frame over a device’s whole life, removing it compounds over the deployed fleet. On the development side, the production model was trained in roughly one day on a single cloud GPU, and each ablation and evaluation run cost a fraction of a dollar of cloud time, a footprint kept deliberately small by the cheap controlled-ablation harness of Chapter 3, which filtered design candidates before any expensive full training run. Compute, in other words, was spent once, on a design already validated by a cheaper process, which is responsible production applied to model building itself.

## **7.5 Summary**

This chapter traced the design properties of the system to their human and environmental consequences. On-device inference keeps imagery local, which protects the privacy of observed people and eases legal and cultural concerns; passive sensing emits no laser and poses no eye-safety hazard; and the low cost of the sensor widens access to assistive and robotic technology. On safety and ethics, connection-independent depth makes robots and drones safer, the thesis reports its errors honestly rather than only its successes, and it acknowledges the dual-use nature of depth sensing while leaning its design choices against misuse. Finally, the work aligns with three Sustainable Development Goals, affordable innovation (SDG 9), safe machines in shared spaces (SDG 11), and responsible production (SDG 12), and it is under the last of these that the environmental case is made: commodity parts over specialized emitters, local computation over a repeated cloud round trip, and a training footprint of about one GPU-day. These impacts follow directly from the edge-deployment constraints that shaped the design.

# Chapter 8

## Thesis Management and Finance

### 8.1 Introduction

This chapter records how we managed the thesis: the schedule we followed, the resources we drew on, what we spent, and how we split the work between the two of us. Paper collection and the initial literature review began during the fourth-year odd semester (the seventh semester), and the work ran from July 2025 to July 2026 across both semesters. The main experimental period ran from December 2025 to July 2026. Model training and the experimental studies of Chapter 5 used rented cloud GPUs, and we also bought a stereo camera and a Jetson Orin Nano board for real-data capture and on-device deployment testing. Section 8.2 describes how we managed the work across eleven overlapping phases and how we coordinated with our supervisor; Section 8.3 reports the overall project expenditure; Section 8.4 sets out how we planned compute, data, software, and our own time; and Section 8.5 states each author’s individual contribution.

### 8.2 Management of the Thesis

We divided the work into phases with defined outputs and let some phases overlap deliberately so that activities could run in parallel. The literature review guided our choice of candidate architectures, and the setup and preliminary training that followed established the experimental pipeline. We kept capturing real-camera data while refining the architecture through the ablation campaign of Chapter 3, then carried the selected design through the benchmarking, three-dimensional reconstruction, and final evaluation of Chapter 6 before consolidating the results in this book. This overlap let data preparation, experimentation, and writing proceed together, since design decisions still depended on experimental evidence that only became available as the ablation campaign progressed.

We met with our supervisor regularly throughout the project to review progress, discuss design decisions, and plan the next steps. Milestone reviews lined up with the end of the literature study, the architecture design, the ablation campaign, and the production training run, and the supervisor’s feedback at each of those points shaped the direction of the work that followed. We also prepared the final manuscript and the defense presentation in consultation with the supervisor.

8.2.1 Time Frame / Gantt Chart

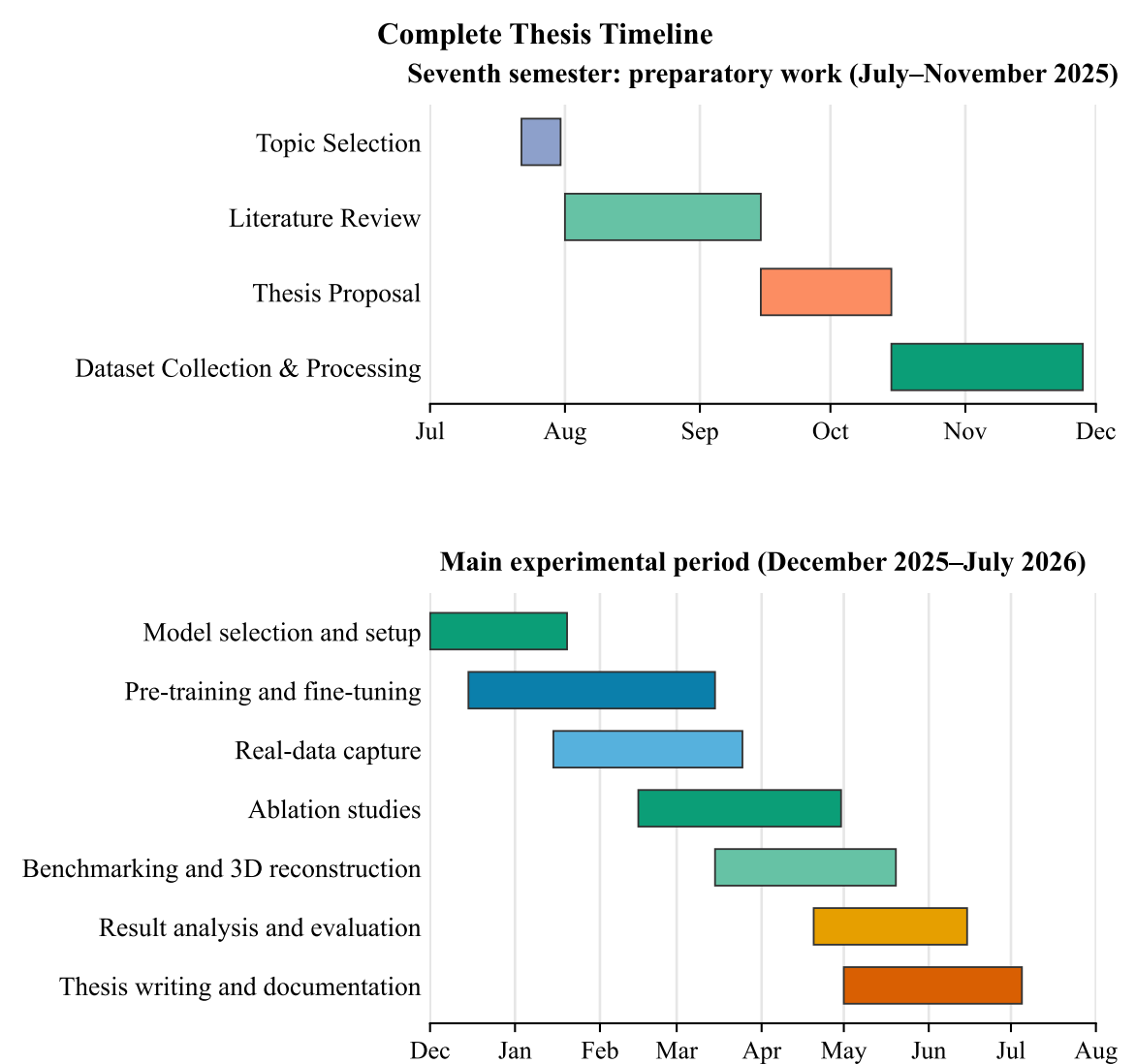


Figure 8.1: Gantt chart of the eleven project phases from July 2025 to July 2026; the main experimental period begins in December 2025.

The full project schedule ran from July 2025 to July 2026 across the fourth-year odd and even semesters. Here, the fourth-year odd semester means the seventh semester, not “Level 4, Term 1.” This semester covered topic selection, the literature review, the thesis proposal, and dataset collection and processing. The main experimental period then ran from December 2025 to July 2026. Table 8.1 lists all eleven phases and their principal outputs, and Figure 8.1 shows how they overlapped.

Table 8.1: Eleven project phases from July 2025 to July 2026.

| Phase                                 | Period                 | Output                                                                 |
|---------------------------------------|------------------------|------------------------------------------------------------------------|
| 1. Topic selection                    | Jul. 2025              | Chosen thesis topic and scope                                          |
| 2. Literature review                  | Aug. to Sep. 2025      | Surveyed paper corpus and research gap                                 |
| 3. Thesis proposal                    | Sep. to Oct. 2025      | Approved proposal                                                      |
| 4. Dataset collection and processing  | Oct. to Nov. 2025      | Prepared training and evaluation datasets                              |
| 5. Model selection and setup          | Dec. 2025 to Jan. 2026 | Selected baseline models and a working experimental environment        |
| 6. Pre-training and fine-tuning       | Dec. 2025 to Mar. 2026 | Initial checkpoints and a stable training procedure                    |
| 7. Real-data capture                  | Jan. to Mar. 2026      | Calibrated and rectified stereo image pairs                            |
| 8. Ablation studies                   | Feb. to Apr. 2026      | Evidence for the final architectural and training choices              |
| 9. Benchmarking and 3D reconstruction | Mar. to May 2026       | Accuracy, efficiency, and reconstruction results                       |
| 10. Result analysis and evaluation    | Apr. to Jun. 2026      | Consolidated in-domain, zero-shot, robustness, and deployment findings |
| 11. Thesis writing and documentation  | May to Jul. 2026       | Final thesis book and supporting documentation                         |

### 8.3 Overall Budget

Table 8.2 lists the approximate project expenditure. Cloud computation made up about one third of the total: the A100 training workload cost approximately USD 100, and the other GPU instances used for ablation and evaluation cost approximately USD 40. The stereo camera and Jetson Orin Nano together cost about BDT 35,000, which is approximately USD 280 at the conversion rate used in this record.

Table 8.2: Project expenditure in USD and BDT across hardware and cloud computing.

| Item                                         | Cost (USD) | Cost (BDT) |
|----------------------------------------------|------------|------------|
| Cloud A100 usage for training                | 100        | 12,500     |
| Other cloud GPUs for ablation and evaluation | 40         | 5,000      |
| Stereo camera and Jetson Orin Nano           | 280        | 35,000     |
| Software licences                            | 0          | 0          |
| Total                                        | 420        | 52,500     |

#### 8.3.1 Component and Software Budget

The budget in Table 8.2 splits into hardware, cloud computing, and software.

Hardware covered the stereo camera and the Jetson Orin Nano, used for real-data acquisition and on-device testing, at a combined cost of about BDT 35,000 (USD 280). Both

count as development hardware for validating the pipeline, not as the deployed system itself.

Cloud computing cost about USD 140 in total: roughly USD 100 on A100 instances for the main training run, and about USD 40 on other GPU instances for ablation studies, intermediate checks, and final evaluation. These are usage totals accumulated over the whole development period, not the rate for a single day, and they will not match a provider’s current hourly tariff, since prices vary by accelerator, region, and storage allocation.

The software stack, described in Chapter 4, uses open-source tools and cost nothing in licence fees. We limited cloud use to development; inference runs locally on the Jetson device, so deployment adds no recurring per-frame cloud charge, only ordinary hardware upkeep and power.

## 8.4 Resource Planning

Beyond the money recorded in Section 8.3, the thesis depended on planning compute, data, software, and our own time so that each phase of Table 8.1 had what it needed when it needed it.

Compute split across three tiers by purpose. Local development and the latency benchmark ran on a single RTX 3050 laptop GPU with roughly 4 gigabytes of memory, which gave us an always-available environment for writing and debugging the training and evaluation code before committing anything to a paid run. The production training run of Chapter 4 used a single rented cloud A100 with 80 gigabytes of memory, sized so the network and its training batch fit comfortably for the one full run recorded in Table 8.2. Ablation studies, intermediate checks, and evaluation instead ran on cheaper rented cloud T4 instances, which cost about a tenth of the A100 workload (Table 8.2) and let us test many architectural variants in Chapter 3 without paying A100 rates for runs that did not need the full card. We committed a design to the A100 tier only after the T4 ablations had already validated it, so the expensive compute trained a design we had confirmed cheaply, rather than searching for that design at A100 prices. The Jetson Orin Nano board, bought alongside the stereo camera in Table 8.2, served a different role again: it is the deployment target itself, and Chapter 6 reports the on-device latency measured directly on it rather than projected from a desktop GPU.

Data planning followed the same tiering. Chapter 5 describes the SceneFlow synthetic corpus and the Middlebury 2014 benchmark that trained and evaluated the network, alongside the real-camera pairs captured with the calibrated binocular rig also described there. The synthetic and benchmark data came from public releases, so the data-collection cost we carried directly was the camera hardware itself, already counted in Table 8.2, together with the time spent calibrating and rectifying our own captures.

Software cost nothing beyond the hardware above. We built the training and evaluation code on PyTorch 2.11 with CUDA 12.8 on Python 3.10 (Chapter 4), and wrote this book in

LaTeX; both toolchains are open source, which kept the software-licence row of Table 8.2 at zero throughout the project.

Finally, we planned our own time against the phase table of Section 8.2.1. Model setup, pre-training, and real-data capture overlapped at the beginning of the main experimental period; ablation studies and benchmarking followed as checkpoints matured; and we reserved the final two months mostly for writing, so that documenting the thesis did not compete with active experimentation for our attention. Section 8.5 records how this time was split between the two of us.

## 8.5 Individual and Team Work Contribution

The thesis was carried out by a two-person team, Md. Raihanul Haque Rahi (Roll 2008011) and Nazib Abrar (Roll 2008026), under the supervision of Md Zunaid Hossen (Lecturer, Department of Mechatronics Engineering), who guided the work as described in Section 8.2 but is not a co-author of this thesis. Rahi led the architecture and training side of the work: proposing the tile-based design, writing the model and training code, and running the investigation and validation that the ablation campaign of Chapter 3 depended on. Abrar led the evaluation and tooling side: building the evaluation pipeline, curating the real-camera and benchmark data described in Chapter 5, and running the validation studies that fed Chapter 6.

Both authors worked across most categories rather than staying inside a single role, which is typical of a two-person thesis team where every phase in Table 8.1 needs both people at once. Table 8.3 records each author’s primary responsibilities against the CRediT (Contributor Roles Taxonomy) categories.

Table 8.3: Primary CRediT responsibilities of each author.

| Author |                |                | Primary CRediT responsibilities                                                                                                                          |
|--------|----------------|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Md.    | Raihanul Haque |                | Conceptualization, Methodology, Software (architecture and training code), Investigation, Validation, Writing (original draft), Visualization            |
| Rahi   | (Roll 2008011) |                |                                                                                                                                                          |
| Nazib  | Abrar          | (Roll 2008026) | Conceptualization, Methodology, Software (evaluation and tooling), Data curation, Investigation, Validation, Writing (review and editing), Visualization |

Both authors reviewed and approved the final manuscript.

## 8.6 Summary

This chapter recorded how we managed and financed the thesis. The overall schedule ran from July 2025 to July 2026 and consisted of eleven overlapping phases. The main experimental period ran from December 2025 to July 2026, from model setup and training through final documentation, punctuated by milestone reviews with our supervisor. Total expenditure

was about USD 420 (BDT 52,500): USD 140 for cloud GPU usage and USD 280 for the stereo camera and Jetson Orin Nano. We planned compute across three tiers (an RTX 3050 laptop GPU for local development, cloud T4 instances for ablations, and a single cloud A100 for production training), used open-source software throughout at no licence cost, and split the work so that architecture and training fell primarily to Rahi and evaluation and tooling fell primarily to Abrar, with both authors reviewing every stage of the manuscript. Because inference runs locally on the Jetson device, deployment itself adds no recurring cloud charge beyond this recorded development cost.

## Chapter 9

### Conclusions, Limitations and Future Works

#### 9.1 Conclusion

This thesis set out to build an edge-deployable deep stereo network meeting two objectives. The first, an efficient AI-refined disparity pipeline for resource-limited platforms, is met by StereoLite: 2.96 million parameters, 0.78 pixel end-point error on the SceneFlow test set, 10.9 percent D1-all zero-shot on Middlebury 2014, and 49.8 milliseconds at half precision on a laptop RTX 3050. The optimized eight-bit form measured 36.3 milliseconds on the Jetson Orin Nano, 27.5 frames per second, near the real-time target. The second, tolerance to imperfect rectification, is met by the perturbation study in Section 6.4.1: accuracy holds across the sub-pixel to one-pixel vertical misalignment a calibrated rig produces, with end-point error rising only from 1.03 to 1.53 pixels from the local correlation lookup in the refinement stage.

The contributions of this thesis are the following.

- StereoLite, a 2.96 million parameter edge-budget stereo network, combines a truncated YOLO feature encoder, a left-image recurrent context stream, plane-aware coarse-to-fine tile refinement, and a fail-soft quarter-resolution geometry encoding volume.
- A controlled training pipeline combining native-crop sampling, augmentation, and full-dataset training gives the compact tile-hypothesis model its cross-domain accuracy, reaching 10.9 percent zero-shot Middlebury 2014 D1-all, within four points of the larger distillation-trained LiteAnyStereo.
- An equivalence-proven inference-optimization pass cuts latency by a factor of 1.74 with no loss of accuracy and identifies the operators that block eight-bit deployment.
- The model reaches 0.78 pixel end-point error on the SceneFlow test set with real-time inference on a laptop GPU and transfers zero-shot to an unseen benchmark and a real camera without adaptation.

#### 9.2 Limitations

This work has several limitations. Results come from a single training run, so run-to-run variance is not characterized. A gap remains in cross-domain accuracy: at 10.9 percent zero-shot D1-all the model trails the distillation-trained LiteAnyStereo (6.9 percent) and the iterative IGEV-Stereo (5.0 percent). The disparity search spans 192 pixels at the 384 by 640 inference resolution, matching the evaluation protocol; supervision is synthetic only; and

the model is not yet on the KITTI leaderboard. Architecturally, recurrent reasoning stops at one quarter resolution, limiting recovery of thin and distant structures, and the network lacks left-right consistency and occlusion reasoning. The rectification tolerance measured in 6 holds only for small vertical offsets, not for large de-rectification.

### 9.3 Future Works

Several directions follow from these limitations, ordered by expected return. The most promising is a three-stage knowledge-distillation pipeline following LiteAnyStereo: synthetic training with dense supervision as in this thesis, then self-distillation under input perturbation with a frozen copy of the model supplying feature targets, then distillation from a frozen foundation teacher on unlabeled real pairs, using the teacher pipeline built for this project. The cross-domain evidence in 6 points to the training pipeline as the largest remaining lever: failure concentrates in a scene type real-data distillation addresses, and the current supervised-only recipe leaves it unused. A second direction is deployment hardening: the eight-bit TensorRT engine already runs on the Jetson Orin Nano at 36.3 milliseconds (from 6), so the next step is a full accuracy audit of the quantized engine against the full-precision model and a port to other edge boards in the same class. A third is completing the zero-shot evaluation quartet with KITTI and ETH3D to characterize generalization beyond the indoor domain, across driving and mixed domains; both datasets are staged in the project’s evaluation infrastructure. A fourth is a temporal extension that propagates tile-plane state across frames, since a small ego-motion warps a slanted-plane hypothesis predictably and would amortize initialization cost over the video stream. Finally, the reconstruction path discussed in 6 invites a mapping extension that fuses per-frame point clouds into a persistent scene model. A compact tile-hypothesis network trained with the distillation pipeline above would serve as an off-the-shelf edge stereo component, the natural next step for this work.

## References

- [1] B. Wen, M. Trepte, J. Aribido, J. Kautz, O. Gallo, and S. Birchfield, “Foundation-stereo: Zero-shot stereo matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [2] D. Scharstein and R. Szeliski, “A taxonomy and evaluation of dense two-frame stereo correspondence algorithms,” *International Journal of Computer Vision*, vol. 47, no. 1–3, pp. 7–42, 2002.
- [3] M. Poggi, F. Tosi, K. Batsos, P. Mordohai, and S. Mattoccia, “On the synergies between machine learning and binocular stereo for depth estimation from images: a survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5314–5334, 2021.
- [4] H. Laga, L. V. Jospin, F. Boussaid, and M. Bennamoun, “A survey on deep learning techniques for stereo-based depth estimation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 4, pp. 1738–1764, 2022.
- [5] H. Hirschmüller and D. Scharstein, “Evaluation of cost functions for stereo matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2007, pp. 1–8.
- [6] R. Zabih and J. Woodfill, “Non-parametric local transforms for computing visual correspondence,” in *European Conference on Computer Vision (ECCV)*, 1994, pp. 151–158.
- [7] J. Žbontar and Y. LeCun, “Stereo matching by training a convolutional neural network to compare image patches,” *Journal of Machine Learning Research*, vol. 17, no. 65, pp. 1–32, 2016.
- [8] H. Hirschmüller, “Stereo processing by semiglobal matching and mutual information,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, 2008.
- [9] N. Mayer, E. Ilg, P. Häusser, P. Fischer, D. Cremers, A. Dosovitskiy, and T. Brox, “A large dataset to train convolutional networks for disparity, optical flow, and scene flow estimation,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [10] A. Kendall, H. Martirosyan, S. Dasgupta, P. Henry, R. Kennedy, A. Bachrach, and A. Bry, “End-to-end learning of geometry and context for deep stereo regression,” in *IEEE International Conference on Computer Vision (ICCV)*, 2017.

- [11] J.-R. Chang and Y.-S. Chen, “Pyramid stereo matching network,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [12] X. Guo, K. Yang, W. Yang, X. Wang, and H. Li, “Group-wise correlation stereo network,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [13] F. Zhang, V. Prisacariu, R. Yang, and P. H. S. Torr, “Ga-net: Guided aggregation net for end-to-end stereo matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [14] H. Xu and J. Zhang, “Aanet: Adaptive aggregation network for efficient stereo matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [15] G. Xu, J. Cheng, P. Guo, and X. Yang, “Attention concatenation volume for accurate and efficient stereo matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [16] Z. Shen, Y. Dai, and Z. Rao, “Cfnet: Cascade and fused cost volume for robust stereo matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [17] L. Lipson, Z. Teed, and J. Deng, “Raft-stereo: Multilevel recurrent field transforms for stereo matching,” in *International Conference on 3D Vision (3DV)*, 2021.
- [18] G. Xu, X. Wang, X. Ding, and X. Yang, “Iterative geometry encoding volume for stereo matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [19] J. Li, P. Wang, P. Xiong, T. Cai, Z. Yan, L. Yang, J. Liu, H. Fan, and S. Liu, “Practical stereo matching via cascaded recurrent network with adaptive correlation,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [20] X. Wang, G. Xu, H. Jia, and X. Yang, “Selective-stereo: Adaptive frequency information selection for stereo matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [21] Z. Li, X. Liu, N. Drenkow, A. Ding, F. X. Creighton, R. H. Taylor, and M. Unberath, “Revisiting stereo depth estimation from a sequence-to-sequence perspective with transformers,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [22] H. Jiang, Z. Lou, L. Ding, R. Xu, M. Tan, W. Jiang, and R. Huang, “Defom-stereo: Depth foundation model based stereo matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

- [23] J. Cheng, L. Liu, G. Xu, X. Wang, Z. Zhang, Y. Deng, J. Zang, Y. Chen, Z. Cai, and X. Yang, “Monster: Marry monodepth to stereo unleashes power,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [24] L. Bartolomei, F. Tosi, M. Poggi, and S. Mattoccia, “Stereo anywhere: Robust zero-shot deep stereo matching even where either stereo or mono fail,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [25] F. Shamsafar, S. Woerz, R. Rahim, and A. Zell, “Mobilestereonet: Towards lightweight deep networks for stereo matching,” in *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2022.
- [26] X. Gu, Z. Fan, S. Zhu, Z. Dai, F. Tan, and P. Tan, “Cascade cost volume for high-resolution multi-view stereo and stereo matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [27] B. Xu, Y. Xu, X. Yang, W. Jia, and Y. Guo, “Bilateral grid learning for stereo matching networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [28] S. Duggal, S. Wang, W.-C. Ma, R. Hu, and R. Urtasun, “Deeppruner: Learning efficient stereo matching via differentiable patchmatch,” in *IEEE International Conference on Computer Vision (ICCV)*, 2019.
- [29] V. Tankovich, C. Häne, Y. Zhang, A. Kowdle, S. Fanello, and S. Bouaziz, “Hitnet: Hierarchical iterative tile refinement network for real-time stereo matching,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [30] X. Guo, C. Zhang, Y. Lu, D. Lan, Y. Wang, L. Wang, and L. Chen, “Lightstereo: Channel boost is all you need for efficient 2d cost aggregation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [31] Y. Pan, W. Pi, X. Hou, X. Wei, and X. Liu, “Distill-then-prune: An efficient compression framework for real-time stereo matching network on edge devices,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [32] J. Jing, W. Mao, Y. Wang, Y. Cao, B. Yu, and L. Mai, “Liteanystereo: Real-time cross-domain stereo matching with foundation knowledge,” *arXiv preprint*, 2025.
- [33] A. Bangunharcana, J. W. Cho, S. Lee, I. S. Kweon, K.-S. Kim, and S. Kim, “Correlate-and-excite: Real-time stereo matching via guided cost volume excitation,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021.

- [34] G. Xu, H. Zhou, and X. Yang, “Cgi-stereo: Accurate and real-time stereo matching via context and geometry interaction,” *arXiv preprint arXiv:2301.02789*, 2023.
- [35] Y. Wang, Z. Lai, G. Huang, B. H. Wang, L. van der Maaten, M. Campbell, and K. Q. Weinberger, “Anytime stereo image depth estimation on mobile devices,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2019.
- [36] A. Tonioni, F. Tosi, M. Poggi, S. Mattoccia, and L. Di Stefano, “Real-time self-adaptive deep stereo,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [37] X. Cheng, Y. Zhong, M. Harandi, Y. Dai, X. Chang, T. Drummond, H. Li, and Z. Ge, “Hierarchical neural architecture search for deep stereo matching,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [38] F. Zhang, X. Qi, R. Yang, V. Prisacariu, B. Wah, and P. Torr, “Domain-invariant stereo matching networks,” in *European Conference on Computer Vision (ECCV)*, 2020.
- [39] T. Chang, X. Yang, T. Zhang, and M. Wang, “Domain generalized stereo matching via hierarchical visual transformation,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [40] X. Guo, C. Zhang, D. Nie, W. Zheng, Y. Zhang, and L. Chen, “Stereo anything: Unifying stereo matching with large-scale mixed data,” in *arXiv:2411.14053*, 2024.
- [41] A. Geiger, P. Lenz, and R. Urtasun, “Are we ready for autonomous driving? the kitti vision benchmark suite,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012.
- [42] M. Menze and A. Geiger, “Object scene flow for autonomous vehicles,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [43] D. Scharstein, H. Hirschmüller, Y. Kitajima, G. Krathwohl, N. Nešić, X. Wang, and P. Westling, “High-resolution stereo datasets with subpixel-accurate ground truth,” in *German Conference on Pattern Recognition (GCPR)*, 2014.
- [44] T. Schöps, J. L. Schönberger, S. Galliani, T. Sattler, K. Schindler, M. Pollefeys, and A. Geiger, “A multi-view stereo benchmark with high-resolution images and multi-camera videos,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [45] NVIDIA, “Jetson orin nano series modules data sheet,” NVIDIA Corporation technical brief, 2024.

- [46] Waveshare, “Ar0144 stereo usb camera (a) product specifications,” Waveshare Electronics. Available at: <https://www.waveshare.com/product/ai/cameras/binocular-cameras/ar0144-stereo-usb-camera-a.htm>, Retrieved: July, 2026, 2026.

## Appendix A

### List of Modern Tools Usage

This appendix collects the modern engineering tools, software libraries, and hardware platforms used to design, train, optimize, deploy, and document StereoLite. The choices favor open, widely supported tools so that the work can be reproduced without proprietary dependencies beyond the cloud compute itself. Table A.1 lists the software and Table A.2 the computing and sensing hardware.

Table A.1: Software tools and libraries used in the thesis.

| Tool / library   | Version / type      | Role in the thesis                                  |
|------------------|---------------------|-----------------------------------------------------|
| Python           | 3.10                | Implementation language for all the code.           |
| PyTorch          | 2.11                | Model definition, autograd, and training framework. |
| CUDA             | 12.8                | GPU backend for training and inference.             |
| Ultralytics YOLO | YOLO26s             | Pretrained detection backbone for the encoder.      |
| NumPy            | array library       | Disparity, metric, and calibration processing.      |
| OpenCV           | 4.x                 | Image I/O, stream splitting, and resizing.          |
| Open3D           | 0.18                | Point-cloud build, filtering, and rendering.        |
| Matplotlib       | plotting            | Disparity, training-curve, and comparison figures.  |
| Modal            | cloud orchestration | Cloud A100 and T4 training and ablation jobs.       |
| NVIDIA TensorRT  | INT8 export         | Graph optimization and eight-bit engine build.      |

Table A.2: Computing and sensing hardware used in the thesis.

| Hardware          | Class              | Role in the thesis                                |
|-------------------|--------------------|---------------------------------------------------|
| NVIDIA A100 80 GB | cloud GPU          | Full-data production training of the final model. |
| NVIDIA T4         | cloud GPU          | Low-cost ablations and cross-domain evaluations.  |
| NVIDIA RTX 3050   | laptop GPU (~4 GB) | Conservative local latency benchmark.             |
| Jetson Orin Nano  | edge accelerator   | Deployment target; on-device eight-bit latency.   |
| Waveshare AR0144  | stereo USB camera  | Low-cost binocular rig for real-camera capture.   |

## Appendix B

### Program Outcome Mapping

Table B.1 maps each program outcome to the section of this thesis that demonstrates it, with a one-line statement of the evidence.

Table B.1: Program outcome attainment mapped to thesis sections.

| PO   | Location                                      | Evidence                                                                                                                  |
|------|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| PO1  | Ch. 1                                         | Engineering knowledge of stereo geometry, sensing trade-offs, and the depth-from-disparity relation.                      |
| PO2  | Ch. 1, 2, 3                                   | Problem analysis: the accuracy-latency-memory constraint, the literature gap, and the formal design requirements.         |
| PO3  | Ch. 3                                         | Design and development of the StereoLite architecture block by block.                                                     |
| PO4  | Ch. 5, 6                                      | Investigation through controlled ablation studies and in-domain, cross-domain, and robustness evaluation.                 |
| PO5  | Ch. 4, App. A, F                              | Modern tool usage: the tools listed in Appendix A, PyTorch, cloud GPU training, mixed precision, TensorRT, and profiling. |
| PO6  | Ch. 7                                         | Engineer and society: on-device privacy and accessibility of a low-cost depth sensor.                                     |
| PO7  | Ch. 7                                         | Environment and sustainability: passive sensing, low power, and a transparent training footprint.                         |
| PO8  | Ch. 7, App. F.4, F.4                          | Ethics: the safety and ethical considerations of Chapter 7, the AI-usage disclosure, and the similarity report.           |
| PO9  | Ch. 8 (Individual and Team Work Contribution) | Individual and teamwork: the CRediT contribution statement of the two authors.                                            |
| PO10 | Ch. 8, whole thesis                           | Communication: the written report, figures, and the supervisor-consultation record.                                       |
| PO11 | Ch. 8                                         | Project management and finance: the timeline and the itemized expense record.                                             |
| PO12 | Ch. 9                                         | Life-long learning: the future-work directions grounded in the field's trajectory.                                        |

## Appendix C

### Addressing Knowledge Profile (KPs) in the Thesis

The Washington Accord’s knowledge-profile scheme, adopted by the BAETE program-outcome system, describes the kind of knowledge an engineering graduate must draw on to solve a genuinely complex problem. A problem only qualifies as complex when it cannot be resolved by natural science or mathematics alone and instead demands in-depth engineering knowledge of a specific, named kind. This appendix identifies which knowledge profiles we drew on in this thesis and points to the chapter where each one is exercised. Appendix D then uses this mapping to ground the mandatory depth-of-knowledge attribute of the thesis’s Complex Engineering Problem.

Designing the stereo-matching network at the center of this thesis drew on four such profiles. It required in-depth engineering knowledge of the stereo pipeline, specialist knowledge of the deep-stereo literature and its building blocks, the design of a novel architecture under hard resource constraints, and grounding in a large body of research literature to select and combine the right mechanisms. In Washington Accord terms these four demands correspond to profiles K3, K4, K5, and K8. Table C.1 lists each one together with the chapter where we exercised it.

Table C.1: Knowledge-profile attainment mapped to thesis chapters.

| Profile | Definition                    | Where exercised in this thesis                                                                                                                             |
|---------|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| K3      | Engineering fundamentals      | The stereo-vision pipeline and cost-volume mechanisms surveyed in Chapter 2 and formalized in the network design of Chapter 3.                             |
| K4      | Specialist knowledge          | The selection and adaptation of tile-hypothesis, recurrent, and geometry-volume building blocks from the deep-stereo literature, carried out in Chapter 3. |
| K5      | Engineering design            | The design of the StereoLite architecture in Chapter 3 and its inference-optimization pass under hard resource constraints in Chapter 4.                   |
| K8      | Research-literature grounding | The literature review of Chapter 2, which grounds every design choice in published evidence rather than in intuition alone.                                |

K3 and K8 sit mostly upstream of the design, in the fundamentals and the literature that make a principled design possible in the first place. K4 and K5 sit inside the design itself, in the specific blocks we chose and the way we put them together under a fixed parameter and latency budget. K5 alone also reaches downstream of the design: an engineering design is only as good as its validation, so the same K5 demand carries into the test protocol of Chapter 5 and the measured outcomes of Chapter 6, where the choices made under K4 and

K5 are checked against held-out data rather than assumed to hold.

None of these four profiles could be dropped without the thesis becoming a routine engineering exercise. Without K3 or K8 the architecture would have no principled basis; without K4 or K5 the literature review would have no original design to apply it to. Appendix D treats this combination as the evidence for the mandatory depth-of-knowledge attribute of the thesis's Complex Engineering Problem.

## **Appendix D**

### **Addressing Complex Engineering Problems**

The Washington Accord's accreditation framework, adopted by the BAETE program-outcome system, distinguishes routine engineering work from work that addresses a Complex Engineering Problem (CEP). A problem is complex when it cannot be resolved without in-depth engineering knowledge. This complexity is captured by a set of P attributes, the first of which, depth of knowledge, is mandatory. This appendix maps each P attribute to the concrete parts of this thesis that exhibit it.

Designing the network at the center of this thesis is a complex engineering problem in the formal sense: it cannot be solved by natural science or mathematics alone. It required in-depth engineering knowledge of the stereo pipeline, specialist knowledge of the deep-stereo literature and its building blocks, the design of a novel architecture under hard resource constraints, and grounding in a large body of research literature to select and combine the right mechanisms. Appendix C maps these four demands onto the Washington Accord's K3, K4, K5, and K8 knowledge profiles and shows where we exercised each one.

Table D.1 maps the P attributes of the Washington Accord definition to the parts of this thesis that satisfy them. Attribute P1 is mandatory and is carried by the knowledge profile detailed in Appendix C; the remaining rows record where the thesis exhibits the other characteristics of a complex problem.

The attributes in the table are not asserted abstractly; each points to a concrete artifact. The conflicting requirements of P2 are the hard design envelope stated in Chapter 3 and verified by the efficiency measurements of Chapter 6. The depth of analysis of P3 is the ablation record of Chapter 6, in which we added candidate mechanisms one at a time under a matched protocol and rejected several because they degraded accuracy. The interdependence of P7 is visible in the same record: the value of one block depends on which others are present, which is why we proceeded by controlled composition rather than independent selection.

This appendix has shown that the thesis addresses a complex engineering problem in the Washington Accord sense. The mandatory attribute P1 is satisfied through the knowledge profile of Appendix C, and Table D.1 grounds the remaining P attributes in concrete parts of the thesis: the conflicting requirements of the edge envelope, the ablation-driven depth of analysis, and the interdependence of the architectural blocks. Appendix E turns to the activities we carried out to reach this design and shows that they, too, meet the Washington Accord's threshold for complexity.

Table D.1: Complex Engineering Problem attributes mapped to thesis sections.

| Attribute | Definition                        | Evidence in this thesis                                                                                                                                                                                                                                        |
|-----------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P1        | Depth of knowledge required       | Cannot be solved without in-depth engineering knowledge (Appendix C): K3 stereo fundamentals (Chapters 2, 3), K4 specialist deep-stereo knowledge (Chapter 3), K5 design under hard constraints (Chapters 3, 4), K8 research-literature grounding (Chapter 2). |
| P2        | Range of conflicting requirements | Accuracy, latency, memory, and parameter count pull against one another; the design criteria of Chapter 3 demand balance across all of them jointly, and the studies of Chapter 6 resolve the conflicts by measurement.                                        |
| P3        | Depth of analysis required        | No obvious correct architecture exists; we analyzed candidates through controlled ablation (Chapter 6) rather than selecting them by inspection, and rejected several plausible additions on evidence.                                                         |
| P4        | Familiarity of issues             | The problem involves infrequently encountered issues: cross-domain generalization failure, surfaced through the test protocol of Chapter 5 and documented in Chapter 6, and edge-inference optimization, both beyond routine practice (Chapter 4).             |
| P5        | Extent of applicable codes        | The work operates outside prescriptive engineering codes; community benchmark protocols and metric definitions (Chapter 3) are the closest standards, and we had to apply and match them carefully rather than follow them mechanically.                       |
| P6        | Extent of stakeholder involvement | The design serves stakeholders with differing needs: platform builders needing real-time latency, end users needing privacy and safety, and cost-sensitive deployments, needs examined in this thesis’s socio-economic impact analysis.                        |
| P7        | Interdependence                   | The system is composed of interacting sub-parts, encoder, initialization, recurrent refinement, geometry volume, and upsampling, whose interactions the ablations of Chapter 6 show are not separable.                                                         |

## Appendix E

### Addressing Complex Engineering Activities

The Washington Accord also characterizes complex engineering activities by five A attributes, distinct from the P attributes of a complex problem discussed in Appendix D. An activity is complex when carrying it out demands diverse resources, conflicting requirements, innovation, and the other characteristics set out below. The work we carried out in this thesis exhibits each of the five.

**A1, range of resources.** We used several distinct kinds of resources: rented cloud GPUs for training and ablation, multiple public stereo datasets for training and cross-domain evaluation, a physical calibrated stereo camera rig for real-data experiments, an embedded accelerator as the deployment target, and the open-source deep-learning tooling on which the implementation of Chapter 4 is built. The management of these resources, including their cost, is documented in this thesis’s management and finance account.

**A2, level of interaction.** The project’s central technical interactions are the conflicts among accuracy, latency, and memory: improving one routinely degrades another. We could not resolve these conflicts analytically. We resolved them instead through the controlled ablation methodology of Chapter 3, under which every candidate change was measured on all metrics jointly and adopted only when the trade-off was favorable. Chapter 6 records these studies.

**A3, innovation.** The activity was creative rather than routine. The architecture of Chapter 3 is a novel composition of building blocks adapted from distinct families of the literature. It also includes the fail-soft fusion of the geometry encoding volume, a mechanism we designed in this work so that the added evidence sharpens the estimate when it is reliable and degrades gracefully, rather than catastrophically, when it is not.

**A4, consequences to society and the environment.** The activity has identifiable consequences beyond the laboratory: on-device privacy, robot and drone safety, eye-safe passive sensing, accessibility of low-cost depth, energy saved by avoiding cloud round trips, and a transparent training footprint. This thesis’s socio-economic impact analysis examines these consequences.

**A5, familiarity.** The activity extends beyond textbook material and prior experience. Its subject matter, compact recurrent stereo architectures, cross-domain generalization, and edge-inference optimization, exists only in the recent research literature we reviewed in Chapter 2. Engaging with it required reading, evaluating, and adapting published research rather than applying established practice.

This appendix has shown that the work carried out in this thesis constitutes complex engineering activities in the Washington Accord sense. The five A attributes are exhibited through the range of resources we used, the measured resolution of conflicting requirements,

the novel block composition with fail-soft fusion, the societal and environmental consequences examined in the socio-economic impact analysis, and the beyond-textbook research grounding of Chapter 2. Together with Appendix D, this completes the case that the thesis addresses both a complex engineering problem and complex engineering activities.

## Appendix F

### Technical Specifications

#### F.1 Model Configuration

The trained architecture is the `gev4_opt_narrow_plane` variant of the StereoLite family; Table F.1 lists the full configuration.

Table F.1: Complete architecture configuration of the trained checkpoint.

| Hyperparameter                      | Value                                    |
|-------------------------------------|------------------------------------------|
| Backbone                            | YOLO26s layers 0–6, pretrained           |
| Context base / output channels      | 24 / 32                                  |
| Tile hidden channels                | 32                                       |
| Refinement-head hidden channels     | 48                                       |
| Initialization groups / disparities | 8 / 24 (at 1/16)                         |
| GEV band                            | $\pm 16$ around tile $d$ (33 bins)       |
| GEV geometry / 3-D hidden channels  | 16 / 16                                  |
| Local correlation half-range        | 2 (five offsets)                         |
| Refinements at 1/16, 1/8, 1/4       | 2, 3, 3                                  |
| Plane rendering                     | on (gated slant supervision, weight 0.3) |
| Trainable parameters                | 2.9631 M                                 |

## F.2 Per-Scene Zero-Shot Results

Table F.2: Per-scene zero-shot results on Middlebury 2014.

| Scene      | EPE (px) | bad-1 (%) | bad-2 (%) | D1-all (%) |
|------------|----------|-----------|-----------|------------|
| Adirondack | 0.59     | 11.4      | 4.0       | 2.38       |
| Backpack   | 0.92     | 13.1      | 8.5       | 6.35       |
| Bicycle1   | 1.64     | 24.4      | 16.3      | 12.55      |
| Cable      | 0.92     | 16.0      | 8.4       | 5.83       |
| Classroom1 | 1.08     | 21.5      | 12.2      | 7.56       |
| Couch      | 1.86     | 25.4      | 14.5      | 8.79       |
| Flowers    | 3.10     | 39.5      | 33.1      | 30.30      |
| Jadeplant  | 9.25     | 57.2      | 44.1      | 34.85      |
| Mask       | 3.14     | 37.4      | 28.4      | 24.68      |
| Motorcycle | 1.04     | 14.4      | 8.2       | 6.33       |
| Piano      | 0.92     | 20.2      | 10.8      | 7.25       |
| Pipes      | 2.26     | 30.0      | 21.7      | 17.90      |
| Playroom   | 1.51     | 31.3      | 17.8      | 12.62      |
| Playtable  | 0.85     | 21.3      | 8.0       | 4.36       |
| Recycle    | 0.59     | 12.1      | 4.3       | 2.90       |
| Shelves    | 0.94     | 25.2      | 11.0      | 6.34       |
| Shopvac    | 1.93     | 22.9      | 12.2      | 7.71       |
| Sticks     | 0.95     | 18.8      | 8.3       | 5.77       |
| Storage    | 0.83     | 10.8      | 5.6       | 4.17       |
| Sword1     | 0.94     | 16.5      | 9.0       | 6.46       |
| Sword2     | 1.42     | 17.9      | 12.1      | 9.55       |
| Umbrella   | 1.15     | 28.8      | 12.6      | 9.08       |
| Vintage    | 1.61     | 33.7      | 21.7      | 15.96      |
| Mean       | 1.71     | 23.9      | 14.5      | 10.86      |

## F.3 Software and Hardware Environment

The model is implemented in PyTorch 2.11 with CUDA 12.8 on Python 3.10. Training runs on a single cloud A100 with 80 gigabytes of memory, ablations and evaluations on a cloud T4, and the local latency benchmark on an RTX 3050 laptop GPU with roughly 4 gigabytes of memory. Real-camera pairs come from a calibrated binocular rig that streams a 2560 by 720 side-by-side image. The intended deployment target is the Jetson Orin Nano class of device.

## F.4 Reproduction Pointers

We can reproduce the reported synthetic-benchmark numbers from the project repository at <https://github.com/abrar-nazib/StereoLite>. The production model is trained by the driver `train_full_sceneflow_a100.py`, the SceneFlow test number by `eval_full_testset.py`, the zero-shot Middlebury 2014 result by

`eval_gev4_middlebury2014.py`, and the rectification-robustness sweep of the second objective by `eval_rectification_robustness.py`. Each driver records its configuration and metrics to a metadata file alongside the checkpoint. The 997-pair real-rig agreement study uses an in-house capture set that we will release with a later, expanded version of that dataset.

## **Appendix G**

### **Plagiarism Report**

The plagiarism (similarity) report is attached at the end of this document.

# 7% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

## Match Groups

-  **137 Not Cited or Quoted 6%**  
Matches with neither in-text citation nor quotation marks
-  **25 Missing Quotations 1%**  
Matches that are still very similar to source material
-  **5 Missing Citation 0%**  
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**  
Matches with in-text citation present, but no quotation marks

## Top Sources

- 6%  Internet sources
- 6%  Publications
- 1%  Submitted works (Student Papers)

## Integrity Flags

### 1 Integrity Flag for Review

-  **Hidden Text**  
9 suspect characters on 1 page  
Text is altered to blend into the white background of the document.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

## Match Groups

-  **137 Not Cited or Quoted** 6%  
Matches with neither in-text citation nor quotation marks
-  **25 Missing Quotations** 1%  
Matches that are still very similar to source material
-  **5 Missing Citation** 0%  
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted** 0%  
Matches with in-text citation present, but no quotation marks

## Top Sources

- 6%  Internet sources
- 6%  Publications
- 1%  Submitted works (Student Papers)

## Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

|    |             |                                                                                   |     |
|----|-------------|-----------------------------------------------------------------------------------|-----|
| 1  | Internet    | arxiv.org                                                                         | 2%  |
| 2  | Internet    | export.arxiv.org                                                                  | <1% |
| 3  | Publication | Jianchuan Zhao, Wenbang Yang, Yong Zhao, Haiwei Sang, Lu Huang, Ziyang Chen,...   | <1% |
| 4  | Internet    | www.arxiv-vanity.com                                                              | <1% |
| 5  | Publication | Zhang Junming, Katherine A. Skinner, Ram Vasudevan, Matthew Johnson-Roberso...    | <1% |
| 6  | Internet    | tr.overleaf.com                                                                   | <1% |
| 7  | Internet    | downloads.hindawi.com                                                             | <1% |
| 8  | Publication | Xuxin Zhang, Kunhong Li, Yongjian Zhang, Zhuo Song, Runqing Jiang, Ye Zhang, Y... | <1% |
| 9  | Internet    | proxy.osapublishing.org                                                           | <1% |
| 10 | Internet    | dspace.bracu.ac.bd                                                                | <1% |

|    |             |                                                                                     |     |
|----|-------------|-------------------------------------------------------------------------------------|-----|
| 11 | Publication | Shuya Chen, Zhiyu Xiang, Peng Xu, Xijun Zhao. "A Normalized Disparity Loss for S... | <1% |
| 12 | Internet    | www.cvlibs.net                                                                      | <1% |
| 13 | Internet    | doaj.org                                                                            | <1% |
| 14 | Internet    | dr.ur.ac.rw                                                                         | <1% |
| 15 | Publication | Chenghao Zhang, Gaofeng Meng, Richard Yi Da Xu, Shiming Xiang, Chunhong Pa...       | <1% |
| 16 | Publication | Wanmin Wu, Ivana Tošić, Kathrin Berkner, Nikhil Balram. "Depth-disparity calibra... | <1% |
| 17 | Internet    | www.springerprofessional.de                                                         | <1% |
| 18 | Internet    | bonndoc.ulb.uni-bonn.de                                                             | <1% |
| 19 | Publication | He Dai, Xuchong Zhang, Yongli Zhao, Hongbin Sun, Nanning Zheng. "Adaptive Dis...    | <1% |
| 20 | Internet    | cvlibs.net                                                                          | <1% |
| 21 | Internet    | gyan.iitg.ac.in                                                                     | <1% |
| 22 | Publication | Ganchao Liu, Sihang Zhang, Qiang Gao, Yuan Yuan. "FSO-VO: Visual Odometry Ba...     | <1% |
| 23 | Publication | Weiqin Chuah, Ruwan Tennakoon, Reza Hoseinnezhad, Alireza Bab-Hadiashar. "D...      | <1% |
| 24 | Internet    | eprints.utar.edu.my                                                                 | <1% |

|    |             |                                                                                       |     |
|----|-------------|---------------------------------------------------------------------------------------|-----|
| 25 | Publication | Haopeng Wang, Zhanhong Chen, Yu Zhang, Yanbiao Sun, Jigui Zhu. "CIRNet: Com...        | <1% |
| 26 | Internet    | repo.undiksha.ac.id                                                                   | <1% |
| 27 | Publication | Liangxun Ou, Yuhui Liu, Zhenyang Li, Xiaoyang Bai, Yifan Peng. "Structure-Groun...    | <1% |
| 28 | Publication | Guang-Zhu Zhang, Qingliang Xu, Hong-Feng Li, Chun-Peng Han, Qiushi Li. "Real-ti...    | <1% |
| 29 | Publication | Junpeng Jing, Jiankun Li, Pengfei Xiong, Jiangyu Liu, Shuaicheng Liu, Yichen Guo, ... | <1% |
| 30 | Internet    | products.climate.ncsu.edu                                                             | <1% |
| 31 | Internet    | webcast.ottawa.ca                                                                     | <1% |
| 32 | Internet    | wiredspace.wits.ac.za                                                                 | <1% |
| 33 | Publication | Iyer, Venkatakrishnan V.. "Integrated Vision-Based UAS and Moving Platform Pos...     | <1% |
| 34 | Publication | Zhong Wu, Hong Zhu, Lili He, Qiang Zhao, Jing Shi, Wenhuan Wu. "Real-time stere...    | <1% |
| 35 | Publication | Zihua Liu, Songyan Zhang, Zhicheng Wang, Masatoshi Okutomi. "Digging Into No...       | <1% |
| 36 | Internet    | www.zemris.fer.hr                                                                     | <1% |
| 37 | Publication | Daud Alam, Uwe Gühl. "Chapter 3 Projektphasen in klassischen Projekten", Sprin...     | <1% |
| 38 | Publication | Mfam Ntonoba Mfam. "Consumer Behavior (The Nigerian Breweries Plc)", Intern...        | <1% |

|    |             |                                                                                       |     |
|----|-------------|---------------------------------------------------------------------------------------|-----|
| 39 | Publication | Zhong Wu, Hong Zhu, Lili He, Jing Shi. "Towards Accurate and Real-Time Binocula...    | <1% |
| 40 | Internet    | dspace.daffodilvarsity.edu.bd:8080                                                    | <1% |
| 41 | Internet    | 123dok.com                                                                            | <1% |
| 42 | Publication | Xiuwen He, Zhengyang Yu, Meng Ouyang, Renxin Liu, Xiaoling Yang. "Design of a ...     | <1% |
| 43 | Internet    | atrium.lib.uoguelph.ca                                                                | <1% |
| 44 | Internet    | lrd.yahooapis.com                                                                     | <1% |
| 45 | Internet    | rps.wku.edu.et                                                                        | <1% |
| 46 | Internet    | www.ajjacobson.us                                                                     | <1% |
| 47 | Internet    | www.bestpfe.com                                                                       | <1% |
| 48 | Publication | Jinhao Huang, Wei Wei, Bifa Liang, Cheng Liu, Wenli Shang, Jun Li. "Real-Time Ster... | <1% |
| 49 | Publication | Yakai Zhang, Jinhui Zhang. "GFANet: Group Fusion Aggregation Network for Real ...     | <1% |
| 50 | Publication | Zeqian Pan, Yan Piao. "Geometry-Enhanced Multi-Level DynamicDisparity Estimatio...    | <1% |
| 51 | Internet    | ir.unisa.ac.za                                                                        | <1% |
| 52 | Publication | Andreas Kuhn, Helmut Mayer, Heiko Hirschmuller, Daniel Scharstein. "A TV Prior f...   | <1% |

|    |             |                                                                                         |     |
|----|-------------|-----------------------------------------------------------------------------------------|-----|
| 53 | Publication | Huanchun Yang, Hongshe Dang, Xuande Zhang, Quanping Chen. "Double Cost-Vo...            | <1% |
| 54 | Publication | Y. V. Venkatesh. "<![CDATA[On the Application of a Modified Self-Organizing Neur...     | <1% |
| 55 | Publication | Ziang Niu, Siqi Liu, Yuan Gao, Ting Huang, Yong He, Zhengjun Qiu. "MaizeStereo: ...     | <1% |
| 56 | Internet    | researchrepository.napier.ac.uk                                                         | <1% |
| 57 | Internet    | www.dpa.gov.eg                                                                          | <1% |
| 58 | Publication | Guo, Xiaoyang. "Towards Learning Data-Efficient and Robust Stereo Matching", T...       | <1% |
| 59 | Publication | Jiaqi Zhu, Bin Li, Xinhua Zhao. "Multi-level pyramid fusion for efficient stereo mat... | <1% |
| 60 | Publication | Marcello Pagano, Kimberlee Gauvreau, Heather Mattie. "Principles of Biostatistic...     | <1% |
| 61 | Publication | Sheng He, Shenhong Li, San Jiang, Wanshou Jiang. "HMSM-Net: Hierarchical multi...       | <1% |
| 62 | Publication | Yuanwei Bi, Chuanbiao Li, Qiang Zheng, Guohui Wang, Shidong Xu, Weiyuan Wan...          | <1% |
| 63 | Internet    | openaccess.thecvf.com                                                                   | <1% |
| 64 | Internet    | www.coursehero.com                                                                      | <1% |
| 65 | Publication | "Medical Image Computing and Computer Assisted Intervention – MICCAI 2022", ...         | <1% |
| 66 | Publication | Bin Wen, Han Zhu, Chao Yang, Zhicong Li, Renxuan Cao. "Feature back-projection ...      | <1% |

|    |             |                                                                                       |     |
|----|-------------|---------------------------------------------------------------------------------------|-----|
| 67 | Publication | Chuhua Wang, Md Alimoor Reza, Vibhas Vats, Yingnan Ju et al. "Deep learning-ba...     | <1% |
| 68 | Publication | Ferede, Fisseha Admasu. "Understanding Higher-Order Motion Terms in Image S...        | <1% |
| 69 | Publication | Haofei Xu, Juyong Zhang. "AANet: Adaptive Aggregation Network for Efficient Ste...    | <1% |
| 70 | Publication | Jin Liu, Jian Gao, Shunping Ji, Chang Zeng, Shaoyi Zhang, JianYa Gong. "Deep learn... | <1% |
| 71 | Publication | Lecture Notes in Computer Science, 2015.                                              | <1% |
| 72 | Publication | S. L. Gupta, Nawal Kishor, Mishra Niraj, Sonali Mathur, Utkarsh Gupta. "Digitaliza... | <1% |
| 73 | Internet    | assets-eu.researchsquare.com                                                          | <1% |
| 74 | Internet    | eprints.usq.edu.au                                                                    | <1% |
| 75 | Internet    | github.com                                                                            | <1% |
| 76 | Internet    | grossarchive.com                                                                      | <1% |
| 77 | Internet    | offseo.com                                                                            | <1% |
| 78 | Internet    | qspace.library.queensu.ca                                                             | <1% |
| 79 | Publication | "Computer Vision – ECCV 2016", Springer Nature, 2016                                  | <1% |
| 80 | Publication | "Computer Vision – ECCV 2016", Springer Science and Business Media LLC, 2016          | <1% |

|    |             |                                                                                       |     |
|----|-------------|---------------------------------------------------------------------------------------|-----|
| 81 | Publication | Changjiang Cai, Matteo Poggi, Stefano Mattoccia, Philippos Mordohai. "Matching-...    | <1% |
| 82 | Publication | Cheng Liu, Wei Wei, Bifa Liang, Xihao Liu, Wenli Shang, Jun Li. "ConvMLP-Mixer Ba...  | <1% |
| 83 | Publication | Hosein Hashemi, Yasser Baleghi, Mohamad Reza Hassanzadeh. "Real-time stereo ...       | <1% |
| 84 | Publication | Huang, Chao. "Controllable Generative Modeling for Multimodal Perception and ...      | <1% |
| 85 | Publication | Jiaqi Leng, Qingxuan Lv, Shu Zhang, Yuan Rao, Yimei Liu, Hao Fan. "Multilevel Inv...  | <1% |
| 86 | Publication | Junren Sun, Hao Xue, Shibo Guo, Xunqi Zheng. "Fisheye omnidirectional stereo de...    | <1% |
| 87 | Publication | Junwei Wang, Wei Zhou, Yujun Tang, Hanming Guo. "Matchability and Uncertaint...       | <1% |
| 88 | Publication | Kai Zeng, Hui Zhang, Wei Wang, Yaonan Wang, Jianxu Mao. "Deep Stereo Networ...        | <1% |
| 89 | Publication | Nalpantidis, L.. "Biologically and psychophysically inspired adaptive support weig... | <1% |
| 90 | Publication | Vlad-Cristian Miclea, Liviu Miclea, Sergiu Nedevschi. "Real-time Stereo Reconstruc... | <1% |
| 91 | Publication | Xiaowei Yang, Yong Zhao, Zhiguo Feng, Haiwei Sang, Zhenbo Zhang, Guiying Zha...       | <1% |
| 92 | Publication | Yeqi Hu, Yuan Rao, Hui Yu, Gaige Wang, Hao Fan, Wei Pang, Junyu Dong. "Out-of-...     | <1% |
| 93 | Publication | Yulan Guo, Yun Wang, Longguang Wang, Zi Wang, Chen Cheng. "CVCNet: Learnin...         | <1% |
| 94 | Internet    | docslib.org                                                                           | <1% |

|     |             |                                                                                    |     |
|-----|-------------|------------------------------------------------------------------------------------|-----|
| 95  | Internet    | eece.mist.ac.bd                                                                    | <1% |
| 96  | Internet    | elib.dlr.de                                                                        | <1% |
| 97  | Internet    | era.ed.ac.uk                                                                       | <1% |
| 98  | Internet    | link.springer.com                                                                  | <1% |
| 99  | Internet    | qspace.qu.edu.qa                                                                   | <1% |
| 100 | Internet    | rcastoragev2.blob.core.windows.net                                                 | <1% |
| 101 | Internet    | repository.up.ac.za                                                                | <1% |
| 102 | Internet    | research.library.mun.ca                                                            | <1% |
| 103 | Internet    | umpir.ump.edu.my                                                                   | <1% |
| 104 | Internet    | wlv.openrepository.com                                                             | <1% |
| 105 | Internet    | www.mdpi.com                                                                       | <1% |
| 106 | Publication | "Computer Vision – ACCV 2020", Springer Science and Business Media LLC, 2021       | <1% |
| 107 | Publication | Anindya Nag, Mehedi Hassan, Riya Sil, Asif Karim. "Transforming Global Healthca... | <1% |
| 108 | Publication | Izzun Nafis Ibadik, Egi Muhammad Idris Hidayat. "EFSNet: Lightweight stereo ma...  | <1% |

109

Publication

Scott Krig. "Computer Vision Metrics", Springer Science and Business Media LLC, ... <1%

---

110

Publication

WeiQin Chuah, Ruwan Tennakoon, Reza Hoseinnezhad, David Suter, Alireza Bab-... <1%

---

111

Publication

Zhongjian Lu, An Chen, Hongxia Gao, Langwen Zhang, Congyu Zhang, Yang Yang.... <1%

---

112

Publication

"The Impact of the 4th Industrial Revolution on Engineering Education", Springer ... <1%

---

113

Publication

Bifa Liang, Hong Yang, Jinhao Huang, Cheng Liu, Ru Yang. "Real-time Stereo Matc... <1%

---

114

Publication

He Yang Sun, Wenchuan Zhao, ZhongFeng Guo. "High-precision stereo matching ... <1%

## **Appendix H**

### **AI Writing Report**

The AI writing report is attached at the end of this document.

## \*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

**Caution: Review required.**

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

### Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.